데이터 분석을 위한

Matplotlib

류영표 강사
ryp1662@gmail.com

Copyright ⓒ “Youngpyo Ryu” All Rights Reserved.
This document was created for the exclusive use of “Youngpyo Ryu”.
It must not be passed on to third parties except with the explicit prior consent of “Youngpyo Ryu”.

강사 소개



류영표

Youngpyo Ryu

동국대학교 수학과/응용수학 석사수료

前 Upstage AI X 네이버 부스트 캠프 AI tech 1~ 6기 멘토

前 Innovation on Quantum & CT(IQCT) 이사

前 한국파스퇴르연구소 Image Mining 인턴(Deep learning)

前 ㈜셈웨어(수학컨텐츠, 데이터 분석 개발 및 연구인턴)

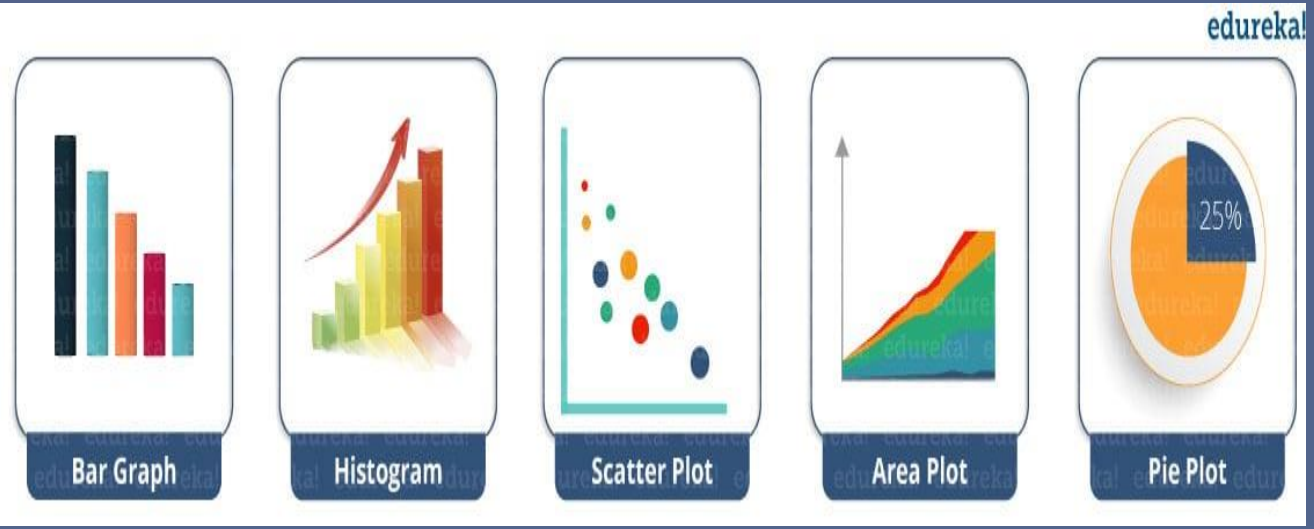
강의 경력

- 현대자동차 연구원 강의 (인공지능/머신러닝/딥러닝/강화학습)
- ㈜ 모두의연구소 Aiffel 1기 퍼실리테이터(인공지능 교육)
- 인공지능 자연어처리(NLP) 기업데이터 분석 전문가 양성과정 멘토
- 공공데이터 청년 인턴 / SW공개개발자대회 멘토
- 고려대학교 선도대학 소속 30명 딥러닝 집중 강의
- 이젠 종로 아카데미(파이썬, ADSP 강사)
- 최적화된 도구(R/파이썬)을 활용한 애널리스트 양성과정(국비과정) 강사
- 한화, 하나금융사, 한전 KDN, IBK 기업은행 교육
- 인공지능 신뢰성 확보를 위한 실무 전문가 자문 활동
- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문 활동
- 보건 · 바이오 AI활용 S/W개발 및 응용전문가 양성과정 강사
- Upstage AI X KT 융합기술원 기업교육 모델최적화 담당 조교

주요 프로젝트 및 기타사항

- 의료 영상을 이용한 갑상선암 연관 유전변이 예측 플랫폼 개발 고도화
- 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 개발 및 고도화(안정화)
- 폐플라스틱 이미지 객체 검출 경진대회 3위
- 인공지능(AI)기반 데이터 사이언티스트 전문가 양성과정 1기 수료
- 제 1회 산업 수학 스터디 그룹 (질병에 영향을 미치는 유전자 정보 분석)
- 제 4,5회 산업 수학 스터디 그룹 (피부암, 유방암 분류)

INDEX



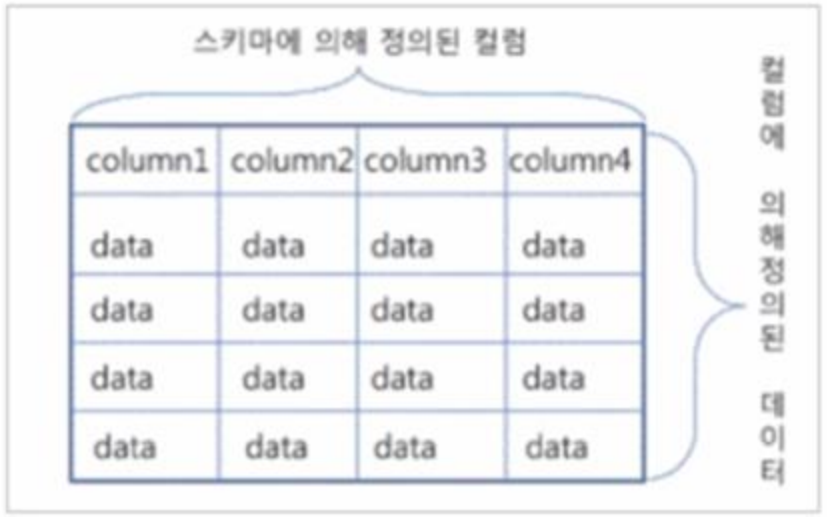
1. Matplotlib
2. Seaborn
3. folium
4. plotnie
5. Plotly
6. Bokeh

출처 : https://www.edureka.co/blog/python-matplotlib-tutorial/

시각화

- 데이터를 시각화를 위해서는 데이터가 우선적으로 필요
- 수 많은 데이터 셋이 존재함.
    - 정형/비정형 데이터
    - 시계열 데이터
    - 지리 데이터
    - 관계형(네트워크) 데이터 등

정형데이터 구조와 예시

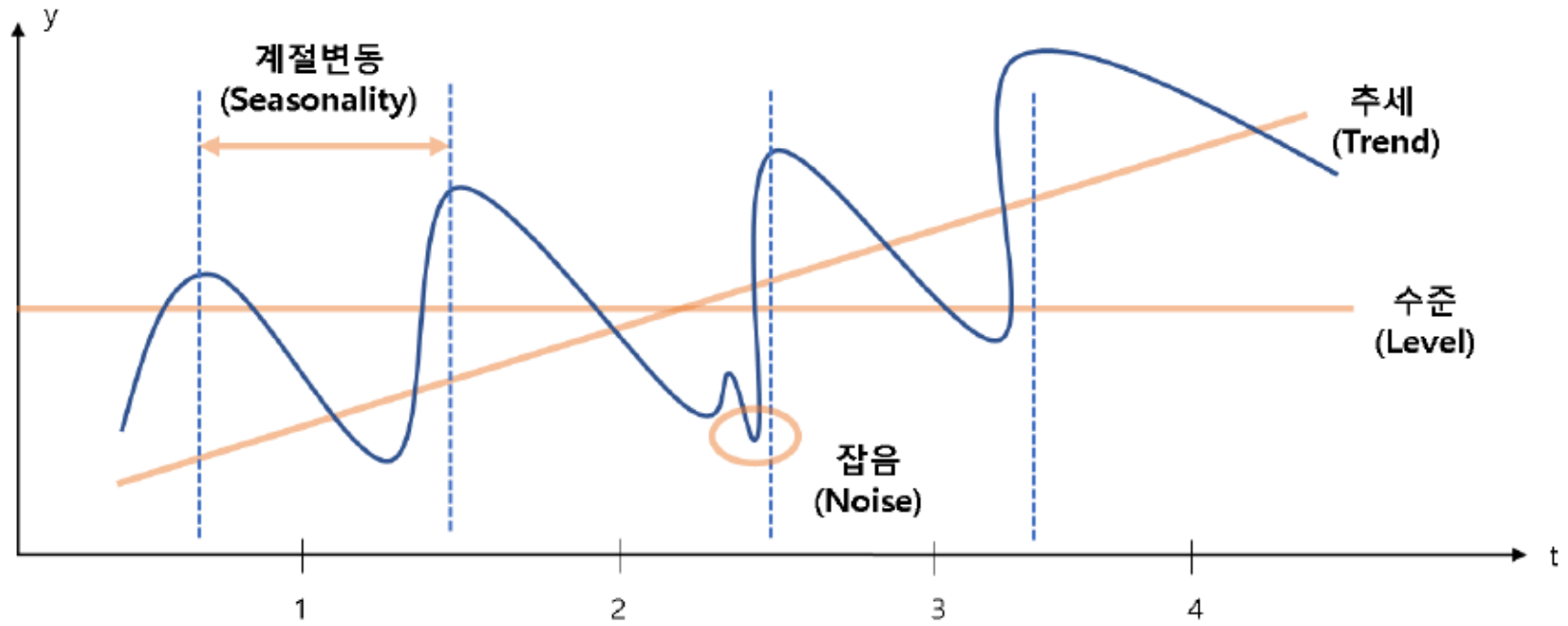


[그림1] 정형데이터 구조
Source : www.dbguide.net

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa

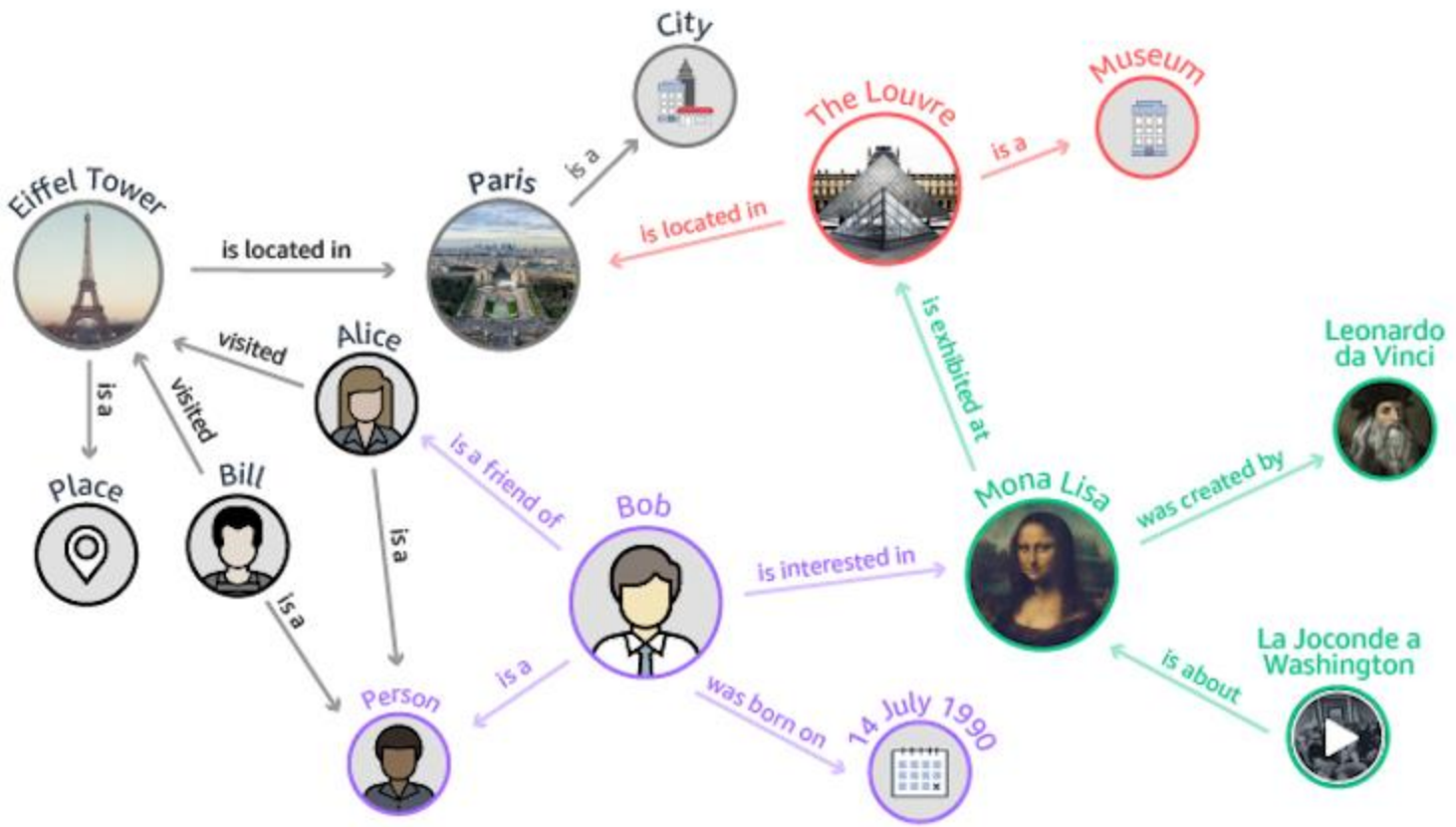
[그림2] 정형데이터 예시
Source : www.dbguide.net

데이터종류_시계열 데이터



시계열 요소	핵심요소	설명
수준 (Level)	시계열의 평균값	- 시계열 데이터의 전체적 평균
추세 (Trend)	전반적인 패턴 변화	- 어떤 현상이 일정한 방향으로 증가/하락하는 경향 - 선형, 2차 함수, 지수 성장, S-곡선 등의 유형 존재
계절변동 (Seasonality)	짧은 기간 동안의 주기적인 패턴	- 특정 기간 동안 유사한 형태의 관측치가 반복되는 구간
잡음 (Noise)	무작위적 변동	- 일반적으로 알 수 없는 이유로 발생하는 무작위적 관측치

시각화_관계 데이터



출처 : https://www.kaggle.com/ferdzso/knowledge-graph-analysis-with-node2vec

Matplotlib

- Matplotlib이란?
  - 데이터를 시각적으로 표현하기 위한 가장 기본적이고 널리 사용되는 라이브러리
  - seaborn, plotnine, Plotly 등 더 다양한 기능을 지원하는 패키지가 많음
  - Python과 Numpy만 있으면 어느 플랫폼에서도 동작.
- 시작하기 앞서

```
import matplotlib.pyplot as plt
```

→ 네이밍 컨벤션

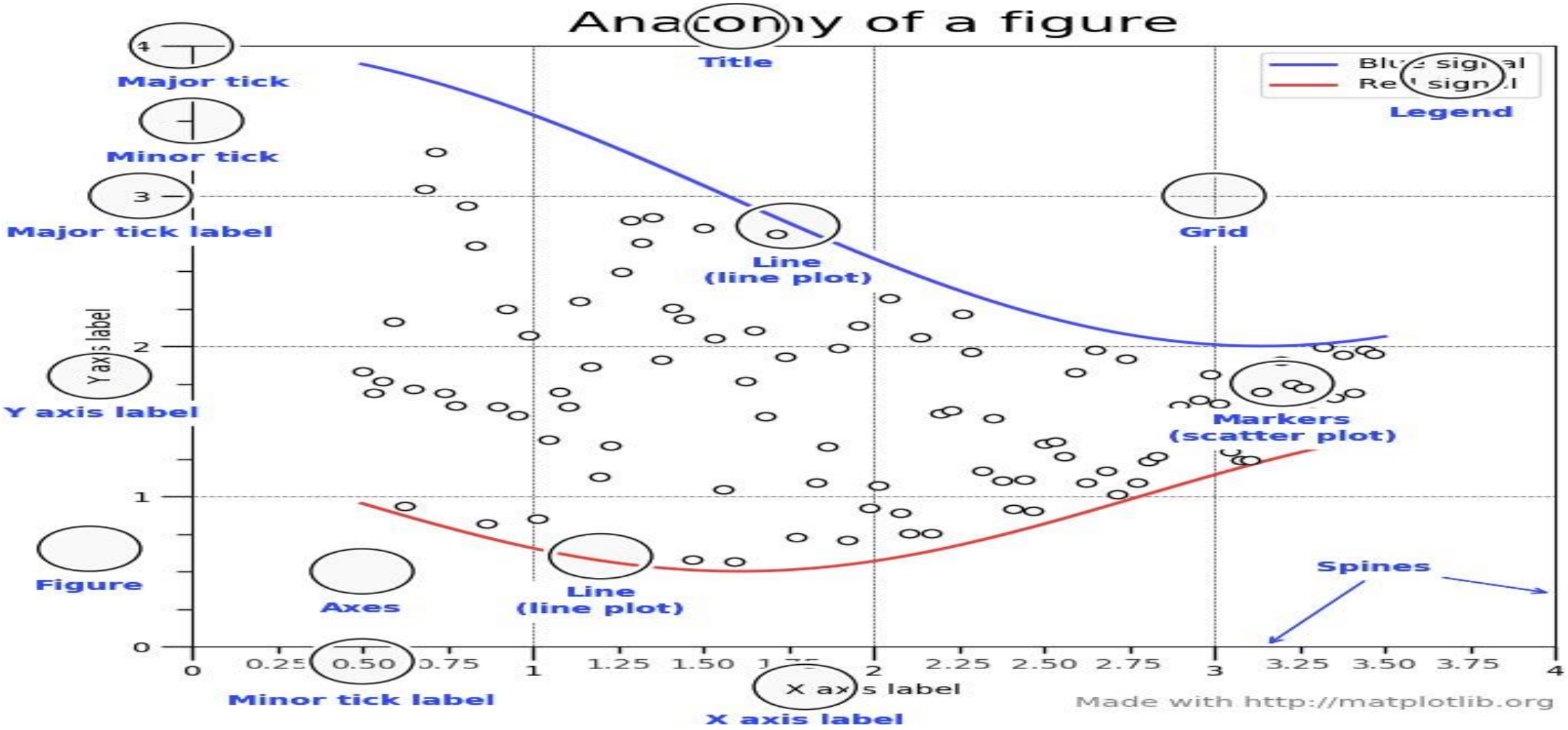
```
[ ] %matplotlib notebook
```

→ 쥬피터 노트북 사용시 대화형 시각화 기능 을 사용하기 위해선 코드를 반드시 실행

```
%matplotlib qt
```

->별도의 팝업창에 그래프 출력(colab X)

Matplotlib

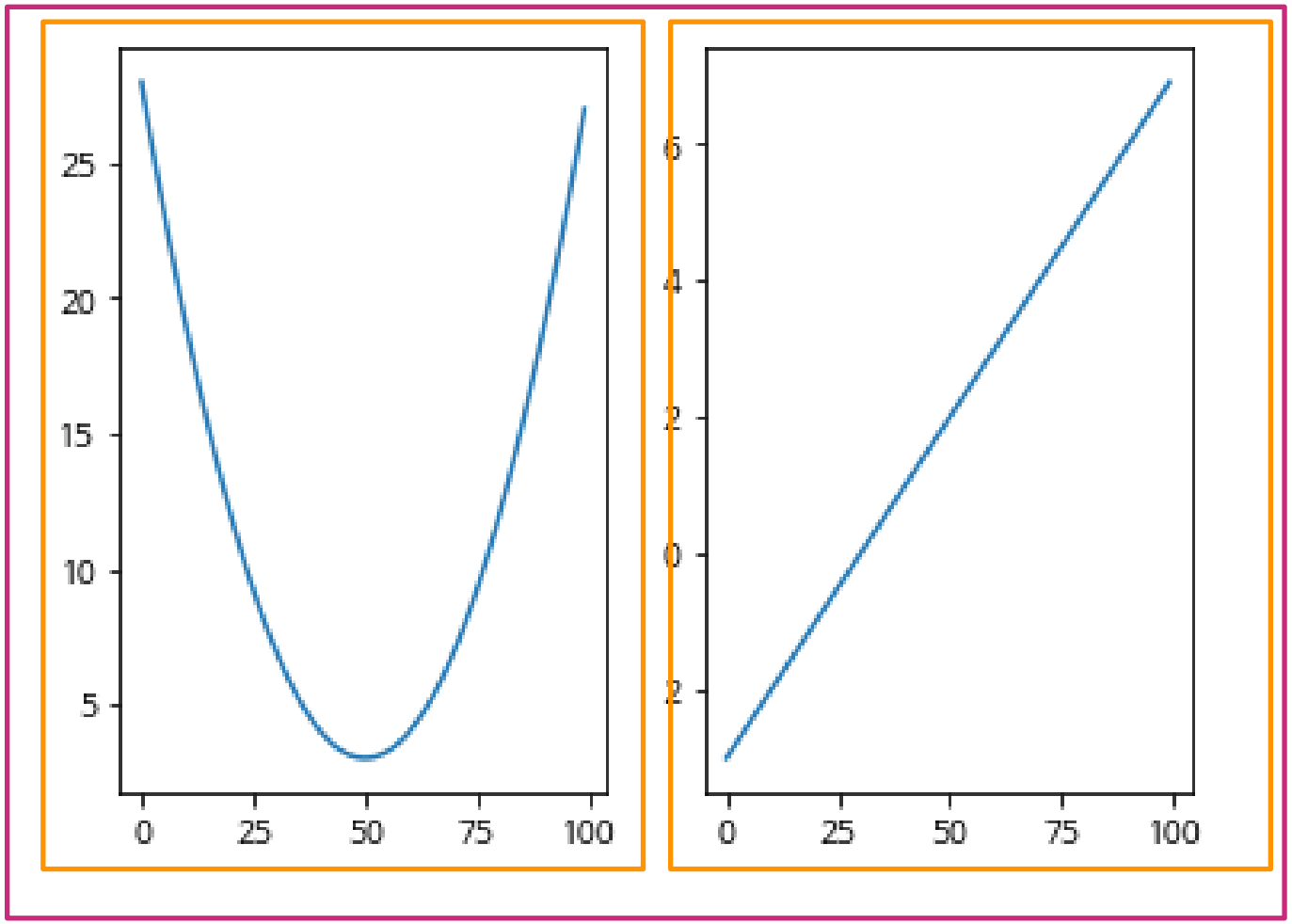


https://matplotlib.org/3.1.3/gallery/showcase/anatomy.html

Figure & Axes

Axes

Figure



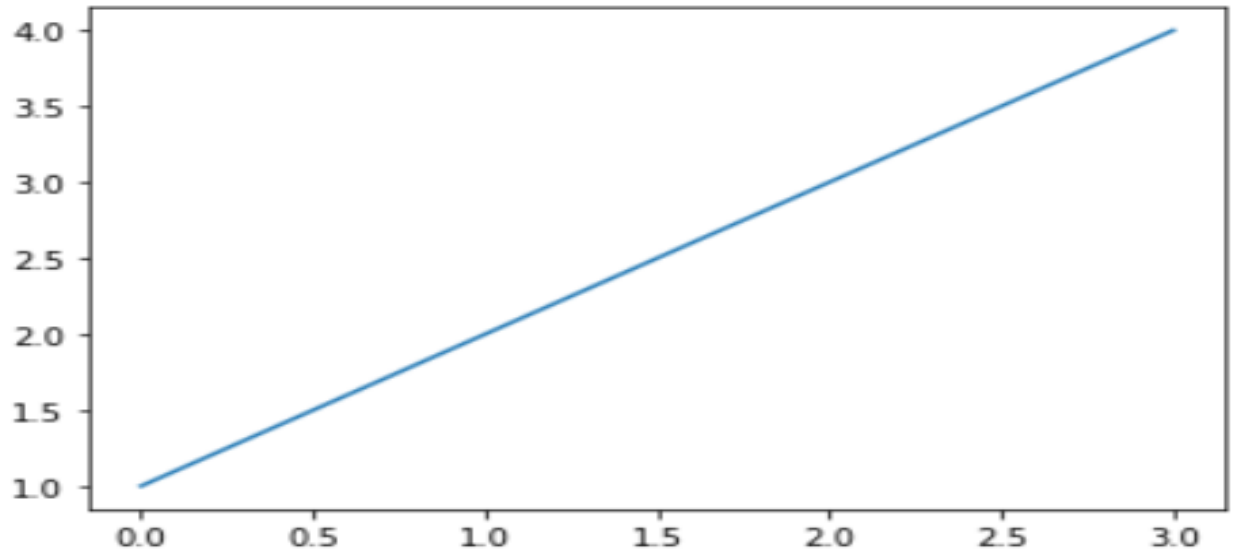
그래프 그리기

```
plot([x], y, [fmt], *, data=None, **kwargs)
plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)
```

[x],[fmt]은 생략가능

```
import matplotlib.pyplot as plt

plt.plot([1,2,3,4])
plt.show()
```



Matplotlib

선 그래프 그리기

```
import matplotlib.pyplot as plt

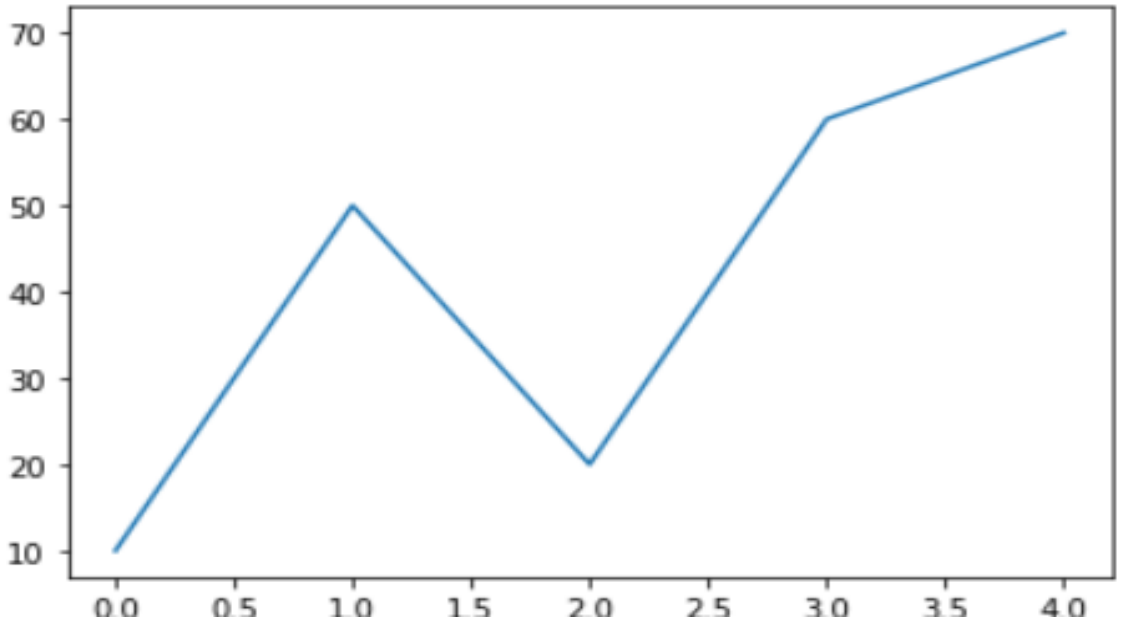
data = [10,50,20,60,70]

%matplotlib inline

plt.plot(data)
```

```
[<matplotlib.lines.Line2D at 0x2052afef308>]
```

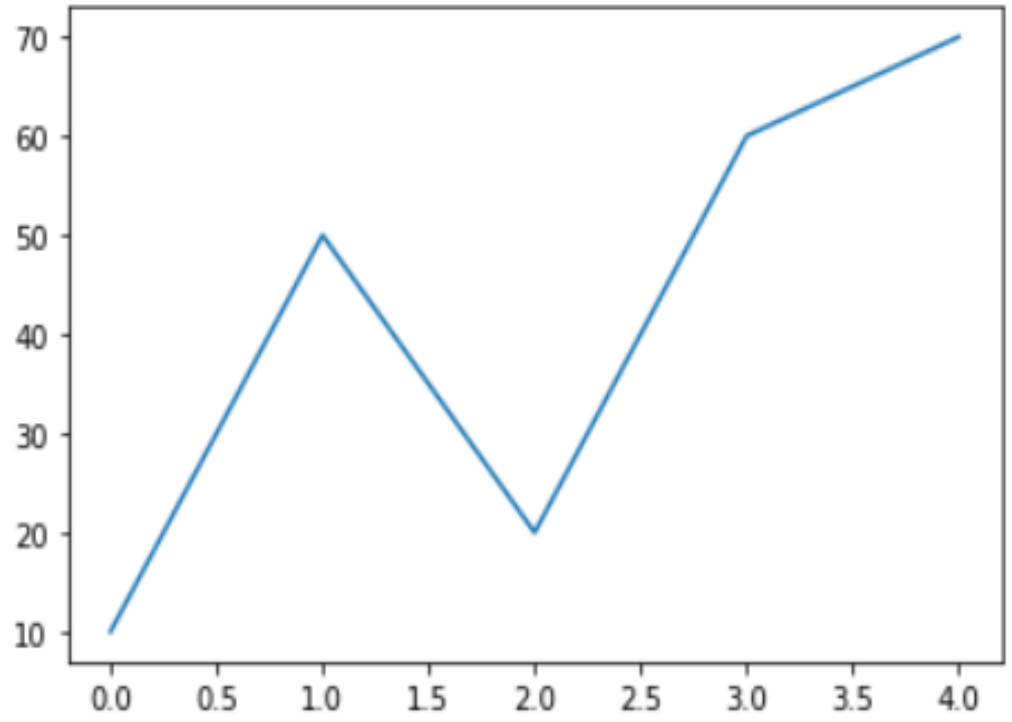
<- 그래프 객체 정보



선 그래프 그리기

```
plt.plot(data)
# 객체 정보없이 그래프만 출력

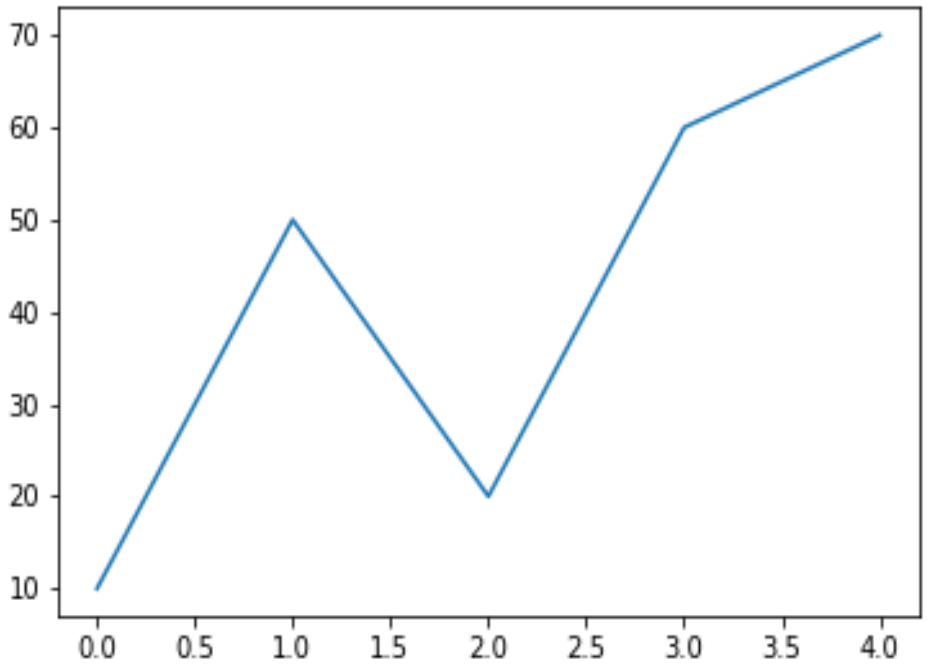
plt.show()
```




```
%matplotlib qt

plt.plot(data)
```

```
[<matplotlib.lines.Line2D at 0x2052ffa3248>]
```



Matplotlib의 Formating의 종류

Colors

character	color
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

Line Styles

character	description
'-'	solid line style
'--'	dashed line style
'-.'	dash-dot line style
':'	dotted line style

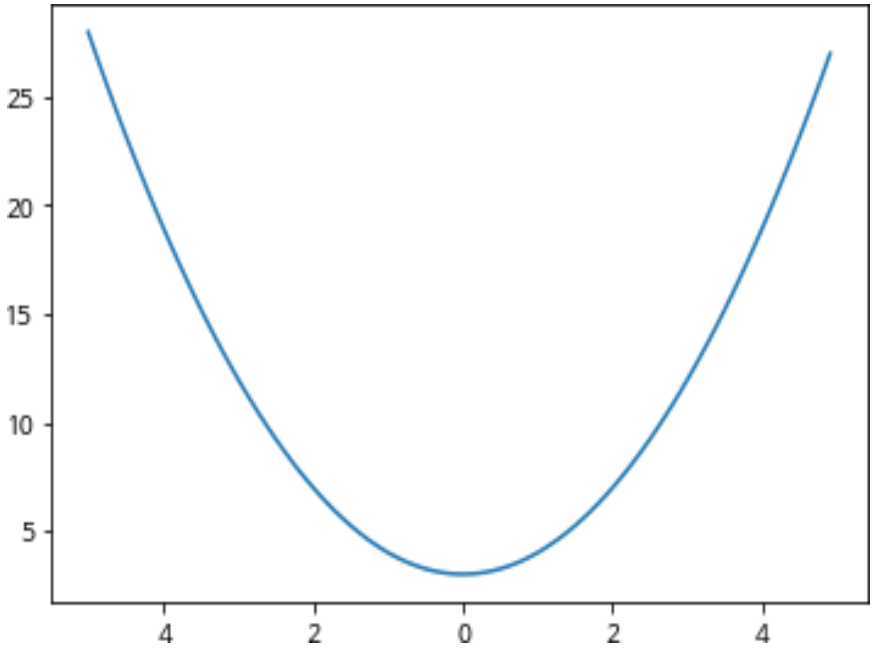
Markers

character	description
'.'	point marker
','	pixel marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker
'>'	triangle_right marker
'1'	tri_down marker
'2'	tri_up marker
'3'	tri_left marker
'4'	tri_right marker
's'	square marker
'p'	pentagon marker
'*'	star marker
'h'	hexagon1 marker
'H'	hexagon2 marker
'+'	plus marker
'x'	x marker
'D'	diamond marker
'd'	thin_diamond marker
'|'	vline marker
'_'	hline marker

Figure & Axes

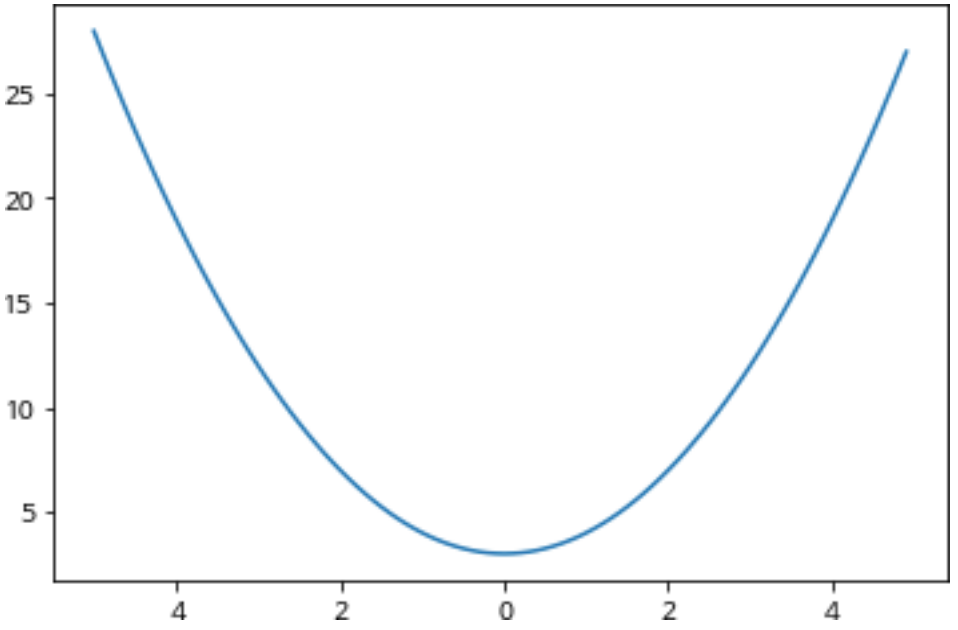
- The pyplot API

```
plt.plot(x, y1)
```

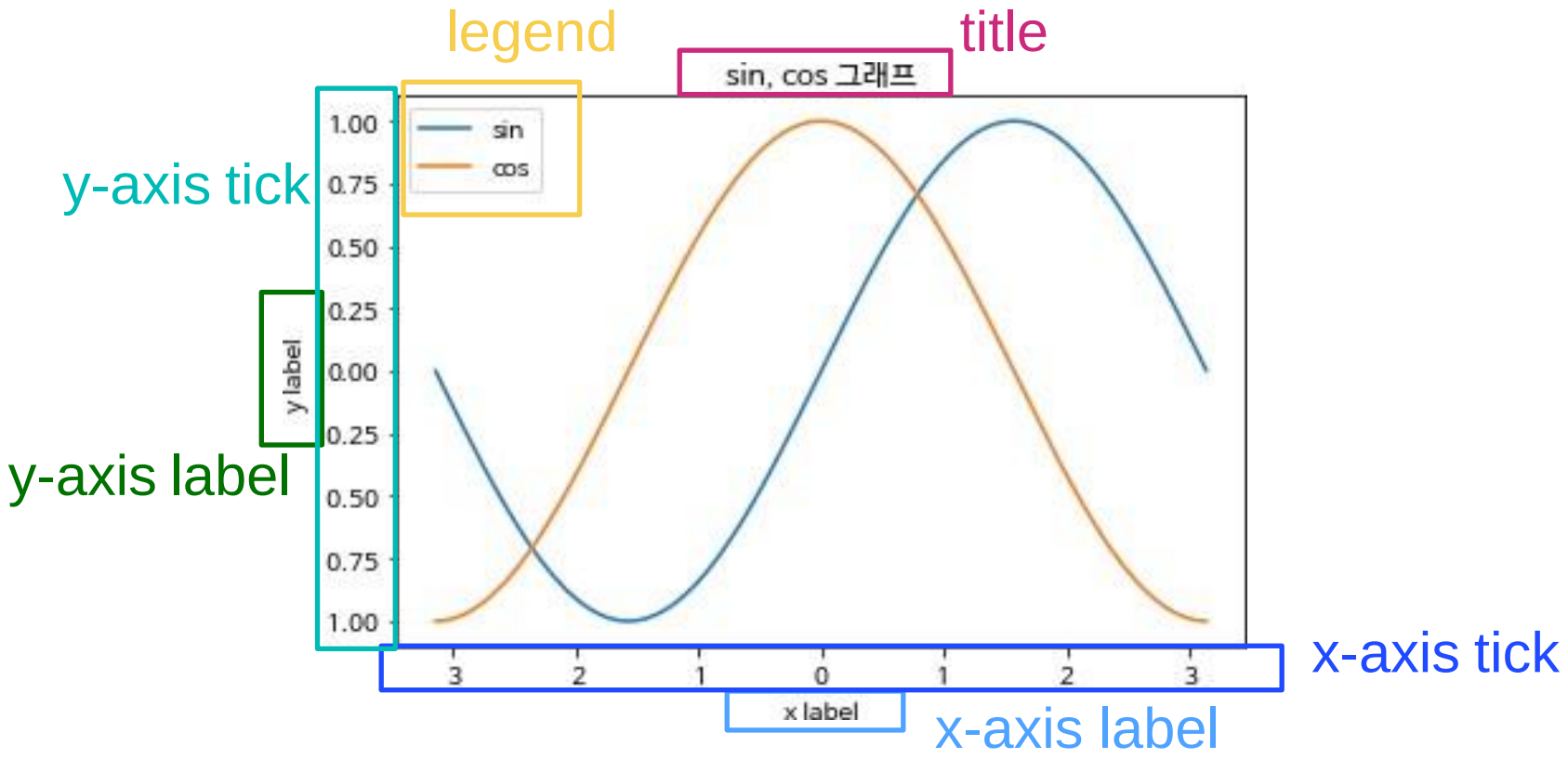


- The object-oriented API

```
fig, ax = plt.subplots()
ax.plot(x, y1)
```



Axis


legend
title
sin, cos 그래프
y-axis tick
y label
y-axis label
sin
cos
1.00
0.75
0.50
0.25
0.00
0.25
0.50
0.75
1.00
3
2
1
0
1
2
3
x label
x-axis tick
x-axis label

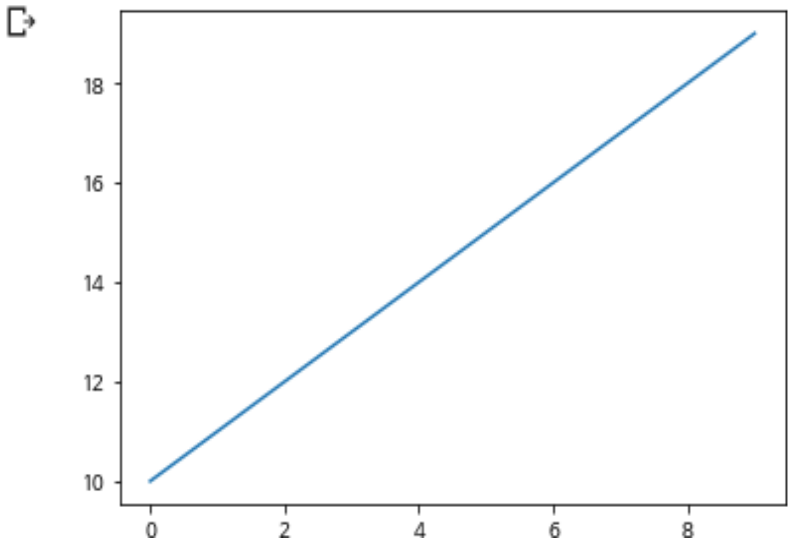
Axis

- xlim, yli

```
[15] x = np.arange(10)
     y = x+10

     plt.plot(x, y)

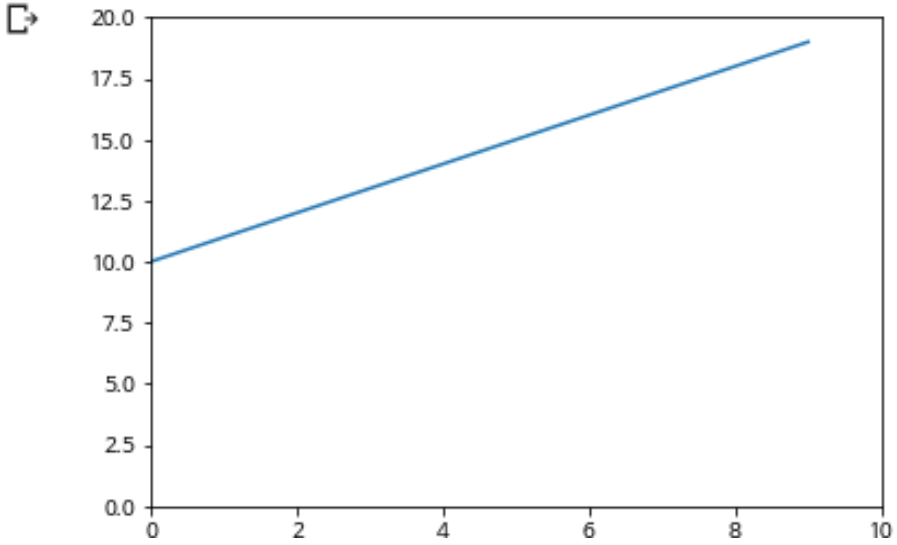
     plt.show()
```




```
plt.xlim([0, 10])
plt.ylim([0, 20])

plt.plot(x, y)

plt.show()
```



Axis

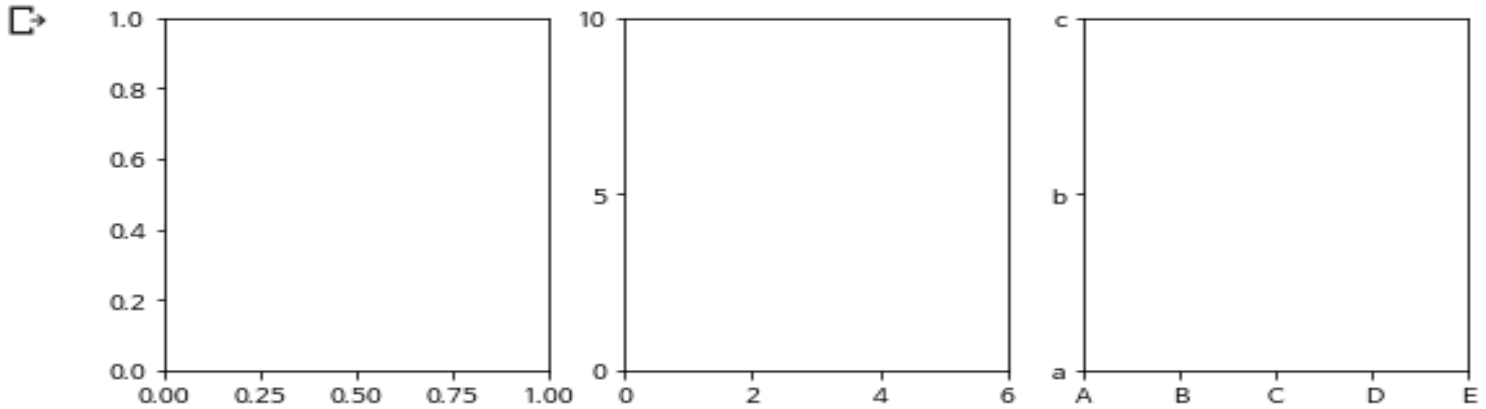
- ticks, tick labels

```
fig, axs = plt.subplots(1, 3, figsize=(9, 3))

axs[1].set_xticks([0,2,4,6])
axs[1].set_yticks([0,5,10])

axs[2].set_xticklabels(['A', 'B', 'C', 'D', 'E'])
axs[2].set_yticks([0,1,2])
axs[2].set_yticklabels(['a', 'b', 'c'])

plt.show()
```



Axis

- 한글 표시

```
import matplotlib

matplotlib.rcParams['font.family'] = 'Malgun Gothic' #맑은 고딕으로 폰트 설정
matplotlib.rcParams['axes.unicode_minus'] = False   #마이너스(-) 폰트 깨짐 방지
```

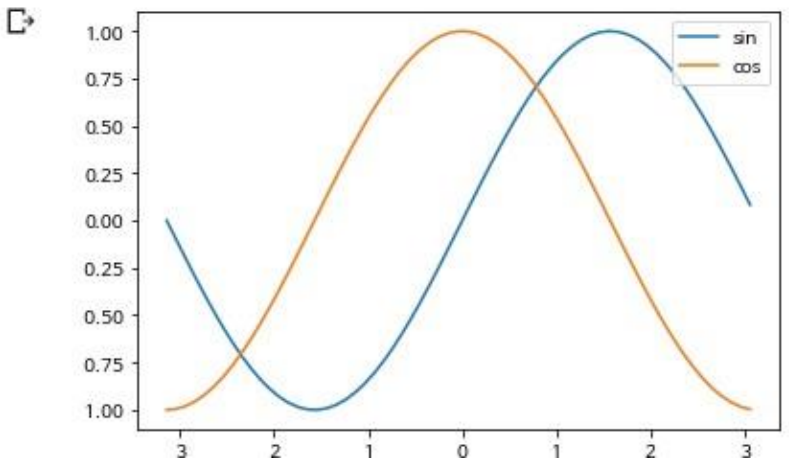
Legend

```
fig, ax = plt.subplots()

ax.plot(x, y1, label = 'sin')
ax.plot(x, y2, label = 'cos')

ax.legend(loc=1)

plt.show()
```




```
import matplotlib.pyplot as plt
import numpy as np

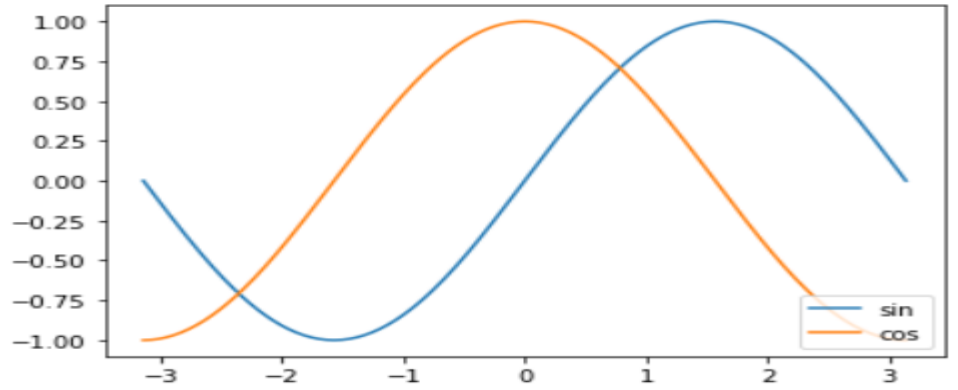
x = np.linspace(-np.pi, np.pi, 201)
y1 = np.sin(x)
y2 = np.cos(x)

fig,ax = plt.subplots()

ax.plot(x, y1, label = 'sin')
ax.plot(x, y2, label = 'cos')

ax.legend(loc='lower right')

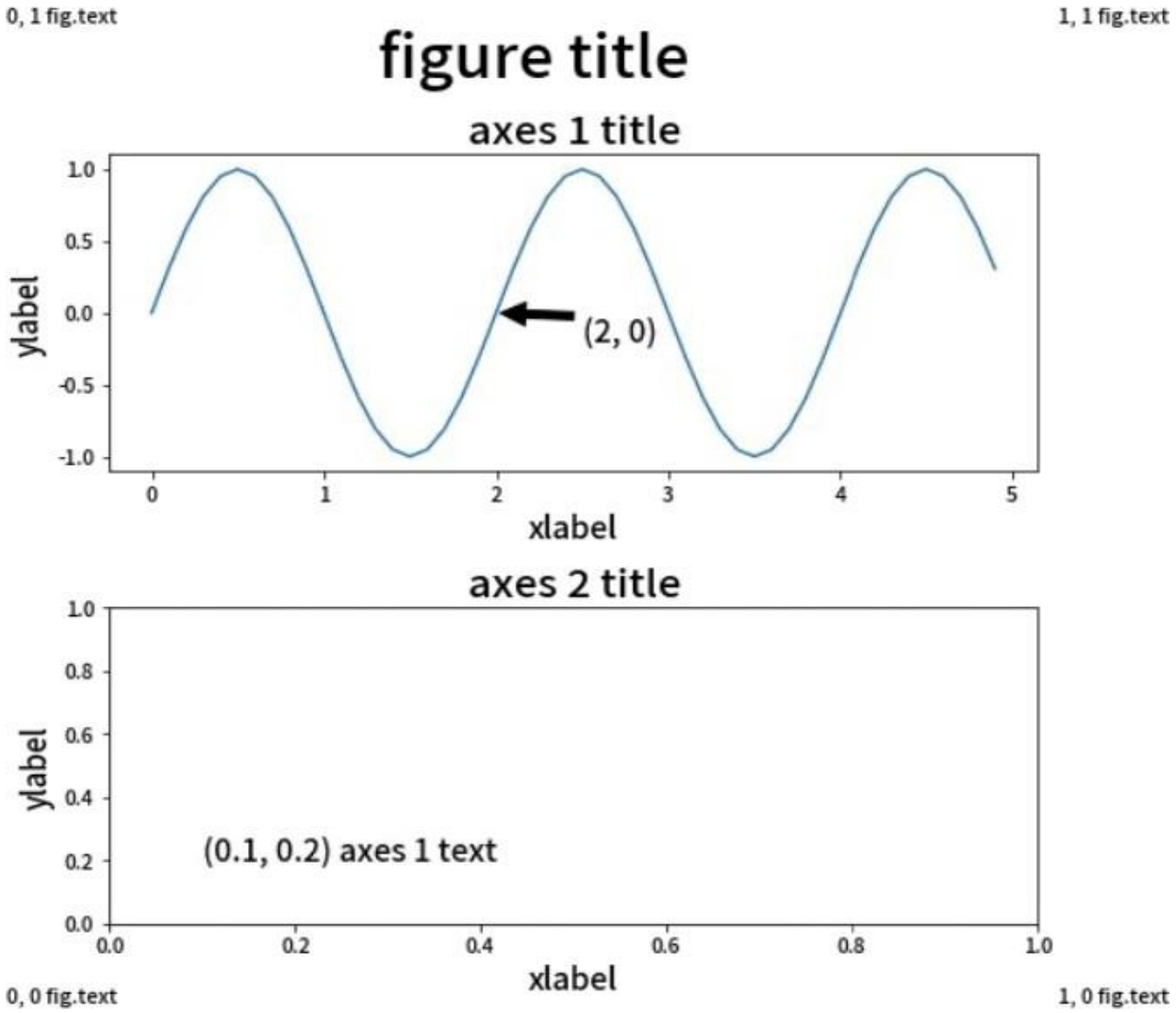

plt.show()
```



참고 :
https://matplotlib.org/3.1.1/api/_as_gen/matplotlib.pyplot.legend.html

Text

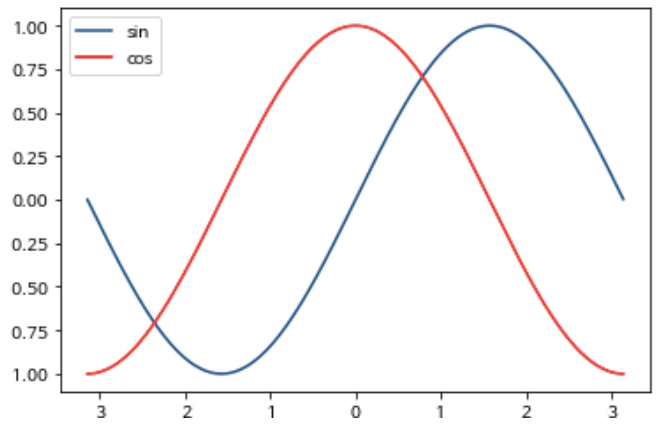
- figure title
- axes title
- x-axis label
- y-axis label
- figure text
- axes text
- annotate



Color

```
x = np.arange(-np.pi, np.pi, 0.02)
y1 = np.sin(x)
y2 = np.cos(x)

plt.plot(x, y1, label = 'sin', color= (0.1, 0.3, 0.5))
plt.plot(x, y2, label = 'cos', color='r')
plt.legend()
plt.show()
```



- color 인자를 명시하여 색을 설정 할 수 있음

1) matplotlib에서 미리 정해둔 값으 로 설정

{'b', 'g', 'r', 'c', 'm', 'y', 'k', 'w'}

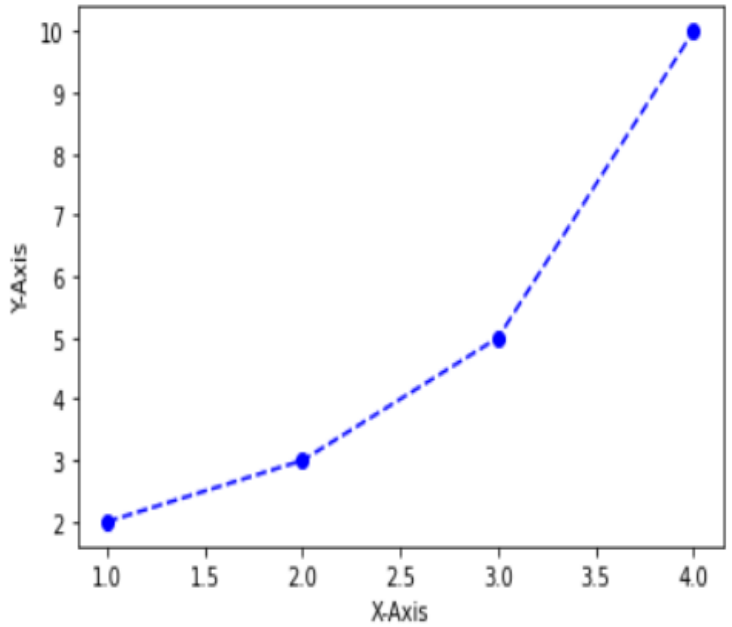
2)RGB 값을 튜플로 묶어서 주기

3)색상팔레트의 값을 string으로 주기

Markers

```
import matplotlib.pyplot as plt

# plt.plot([1, 2, 3, 4], [2, 3, 5, 10], 'bo-')    # 파란색 + 실선 + 마커
plt.plot([1, 2, 3, 4], [2, 3, 5, 10], 'bo--')     # 파란색 + 점선 + 마커
plt.xlabel('X-Axis')
plt.ylabel('Y-Axis')
plt.show()
```



- Marker 인자를 명시하여 색을 설정 할 수 있음

Colors

character	color
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

Line Styles

character	description
'-'	solid line style
'--'	dashed line style
'-.'	dash-dot line style
':'	dotted line style

Markers

character	description
'.'	point marker
','	pixel marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker
'>'	triangle_right marker
'1'	tri_down marker
'2'	tri_up marker
'3'	tri_left marker
'4'	tri_right marker
's'	square marker
'p'	pentagon marker
'*'	star marker
'h'	hexagon1 marker
'H'	hexagon2 marker
'+'	plus marker
'x'	x marker
'D'	diamond marker
'd'	thin_diamond marker
'|'	vline marker
'_'	hline marker

Face Color

- 바탕색

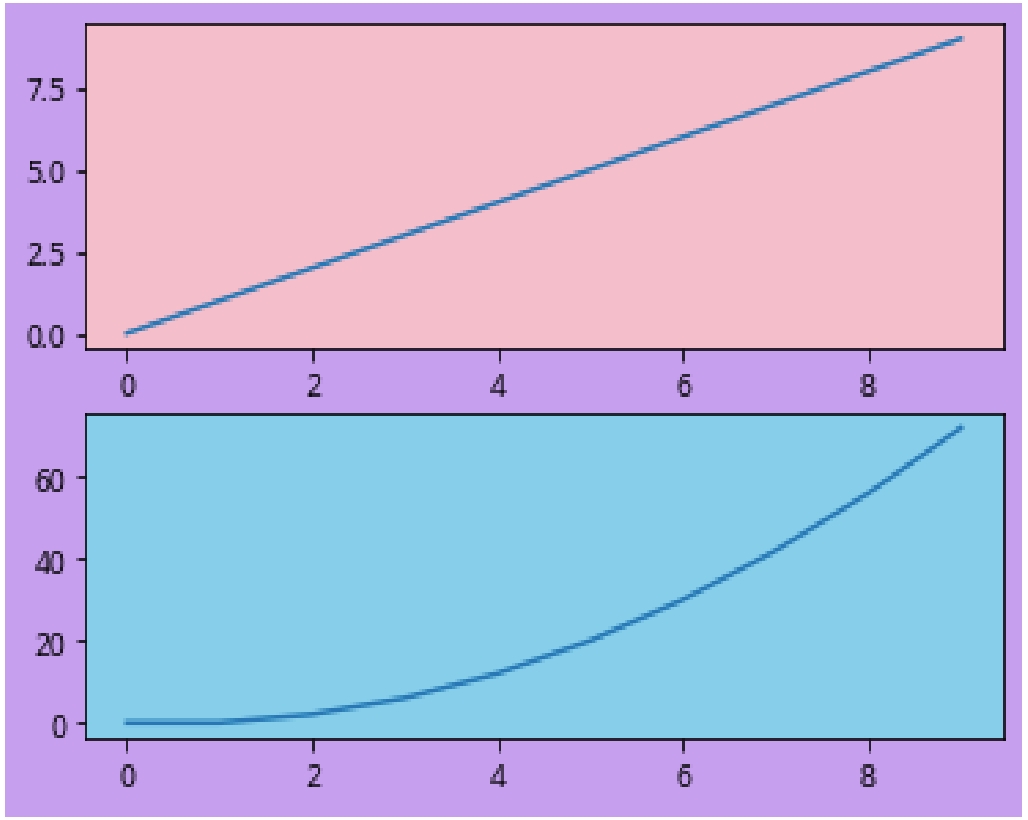
```
x = np.arange(10)
y1 = x
y2 = x**2 - x

fig, axs = plt.subplots(2, 1)
fig.set_facecolor('#c79fef')

axs[0].plot(x, y1)
axs[1].plot(x, y2)

axs[0].set_facecolor('pink')
axs[1].set_facecolor('skyblue')

plt.show()
```



savefig

- plot을 image로 저장

```
fig, ax = plt.subplots()
x = np.arange(10)
y1 = x**2
ax.plot(x, y1, label = 'sin')
fig.savefig('image_matplot_tmp.jpg')
```

Line Plot

- plot( )

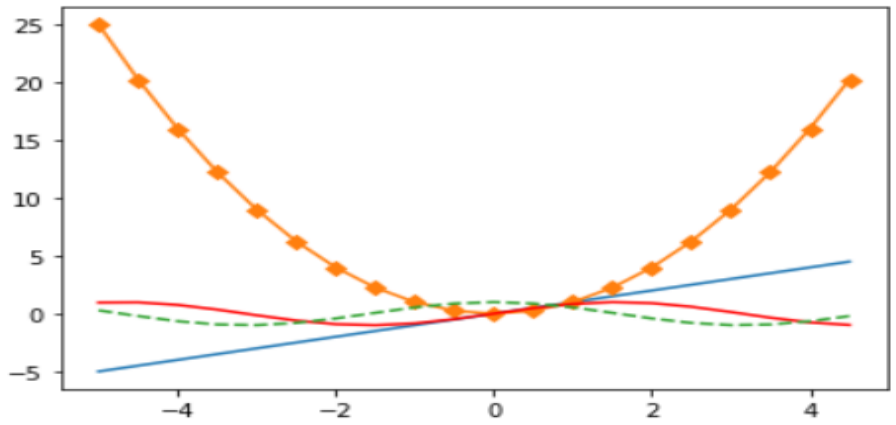
```
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline

x = np.arange(-5, 5, 0.5)
y1 = x
y2 = x **2
y3 = np.sin(x)
y4 = np.cos(x)

plt.plot(x,y1)
plt.plot(x,y2, marker = 'D')
plt.plot(x,y3, color ='r')
plt.plot(x,y4, linestyle = 'dashed')
plt.show
```

```
<function matplotlib.pyplot.show(*args, **kw)>
```



Line Plot

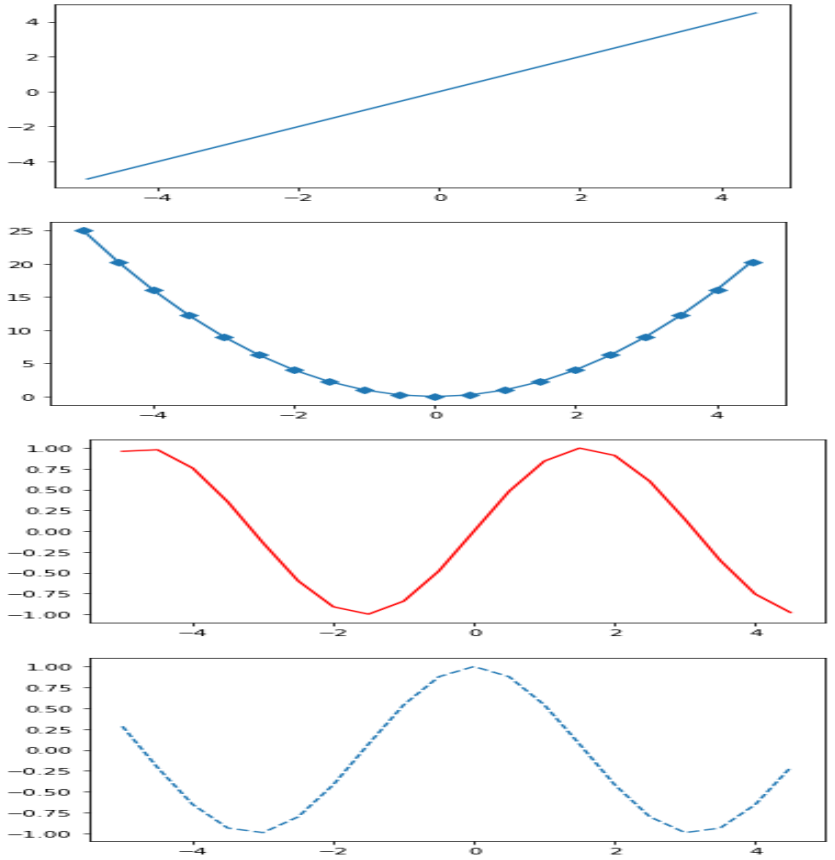
- plot( )

```
plt.plot(x,y1)

plt.figure()
plt.plot(x,y2, marker = 'D')

plt.figure()
plt.plot(x,y3, color ='r')

plt.figure()
plt.plot(x,y4, linestyle = 'dashed')
plt.show()
```



참고 :
https://matplotlib.org/3.1.1/api/_as_gen/matplotlib.lines.Line2D.html#matplotlib.lines.Line2D.set_marker

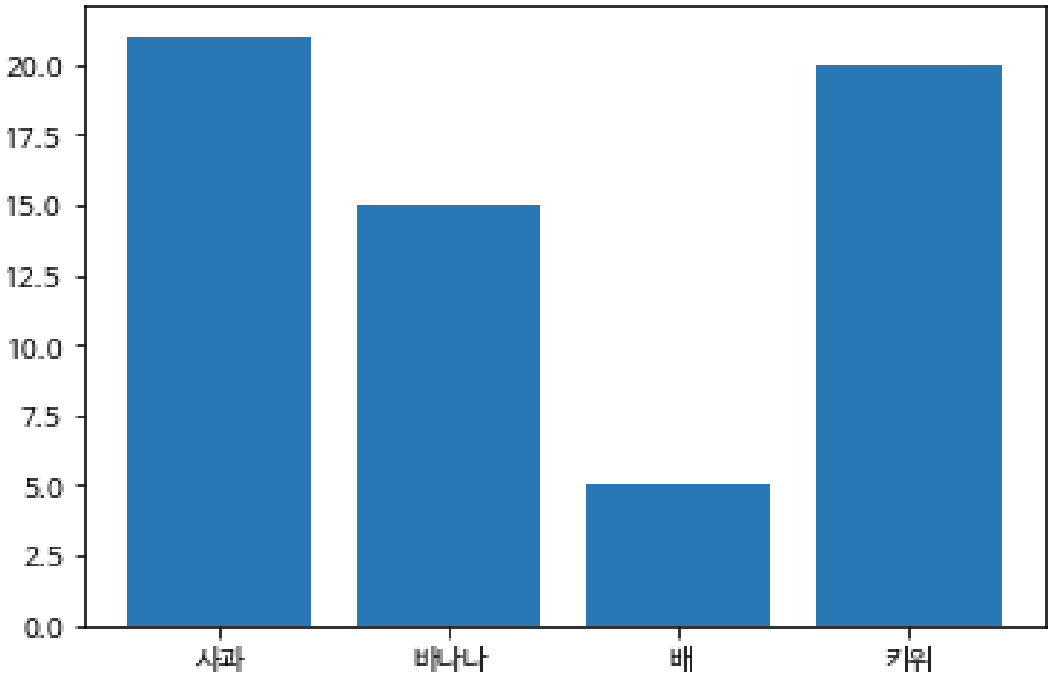
Bar Plot

- bar( )

```
data = {'사과': 21, '바나나': 15, '배': 5, '키위': 20}
names = list(data.keys())
values = list(data.values())

fig, ax = plt.subplots()
ax.bar(names, values)
```

```
<BarContainer object of 4 artists>
```



Matplotlib

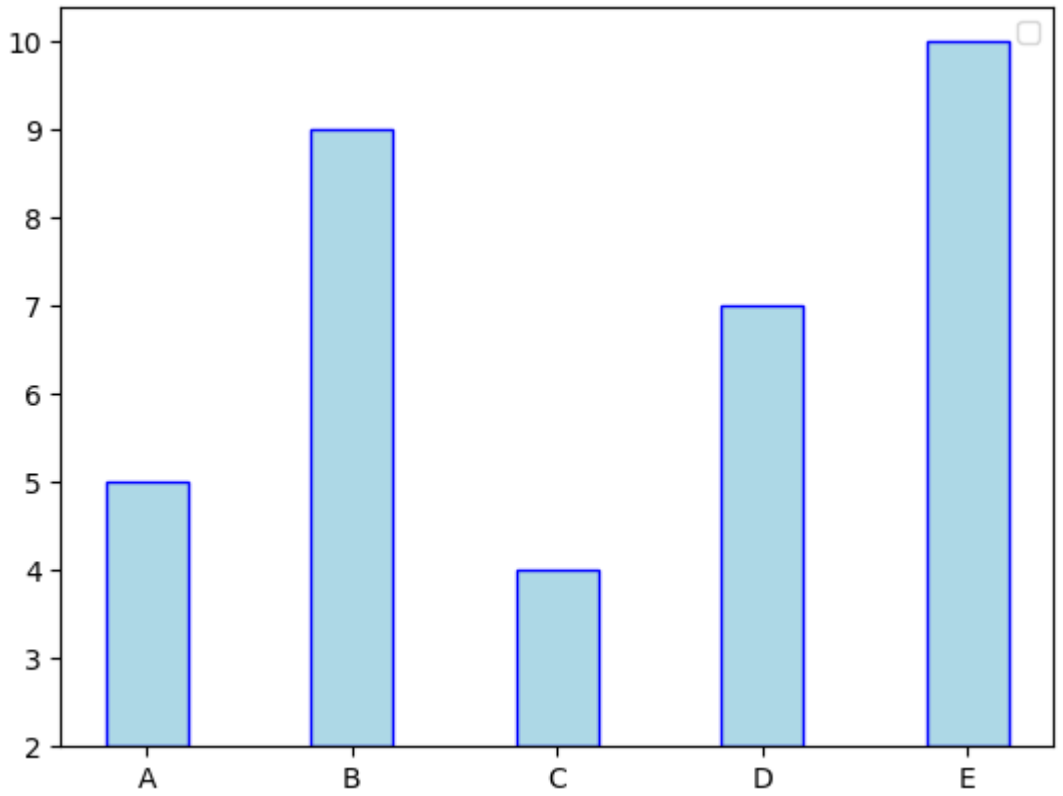
- bar( )

```
import matplotlib.pyplot as plt

# 데이터 설정
labels = ['A', 'B', 'C', 'D', 'E']  # 막대의 레이블
values = [3, 7, 2, 5, 8]            # 막대의 높이 값

# 막대 그래프 생성
plt.bar(
    labels,
    values,
    width=0.4,           # 막대의 너비 설정 (0.4)
    bottom=2,            # 막대 시작 위치를 y=2로 설정
    align='center',      # 막대 레이블 정렬 방식 (중앙 정렬)
    color='lightblue',   # 막대의 색상을 밝은 파란색으로 설정
    edgecolor='blue'     # 막대 테두리 색상을 파란색으로 설정
)

plt.legend()  # 범례 추가
plt.show()    # 그래프 출력
```



Line Plot

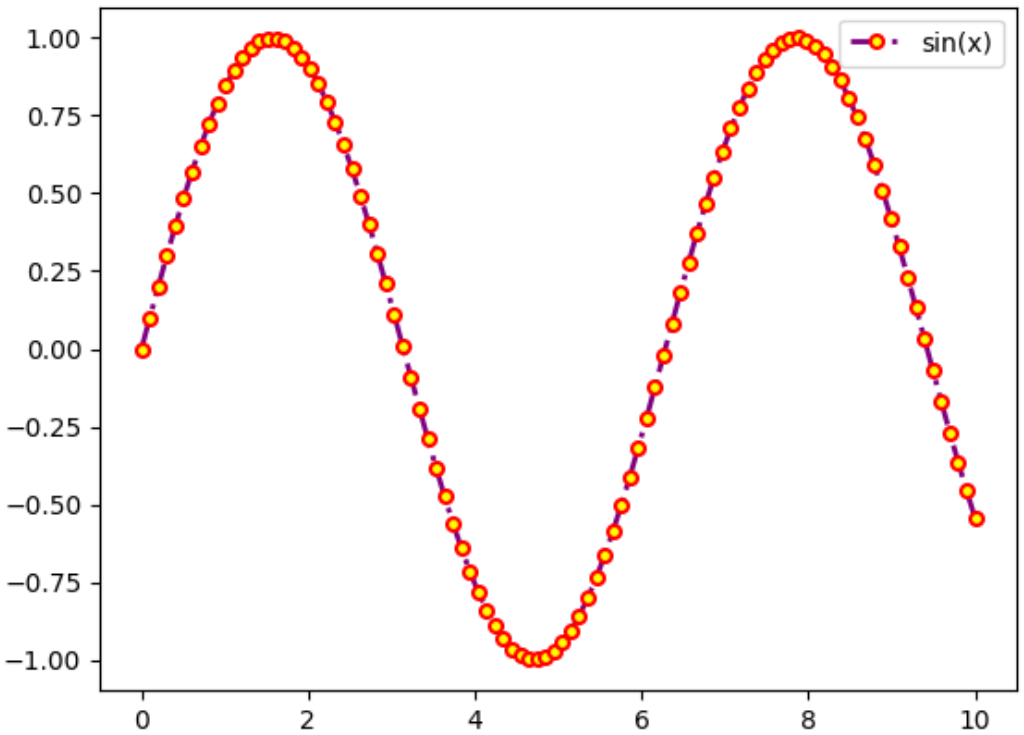
```
import matplotlib.pyplot as plt
import numpy as np

# 데이터 생성
x = np.linspace(0, 10, 100)
y = np.sin(x)

# 플롯 설정
plt.plot(x, y,
         color='purple',            # 선의 색상을 보라색으로 설정
         linestyle='dashdot',       # 선의 스타일을 대시-점(dashdot)으로 설정
         linewidth=2,               # 선의 너비를 2로 설정
         marker='o',                # 마커 스타일을 원형('o')으로 설정
         markersize=5,              # 마커의 크기를 5로 설정
         markerfacecolor='yellow',  # 마커의 색상을 노란색으로 설정
         markeredgecolor='red',     # 마커 테두리의 색상을 빨간색으로 설정
         markeredgewidth=1.5,       # 마커 테두리의 너비를 1.5로 설정
         label='sin(x)')            # 범례에서 표시될 라벨 설정

# 범례 표시
plt.legend()

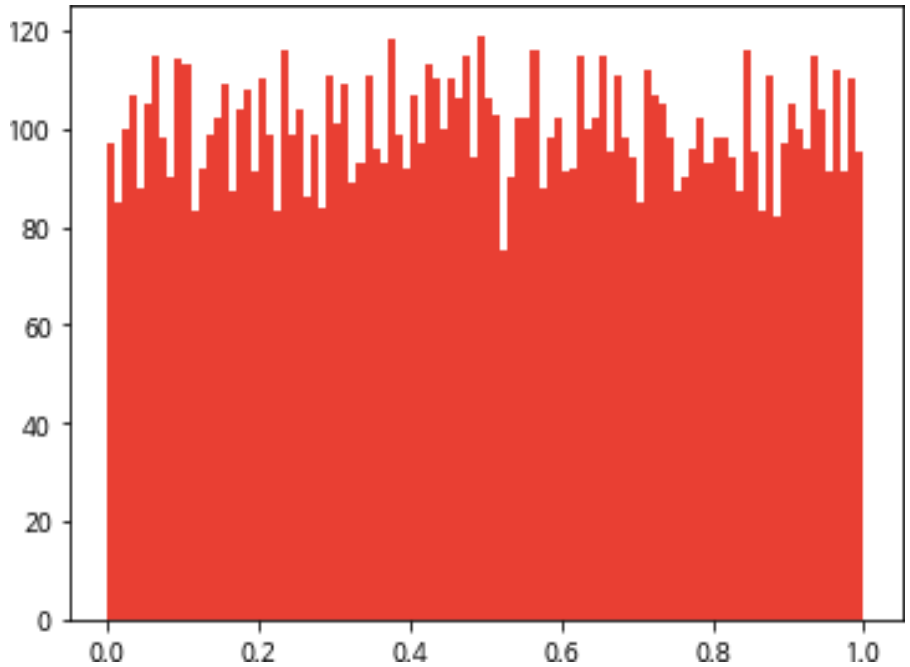
# 그래프 출력
plt.show()
```



Histogram

- hist( )

```
data = np.random.rand(10000)
fig, ax = plt.subplots()
ax.hist(data, bins = 100, facecolor='r')
plt.show()
```



Matplotlib

- hist( )

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import norm

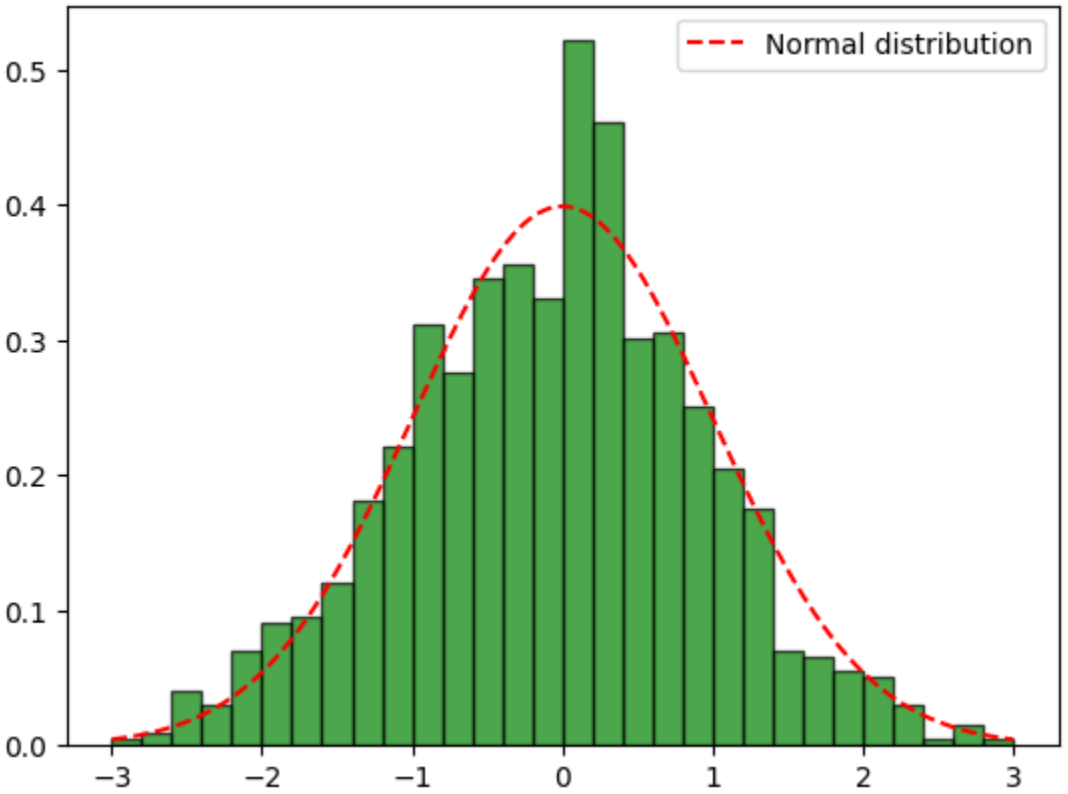
# 무작위 데이터 생성
data = np.random.randn(1000)  # 평균 0, 표준편차 1을 가지는 정규분포에서 1000개의 난수 생성

# 히스토그램 그리기
plt.hist(
    data,
    bins=30,                # 히스토그램의 막대 개수 설정 (30개)
    range=(-3, 3),          # x축 데이터의 범위 설정 (-3에서 3까지)
    density=True,           # 히스토그램의 높이를 확률 밀도로 정규화
    alpha=0.7,              # 막대의 투명도 (0.0: 완전 투명, 1.0: 불투명)
    color='g',              # 막대의 색상을 초록색으로 설정
    edgecolor='black'       # 막대 테두리 색상을 검정색으로 설정
)

# 표준 정규 분포 그리기
x = np.linspace(-3, 3, 1000)  # -3에서 3까지 1000개의 점 생성
y = norm.pdf(x, 0, 1)         # 평균 0, 표준편차 1인 정규 분포의 확률 밀도 함수 값 계산
plt.plot(
    x,
    y,
    'r--',                  # 선 스타일 설정: 빨간색(r) 점선(--)
    label='Normal distribution'  # 범례에 표시될 텍스트 설정
)

# 범례 추가
plt.legend()

# 그래프 출력
plt.show()
```



2D Scatter Plot

- Scatter( )

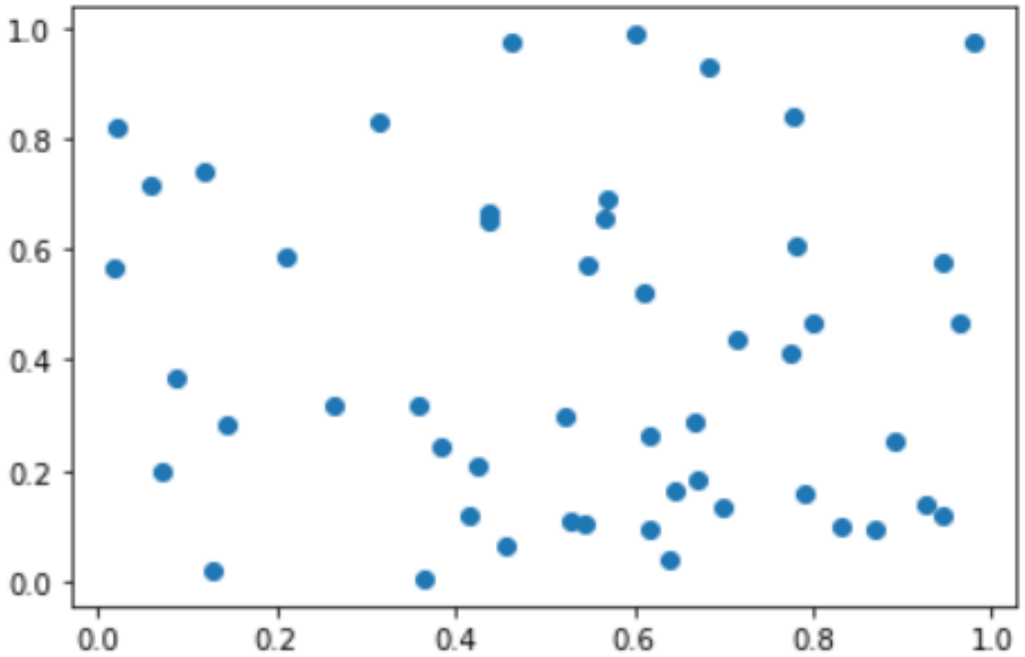
  : 두 변수 간의 관계를 표현할 때 사용

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)

n = 50
x = np.random.rand(n)
y = np.random.rand(n)

plt.scatter(x, y)
plt.show()
```



Matplotlib

- Scatter( )

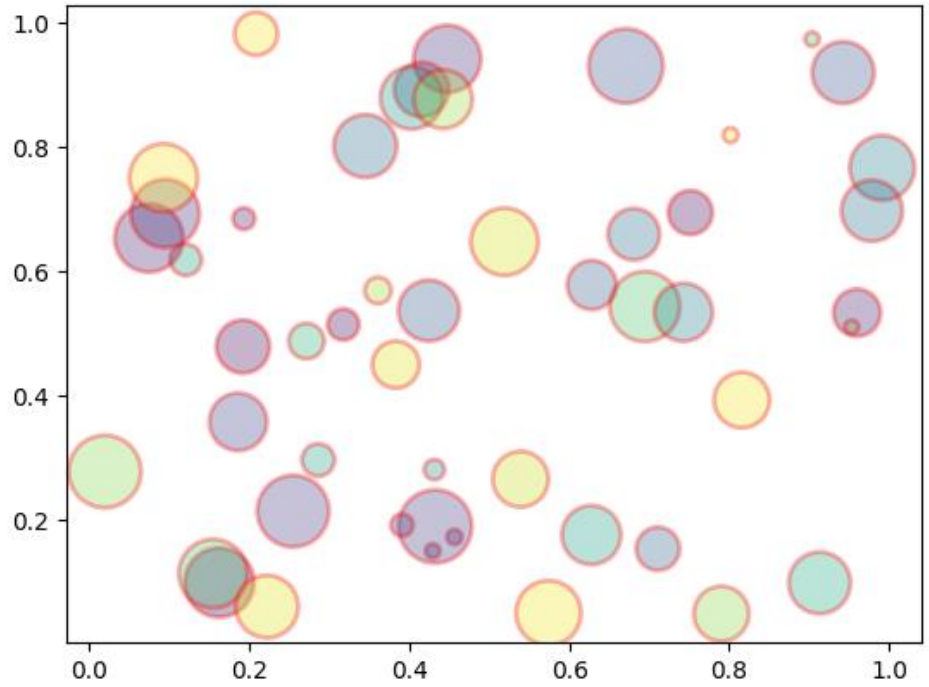
  : 두 변수 간의 관계를 표현할 때 사용

```
import matplotlib.pyplot as plt
import numpy as np

# 데이터 생성
x = np.random.rand(50)              # x축 데이터: 0과 1 사이의 랜덤 값 50개 생성
y = np.random.rand(50)              # y축 데이터: 0과 1 사이의 랜덤 값 50개 생성
colors = np.random.rand(50)         # 색상을 결정하는 값: 0과 1 사이의 랜덤 값 50개
sizes = 1000 * np.random.rand(50)   # 마커 크기를 결정하는 값: 0과 1000 사이의 랜덤 값 50개

# scatter 플롯 설정
plt.scatter(
    x,
    y,
    c=colors,                       # 마커 색상을 설정 (colors 배열로 전달)
    s=sizes,                        # 마커 크기를 설정 (sizes 배열로 전달)
    alpha=0.3,                      # 마커 투명도 (0.0은 완전히 투명, 1.0은 불투명)
    cmap='viridis',                 # 컬러맵 설정 (viridis는 기본적인 컬러맵)
    edgecolors='red',               # 마커 테두리 색상 설정 (빨간색)
    linewidths=2                    # 마커 테두리의 두께 설정 (2 픽셀)
)

# 그래프 출력
plt.show()
```



3D Scatter Plot

- Scatter( )

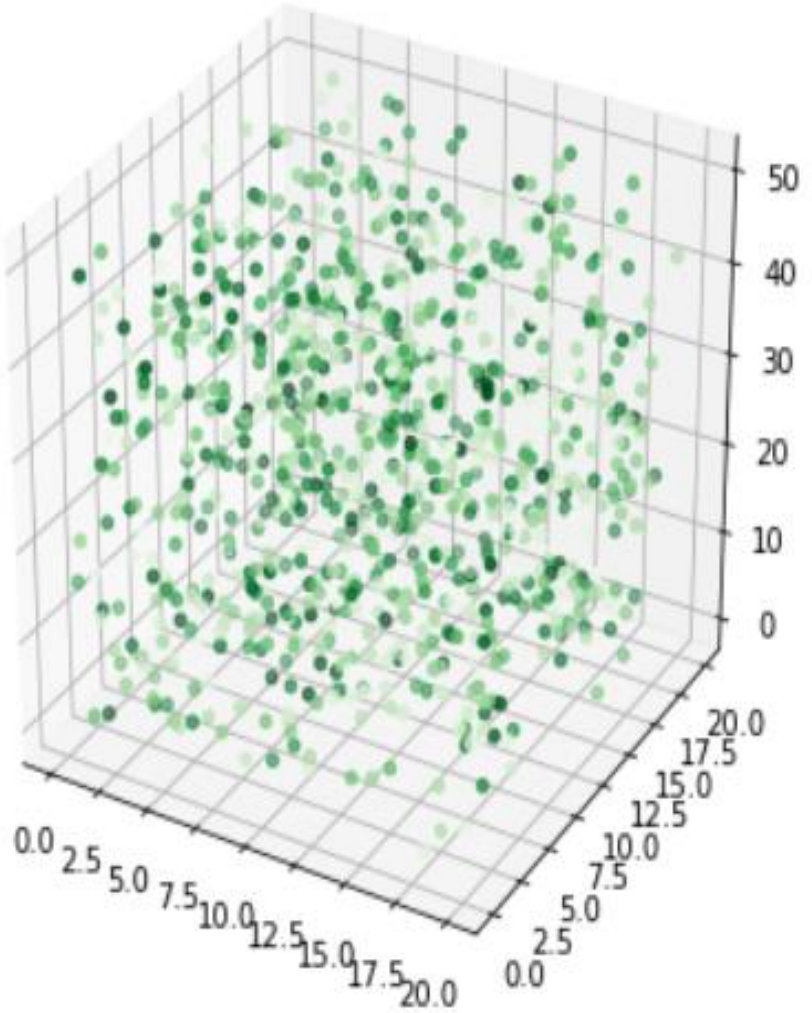
```
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
import numpy as np

n = 1000
xmin, xmax, ymin, ymax, zmin, zmax = 0, 20, 0, 20, 0, 50
cmin, cmax = 0, 2

xs = np.array([(xmax - xmin) * np.random.random_sample() + xmin for i in range(n)])
ys = np.array([(ymax - ymin) * np.random.random_sample() + ymin for i in range(n)])
zs = np.array([(zmax - zmin) * np.random.random_sample() + zmin for i in range(n)])
color = np.array([(cmax - cmin) * np.random.random_sample() + cmin for i in range(n)])

fig = plt.figure(figsize=(6, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(xs, ys, zs, c=color, marker='o', s=15, cmap='Greens')

plt.show()
```



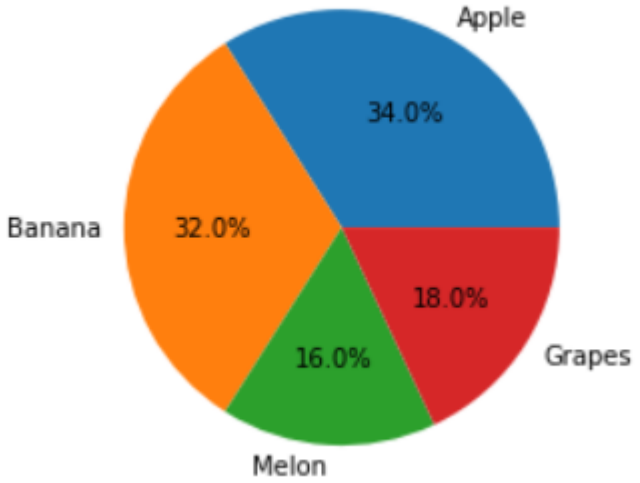
Pie chart

- Pie( )

```
import matplotlib.pyplot as plt

ratio = [34, 32, 16, 18]
labels = ['Apple', 'Banana', 'Melon', 'Grapes']

plt.pie(ratio, labels=labels, autopct='%.1f%%')
plt.show()
```



Matplotlib

- Pie( )

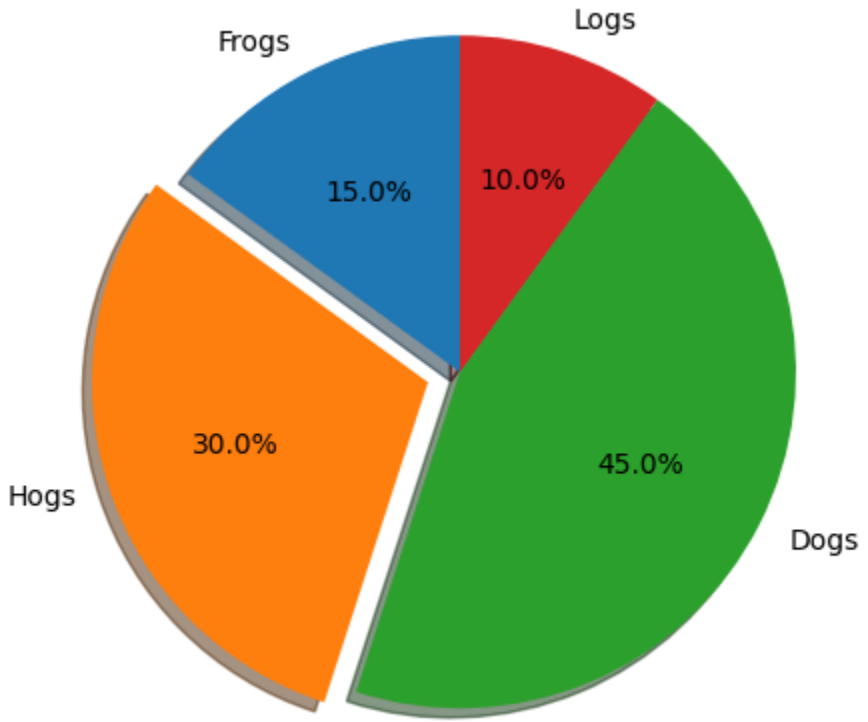
```
import matplotlib.pyplot as plt

# 데이터 설정
labels = ['Frogs', 'Hogs', 'Dogs', 'Logs']  # 각 파이 조각의 레이블
sizes = [15, 30, 45, 10]                    # 각 파이 조각의 크기 (비율)
explode = (0, 0.1, 0, 0)                    # 특정 파이 조각을 분리 (2번째 조각: 'Hogs'만 분리)

# 파이 차트 생성
fig1, ax1 = plt.subplots()
ax1.pie(
    sizes,                                  # 각 조각의 크기
    explode=explode,                        # 분리 설정 (explode)
    labels=labels,                          # 각 조각의 레이블
    autopct='%1.1f%%',                      # 각 조각의 비율을 소수점 1자리로 표시
    shadow=True,                            # 파이 차트에 그림자 추가
    startangle=90                           # 파이 차트의 시작 각도 설정 (90도)
)

# 파이 차트의 비율 설정
ax1.axis('equal')  # 차트를 원형으로 보이게 설정

# 차트 출력
plt.show()
```



Box Plot

```
import matplotlib.pyplot as plt
import numpy as np

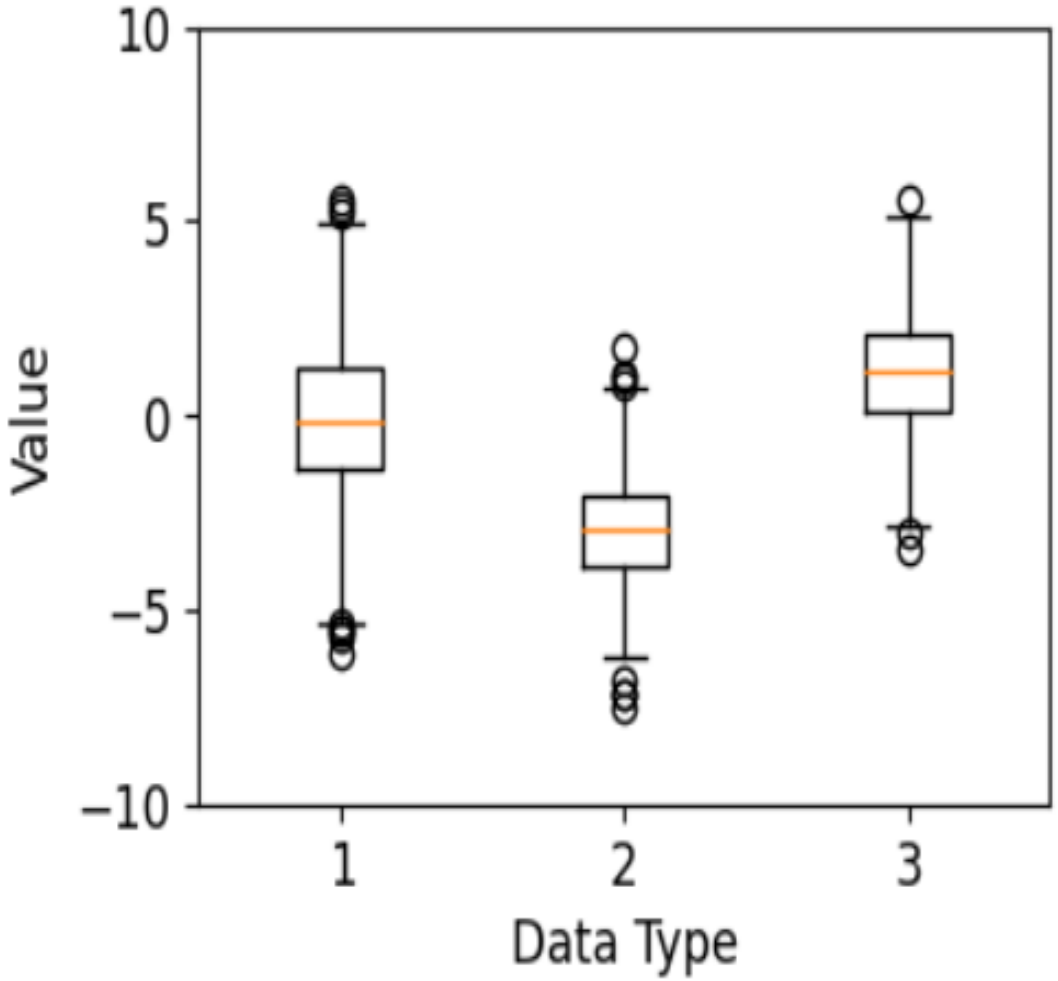
# 1. 기본 스타일 설정
plt.style.use('default')
plt.rcParams['figure.figsize'] = (4, 3)
plt.rcParams['font.size'] = 12

# 2. 데이터 준비
np.random.seed(0)
data_a = np.random.normal(0, 2.0, 1000)
data_b = np.random.normal(-3.0, 1.5, 500)
data_c = np.random.normal(1.2, 1.5, 1500)

# 3. 그래프 그리기
fig, ax = plt.subplots()

ax.boxplot([data_a, data_b, data_c])
ax.set_ylim(-10.0, 10.0)
ax.set_xlabel('Data Type')
ax.set_ylabel('Value')

plt.show()
```



Box Plot

```python
import matplotlib.pyplot as plt
import numpy as np

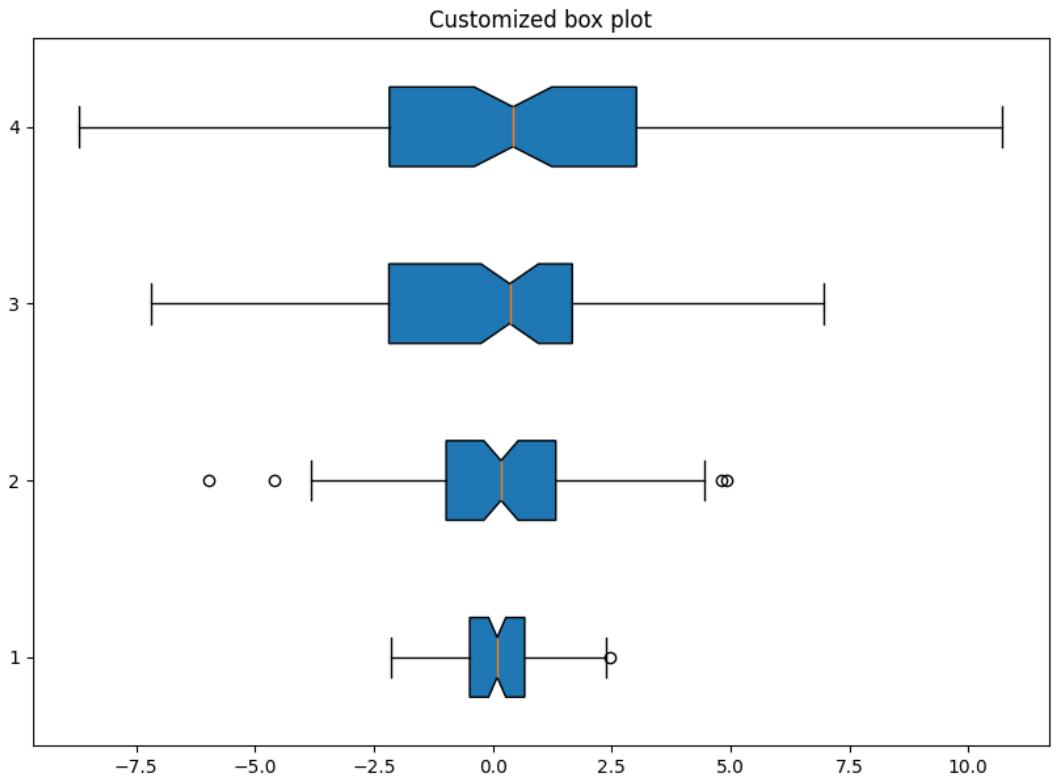
# 랜덤 데이터 생성
np.random.seed(10)  # 랜덤 시드 설정으로 재현 가능한 결과 생성
data = [np.random.normal(0, std, 100) for std in range(1, 5)]
# 평균 0, 표준편차 std인 정규분포에서 100개의 샘플을 생성 (std는 1에서 4까지)

# 플롯 설정
fig = plt.figure(figsize=(10, 7))  # 전체 플롯의 크기 설정 (가로 10, 세로 7)
ax = fig.add_subplot(111)          # 하나의 서브플롯 추가 (1행 1열의 첫 번째 플롯)

# Boxplot 생성
bp = ax.boxplot(
    data,
    patch_artist=True,  # 박스를 채우는 색상 활성화
    notch=True,         # 박스를 notched boxplot(홈이 있는 형태)로 설정
    vert=0              # 수평 방향으로 박스플롯 표시 (0은 수평, 1은 수직)
)

# 플롯 제목 추가
plt.title("Customized box plot")  # 플롯 제목 설정

# 플롯 보여주기
plt.show()
```



Violin Plot

```python
# 기본 사용
import matplotlib.pyplot as plt
import numpy as np

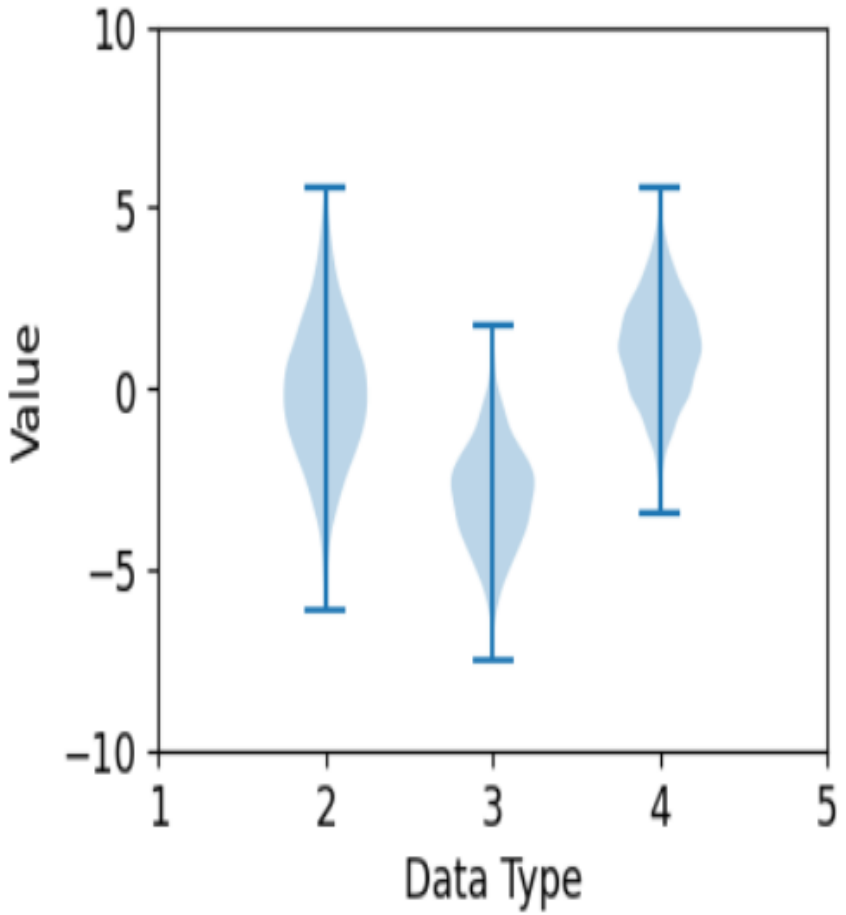
# 1. 기본 스타일 설정
plt.style.use('default')
plt.rcParams['figure.figsize'] = (4, 3)
plt.rcParams['font.size'] = 12

# 2. 데이터 준비
np.random.seed(0)
data_a = np.random.normal(0, 2.0, 1000)
data_b = np.random.normal(-3.0, 1.5, 500)
data_c = np.random.normal(1.2, 1.5, 1500)

# 3. 그래프 그리기
fig, ax = plt.subplots()

violin = ax.violinplot([data_a, data_b, data_c], positions=[2, 3, 4])
ax.set_ylim(-10.0, 10.0)
ax.set_xticks([1, 2, 3, 4, 5])
ax.set_xlabel('Data Type')
ax.set_ylabel('Value')

plt.show()
```



Error bar

: 데이터의 불확실성을 표현할 때 사용됨. 데이터 주변에 오차를 나타내는 선을 그립니다.

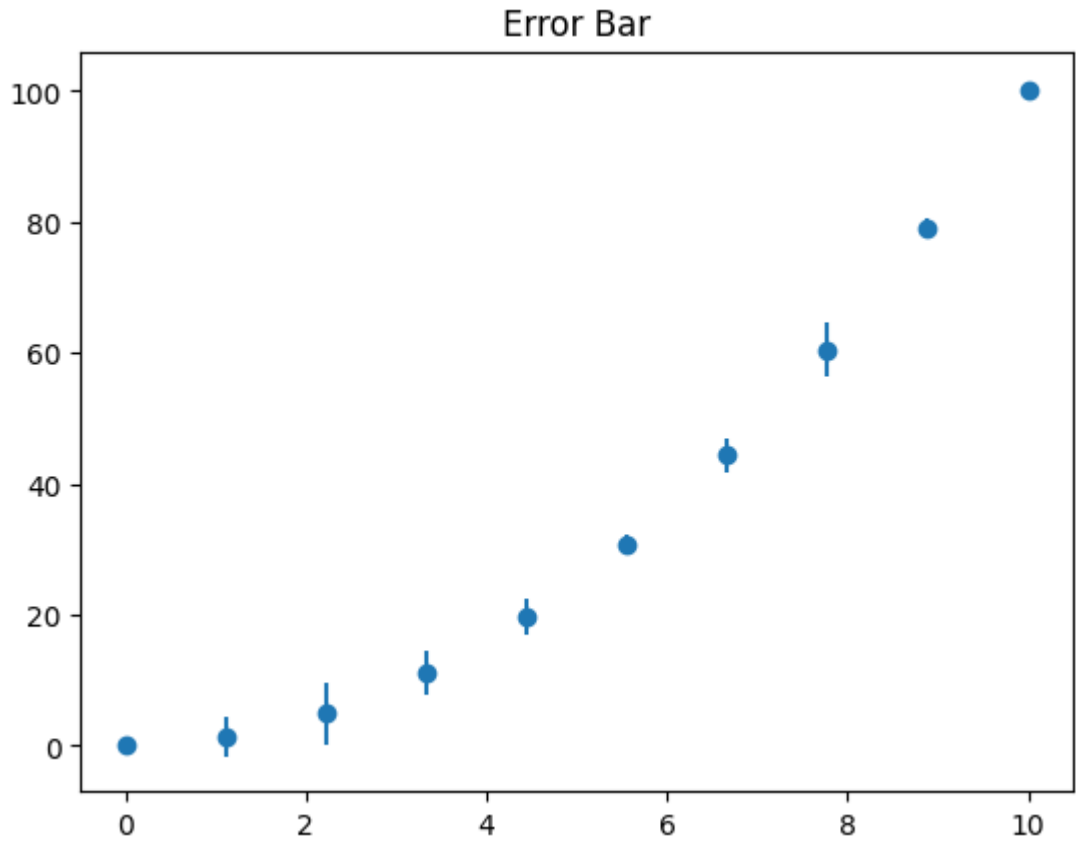
```
import matplotlib.pyplot as plt
import numpy as np

# 데이터 생성
x = np.linspace(0, 10, 10)          # 0에서 10까지 균등하게 나눈 10개의 점 생성
y = x ** 2                          # y 값은 x의 제곱
errors = np.random.rand(10) * 5     # 랜덤으로 생성된 에러 값 (0~5 범위)

# Error bar 그리기
plt.errorbar(
    x, y,
    yerr=errors,    # y축 방향의 에러 범위
    fmt='o'         # 데이터 포인트를 원형 마커로 표시
)

# 그래프 제목 추가
plt.title('Error Bar')              # 그래프 제목 설정

# 그래프 보여주기
plt.show()
```



Error bar

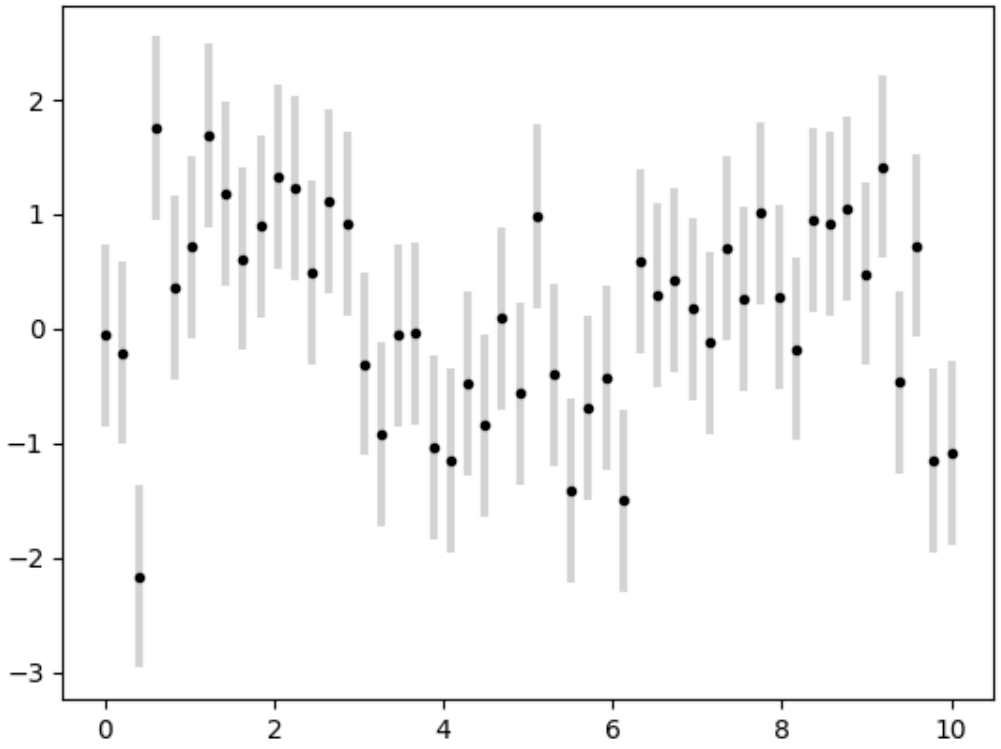
: 데이터의 불확실성을 표현할 때 사용됨. 데이터 주변에 오차를 나타내는 선을 그립니다.

```
import numpy as np
import matplotlib.pyplot as plt

# 데이터 생성
x = np.linspace(0, 10, 50)                  # 0에서 10까지 50개의 균일한 점 생성
dy = 0.8                                    # y값에 추가될 에러 값의 스케일
y = np.sin(x) + dy * np.random.randn(50)    # y값은 sin(x)에 랜덤한 잡음을 추가

# 오차 막대 그래프 생성
plt.errorbar(
    x, y,
    yerr=dy,                                # y축 방향의 오차 크기
    fmt='.k',                               # 데이터 포인트의 스타일: 검은색 점('.k')
    ecolor='lightgray',                     # 오차 막대의 색상: 밝은 회색
    elinewidth=3,                           # 오차 막대의 두께
    capsize=0                               # 오차 막대 끝 부분의 캡 크기 (0으로 설정하여 캡 없음)
)

# 그래프 출력
plt.show()
```



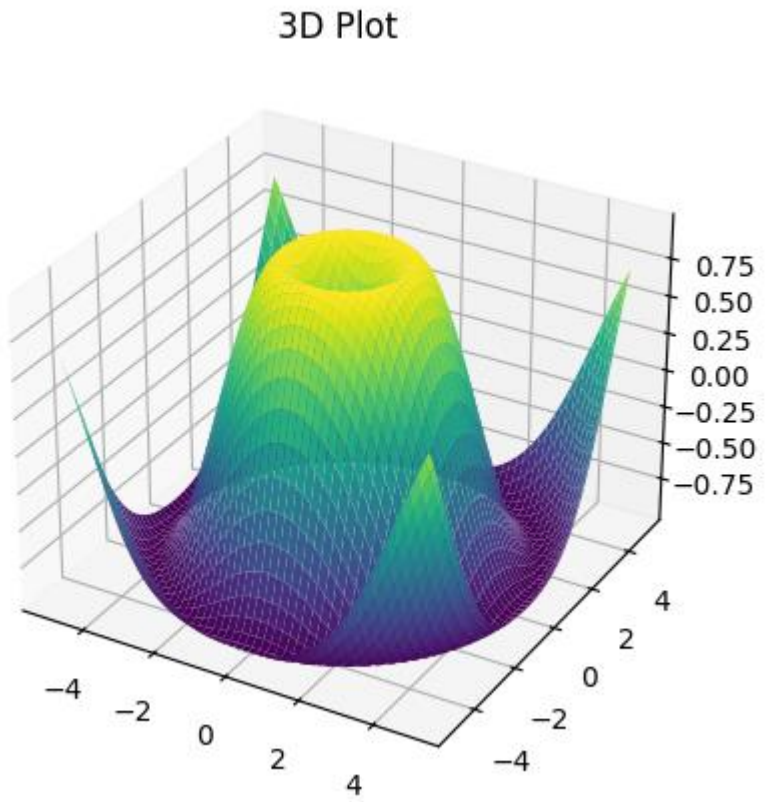
3D Plot

: 3차원 데이터를 표현할 때 사용함. X, Y, Z 값을 가지는 3차원 공간에 데이터를 표현함.

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

# 데이터 생성
x = np.linspace(-5, 5, 100)                    # x축 데이터를 -5에서 5까지 100개 생성
y = np.linspace(-5, 5, 100)                    # y축 데이터를 -5에서 5까지 100개 생성
X, Y = np.meshgrid(x, y)                       # x, y 데이터를 그리드 형태로 변환
Z = np.sin(np.sqrt(X**2 + Y**2))               # Z 데이터는 x^2 + y^2의 제곱근의 사인 값

# 3D plot 그리기
fig = plt.figure()                             # 새로운 Figure 생성
ax = fig.add_subplot(111, projection='3d')     # 3D 플롯 추가
ax.plot_surface(X, Y, Z, cmap='viridis')       # 3D 표면 플롯 생성, viridis 컬러맵 적용
plt.title('3D Plot')                           # 그래프 제목 설정
plt.show()                                     # 그래프 출력
```



Image

```
[48] import matplotlib.image as mpimg
```

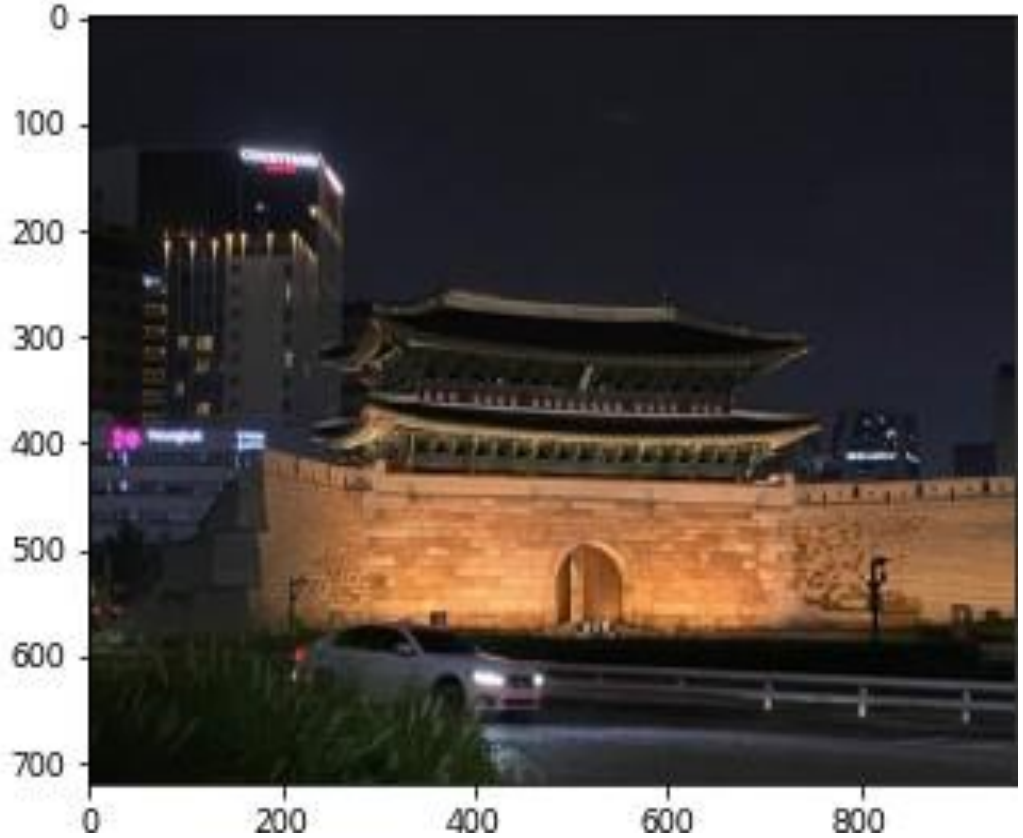
```
img = mpimg.imread('img_.jpeg')
print(type(img))
```

```
<class 'numpy.ndarray'>
```

```
imgplot = plt.imshow(img)
print(type(imgplot))
```

```
<class 'matplotlib.image.AxesImage'>
```

```
<class 'matplotlib.image.AxesImage'>
```



고급 기법 활용하기

```python
import matplotlib.pyplot as plt

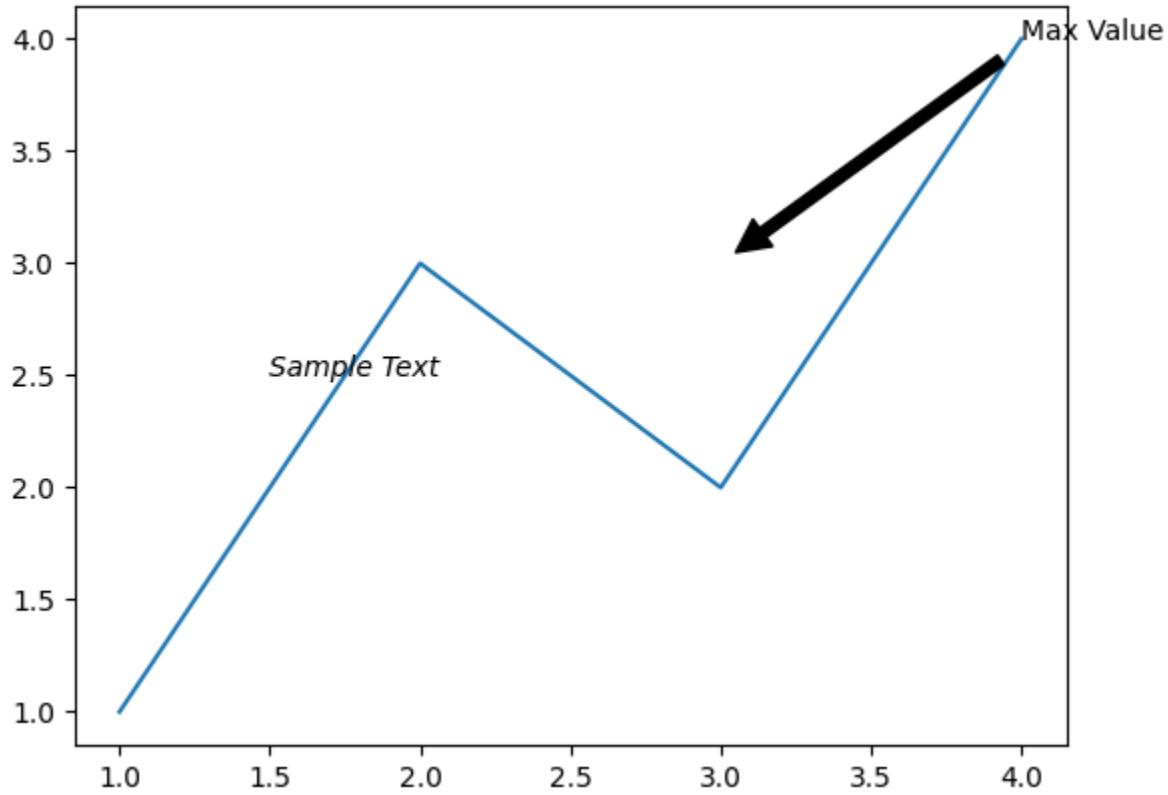
# 플롯 생성
fig, ax = plt.subplots()

# 데이터 플롯
ax.plot([1, 2, 3, 4], [1, 3, 2, 4])  # x=[1, 2, 3, 4], y=[1, 3, 2, 4] 데이터를 플롯

# 그래프에 텍스트 추가
ax.text(
    1.5, 2.5,                  # 텍스트가 위치할 x, y 좌표
    'Sample Text',             # 표시할 텍스트
    style='italic'             # 텍스트 스타일을 기울임(italic)으로 설정
)

# 그래프에 주석 추가
ax.annotate(
    'Max Value',               # 주석에 표시할 텍스트
    xy=(3, 3),                 # 화살표가 가리키는 데이터 포인트
    xytext=(4, 4),             # 텍스트가 표시될 위치
    arrowprops=dict(
        facecolor='black',     # 화살표 색상 설정 (검정색)
        shrink=0.05            # 화살표 줄어드는 정도 (5%)
    )
)

# 그래프 출력
plt.show()
```



Seaborn

- seaborn은 matplotlib을 기반으로 다양한 색 테마, 차트 기능을 추가한 라이브러리입니다
- seaborn은 Matplotlib을 기반하여 통계용 그래프를 그리는 데 특화가 되어있음.
  -> 좀 더 복잡한 그래프를 간단한 코드로 구현할 수 있게 해줌.
  -> 퀄리티 있는 그래픽과 복잡한 그래프 기능을 제공하며, Pandas 데이터프레임과 잘 연동됨.

설치

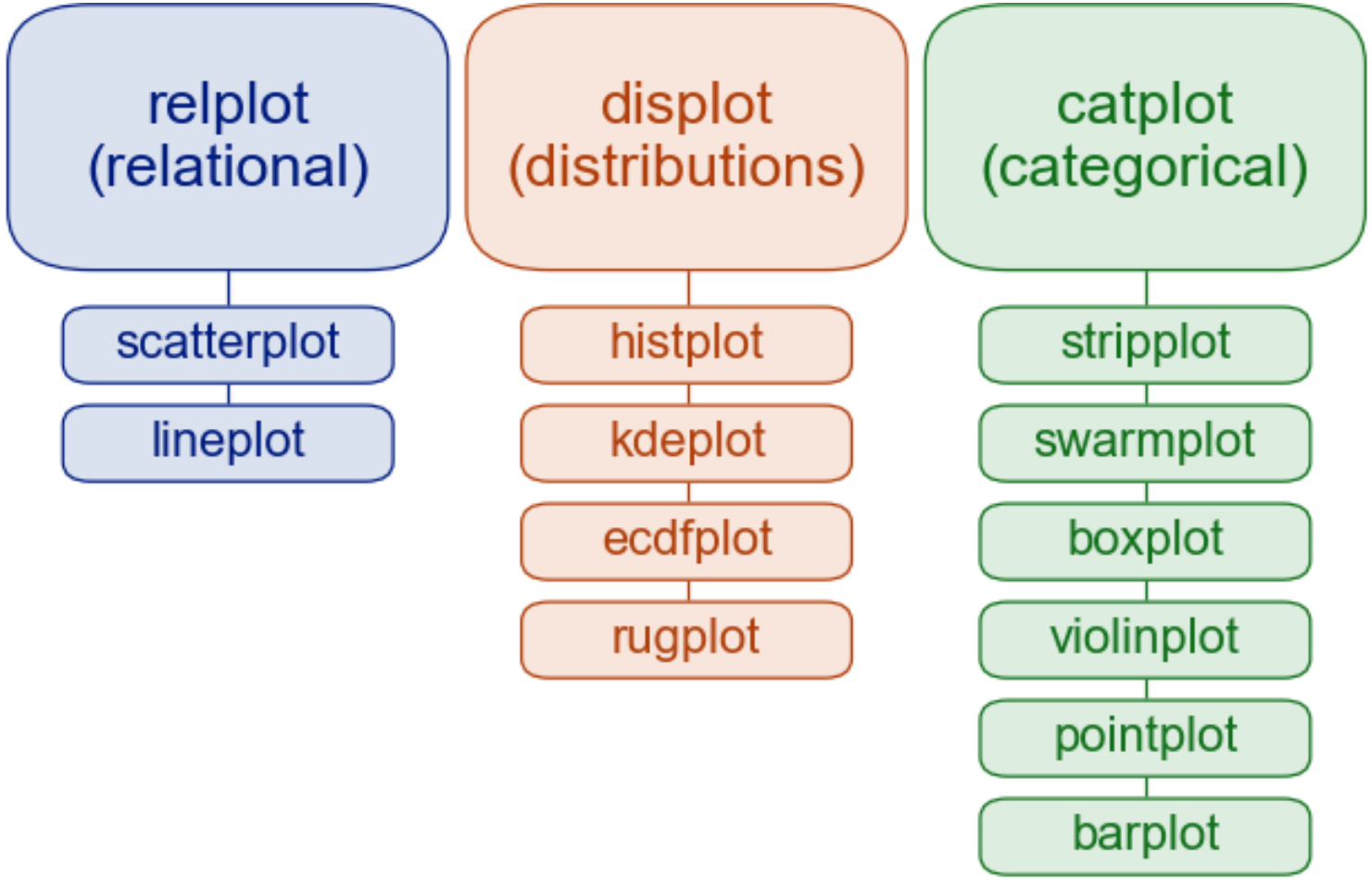
pip install seaborn

```
import seaborn as sns

print("Seaborn version : ", sns.__version__)
```

```
Seaborn version :  0.9.0
```

Seaborn의 종류



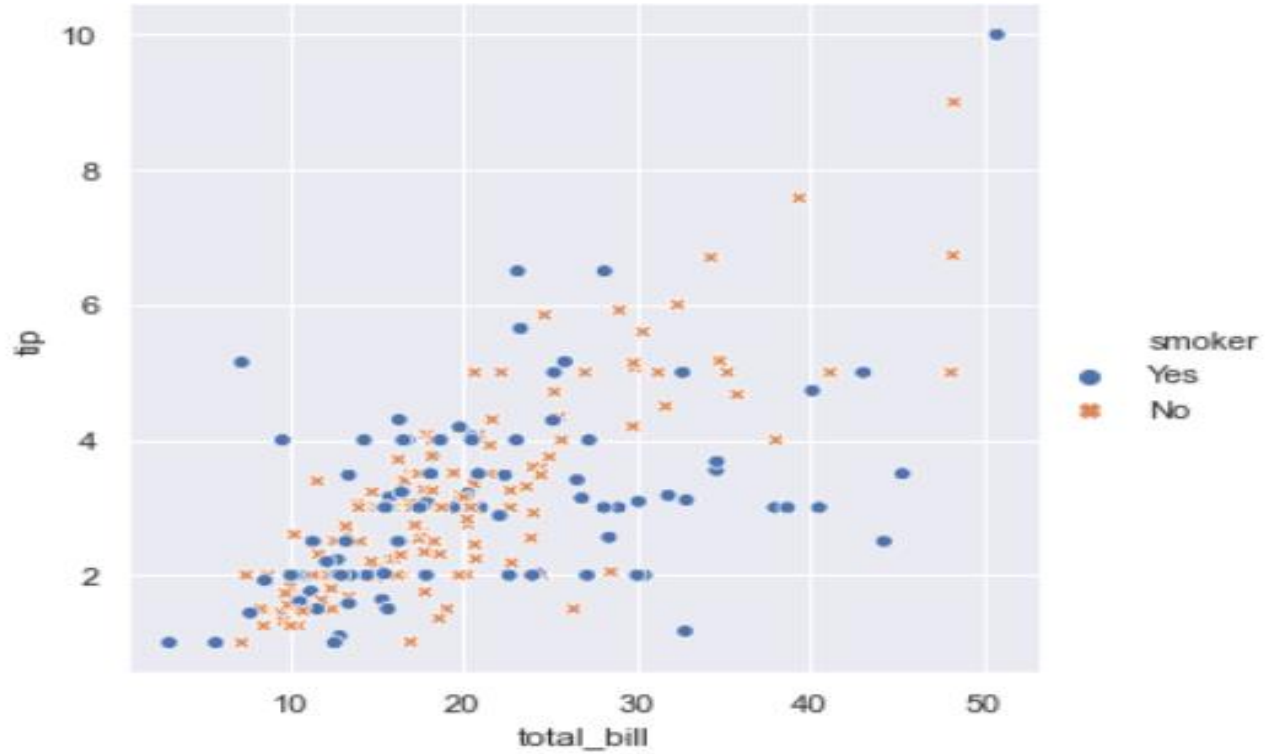
참고 : https://seaborn.pydata.org/tutorial.html

Seaborn_relplot

```
tips = sns.load_dataset("tips")

sns.relplot(x="total_bill", y="tip", hue="smoker", style="smoker",
            data=tips)
```

```
<seaborn.axisgrid.FacetGrid at 0x1bd18b8b390>
```



Seaborn_relplot

```
import seaborn as sns
import pandas as pd
import numpy as np
df = pd.DataFrame(dict(time=np.arange(500),
                       value=np.random.randn(500).cumsum()))
g = sns.relplot(x="time", y="value", kind="line", data=df)
g.fig.autofmt_xdate()
```



Seaborn_catplot

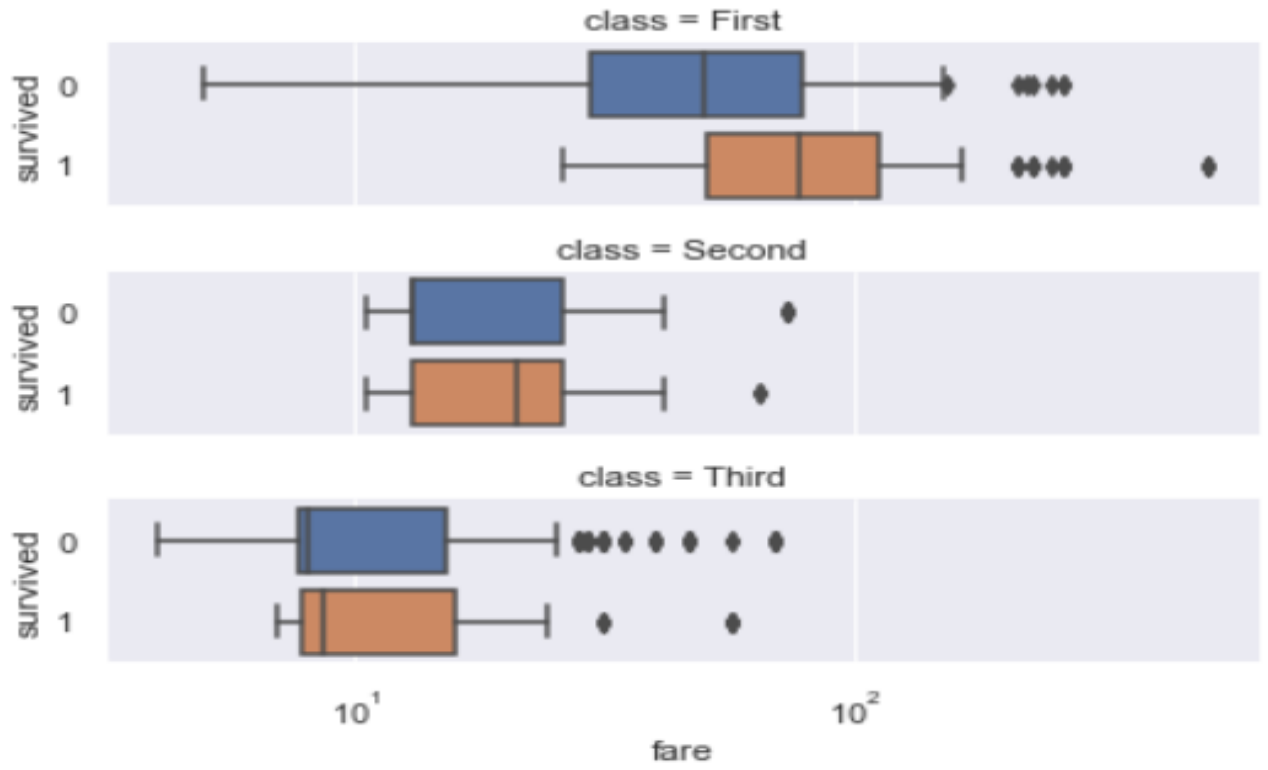
```
sns.catplot(x="day", y="total_bill", hue="smoker",
            col="time", aspect=.6,
            kind="swarm", data=tips)
```

```
<seaborn.axisgrid.FacetGrid at 0x1bd1acb92e8>
```



Seaborn_catplot

```
titanic = sns.load_dataset("titanic")
g = sns.catplot(x="fare", y="survived", row="class",
                kind="box", orient="h", height=1.5, aspect=4,
                data=titanic.query("fare > 0"))
g.set(xscale="log");
```



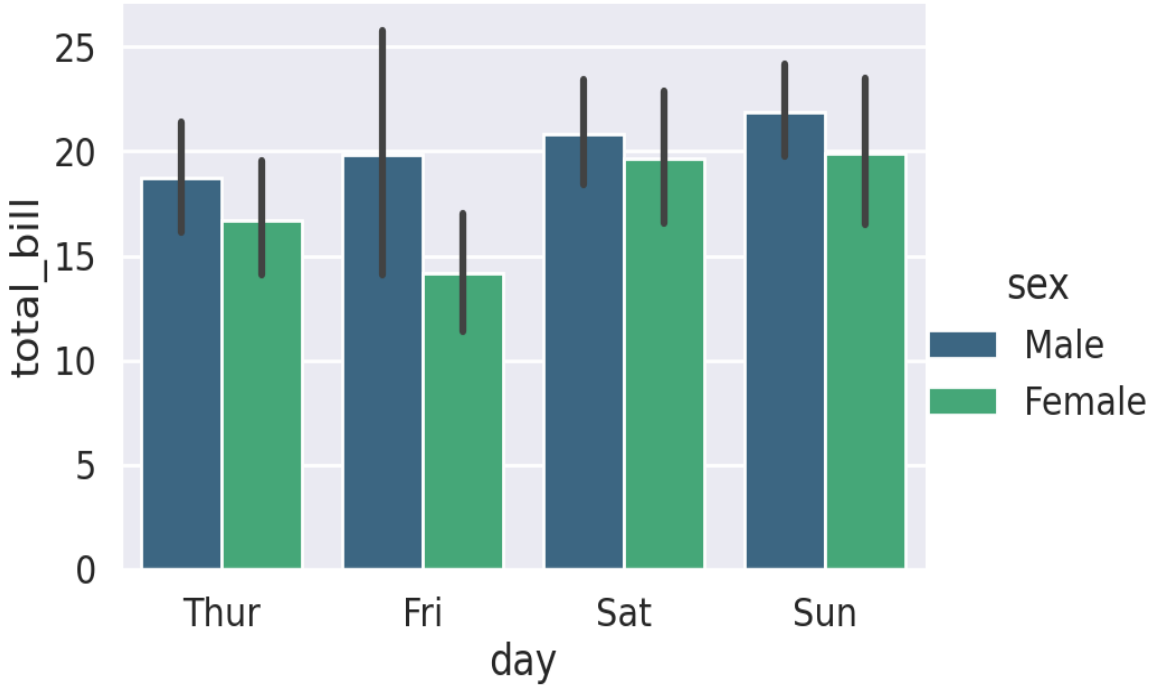
Seaborn_catplot

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

# Catplot 생성
sns.catplot(
    x="day",                # x축: 요일
    y="total_bill",         # y축: 총 계산 금액
    hue="sex",              # 색상을 성별로 구분
    data=tips,              # 데이터셋 설정
    kind="bar",             # 그래프 유형: 막대그래프
    palette="viridis",      # 색상 팔레트: viridis
    height=6,               # 그래프 높이 설정 (단위: 인치)
    aspect=1.5              # 그래프 가로-세로 비율 설정 (1.5는 더 넓게)
)

# 그래프 출력
plt.show()
```



Seaborn_Scatterplot

```python
import seaborn as sns
import matplotlib.pyplot as plt

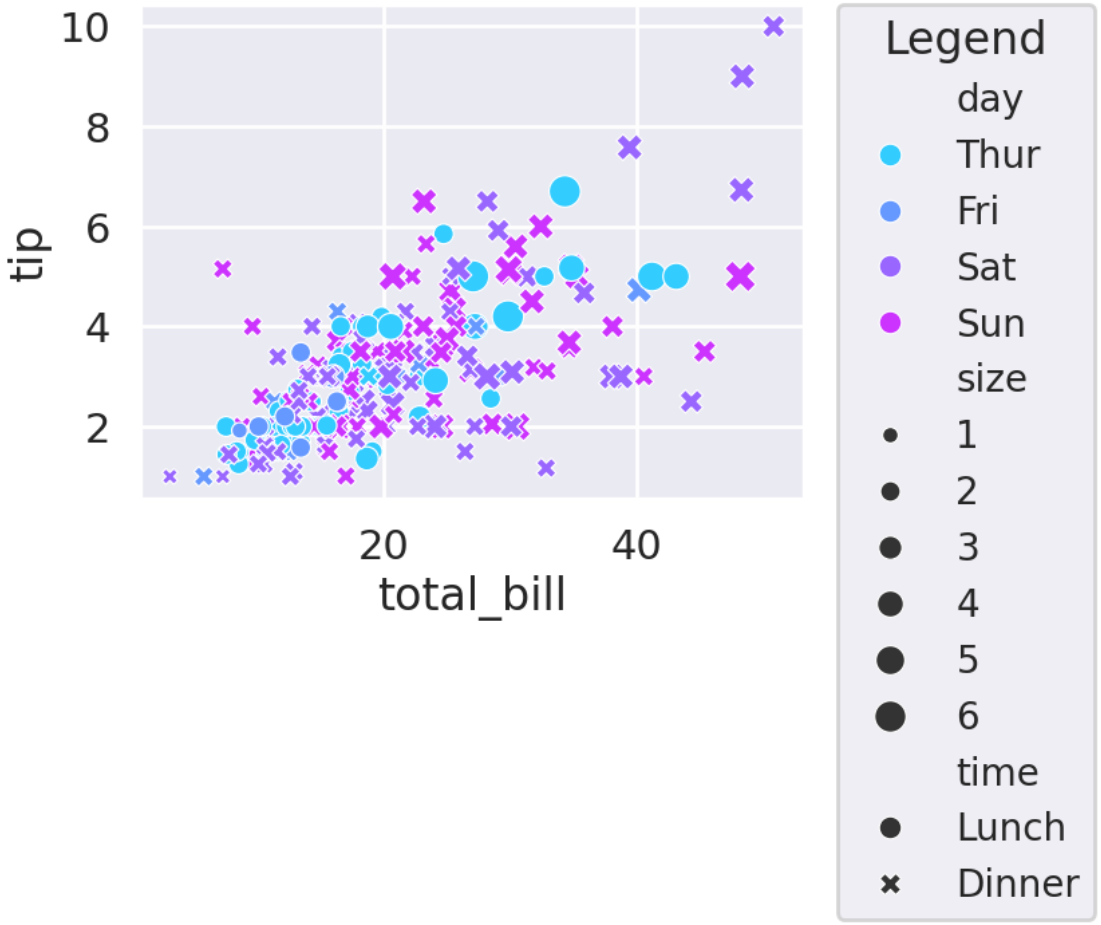
# 데이터셋 로드
tips = sns.load_dataset("tips")

# 산점도 그리기
sns.scatterplot(
    x="total_bill",  # x축: 총 계산 금액
    y="tip",         # y축: 팁 금액
    hue="day",       # 점의 색상을 요일별로 구분
    style="time",    # 점의 스타일을 식사 시간(Lunch/Dinner)으로 구분
    size="size",     # 점의 크기를 테이블 크기(손님 수)로 구분
    data=tips,       # 데이터셋 설정
    palette="cool"   # 팔레트를 'cool'로 설정
)

# 범례 조정
legend = plt.legend(
    bbox_to_anchor=(1.05, 1),      # 범례를 플롯 오른쪽으로 배치
    loc='upper left',              # 범례의 기준점 설정
    borderaxespad=0.,              # 그래프와의 간격 설정
    fontsize='small',              # 범례 텍스트 크기 축소
    title="Legend"                 # 범례 제목
)

# 범례 아이템 크기 줄이기
for handle in legend.legendHandles:
    handle._sizes = [20]  # 각 아이템 크기 축소

# 그래프 출력
plt.show()
```



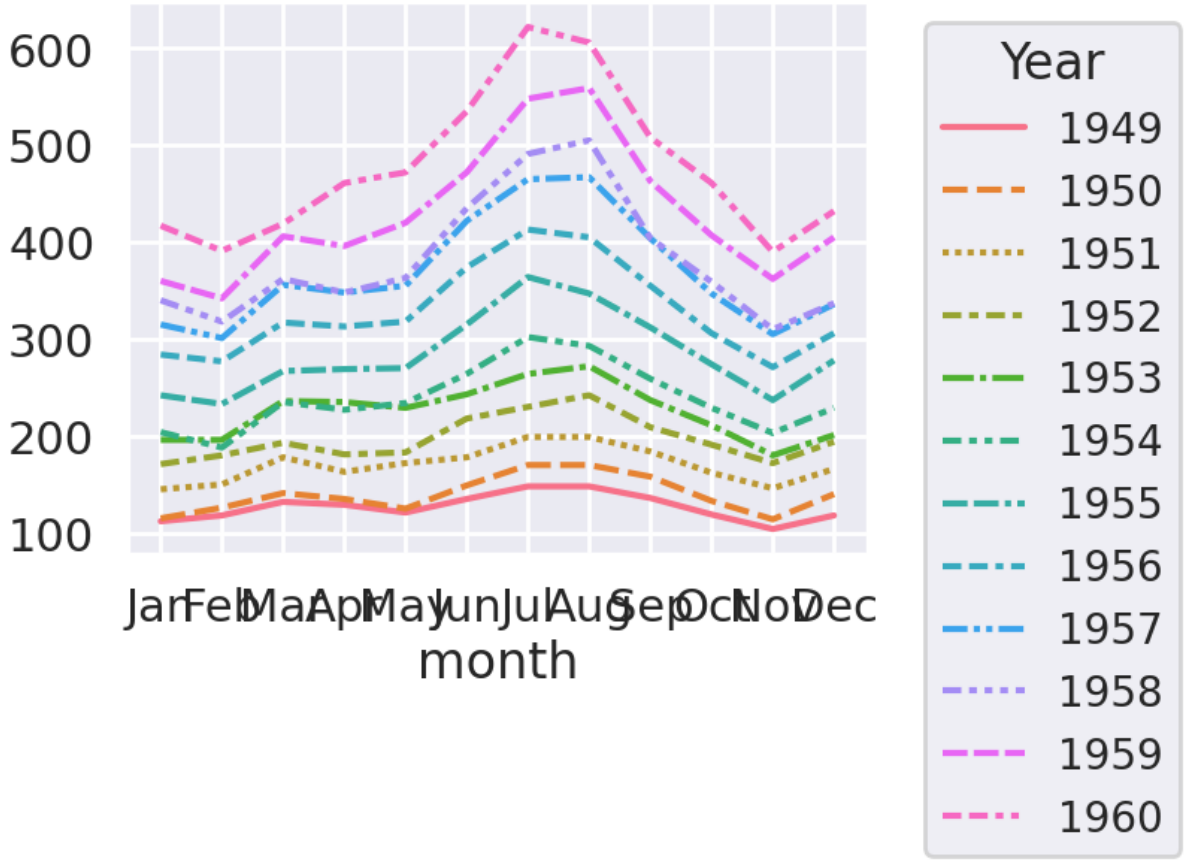
Seaborn_lineplot

```python
import seaborn as sns
import matplotlib.pyplot as plt

# flights 데이터셋 로드
flights = sns.load_dataset("flights")
flights_pivot = flights.pivot(index="month", columns="year", values="passengers")

# 라인 플롯 그리기
sns.lineplot(data=flights_pivot)

# 범례 조정
plt.legend(
    title="Year",              # 범례 제목 설정
    bbox_to_anchor=(1.05, 1),  # 범례를 그래프 밖 오른쪽에 배치
    loc='upper left',          # 범례의 위치 조정
    fontsize='small',          # 범례 텍스트 크기 축소
    title_fontsize='medium'    # 범례 제목 크기 설정
)
# 그래프 출력
plt.show()
```



categorical_Boxplot

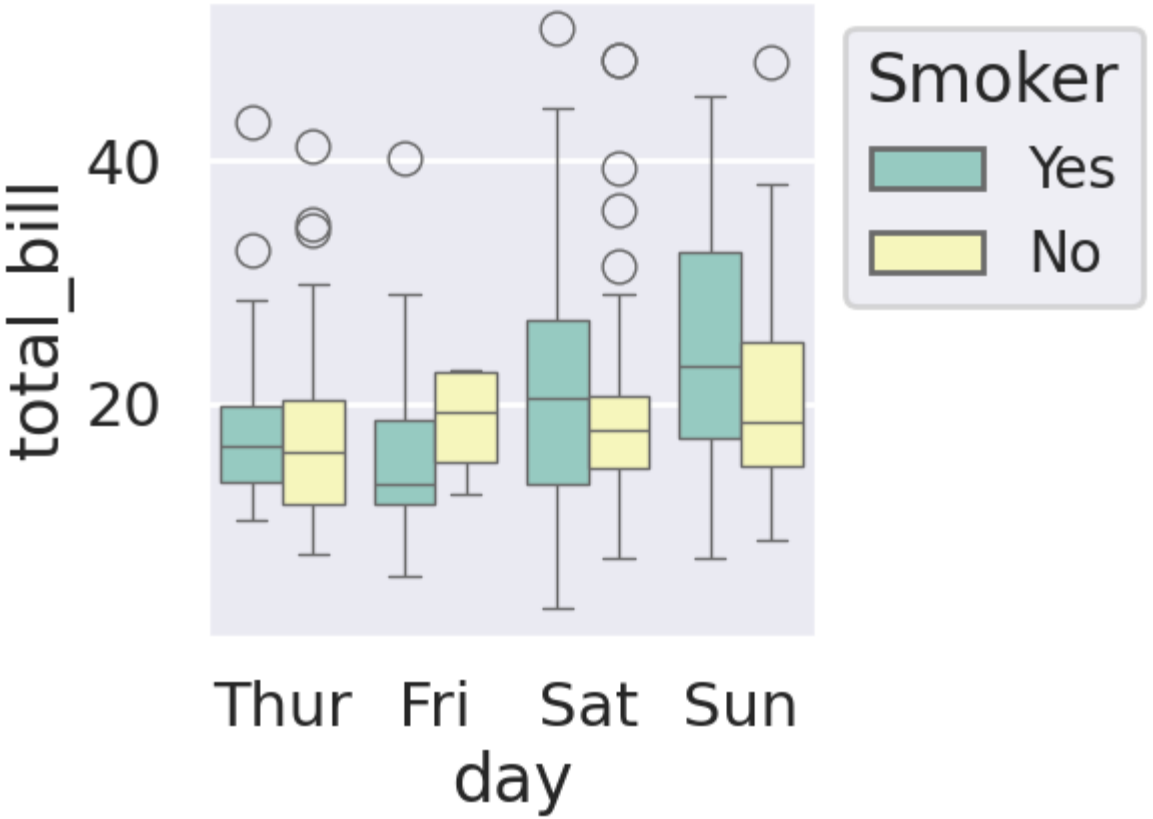
```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")

# Boxplot 생성
sns.boxplot(
    x="day",              # x축: 요일
    y="total_bill",       # y축: 총 계산 금액
    hue="smoker",         # 색상을 흡연 여부(smoker)로 구분
    data=tips,            # 데이터셋 설정
    palette="Set3"        # 색상 팔레트: Set3
)

# 범례 조정
plt.legend(
    title="Smoker",          # 범례 제목
    loc='upper left',        # 범례 위치: 그래프의 왼쪽 위
    bbox_to_anchor=(1, 1),   # 범례를 그래프 밖으로 이동
    fontsize='small',        # 범례 텍스트 크기 축소
    title_fontsize='medium'  # 범례 제목 크기 설정
)

# 그래프 출력
plt.tight_layout()  # 그래프 간격 조정
plt.show()
```



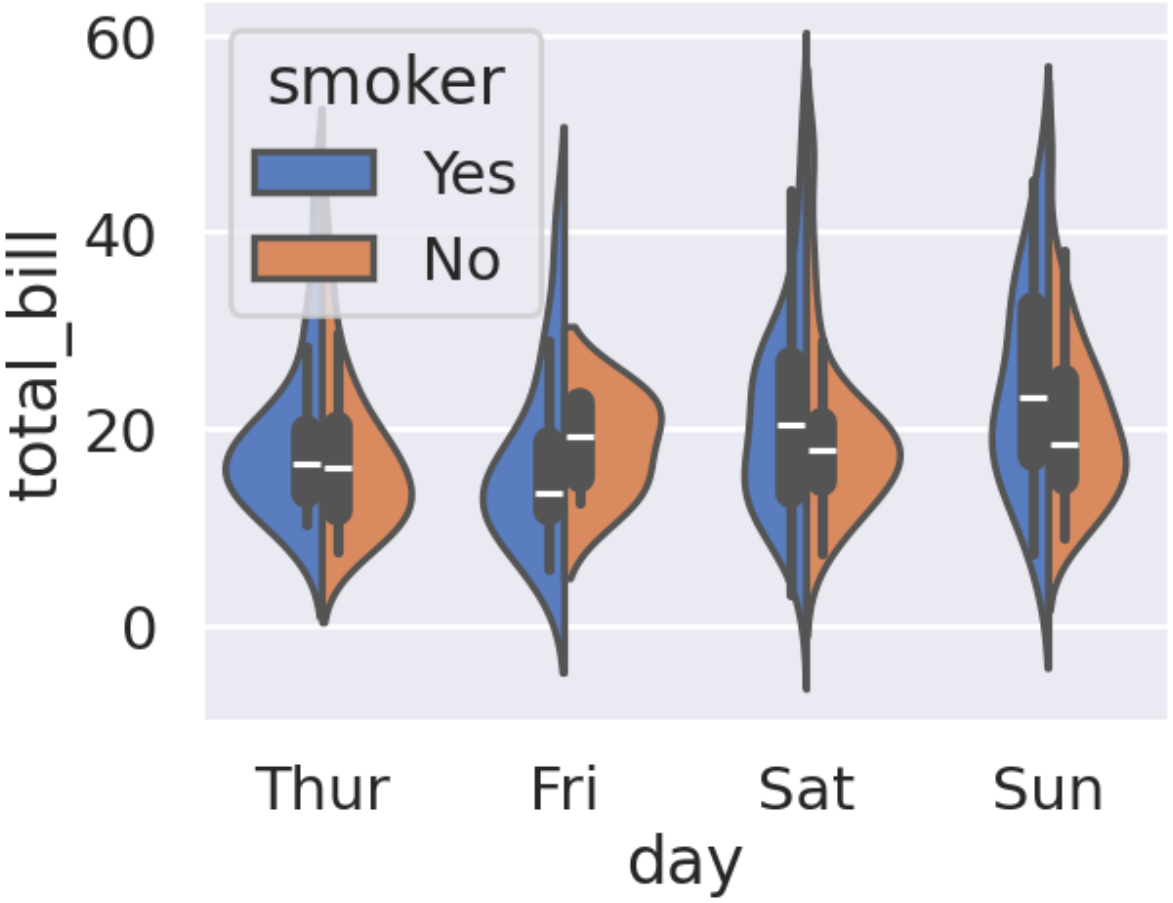
categorical_violinplot()

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

# Violinplot 생성
sns.violinplot(
    x="day",                # x축: 요일
    y="total_bill",         # y축: 총 계산 금액
    hue="smoker",           # 색상을 흡연 여부(smoker)로 구분
    data=tips,              # 데이터셋 설정
    palette="muted",        # 색상 팔레트: muted
    split=True              # 흡연 여부 데이터를 한 플롯에 분리해서 표시
)

# 그래프 출력
plt.show()
```



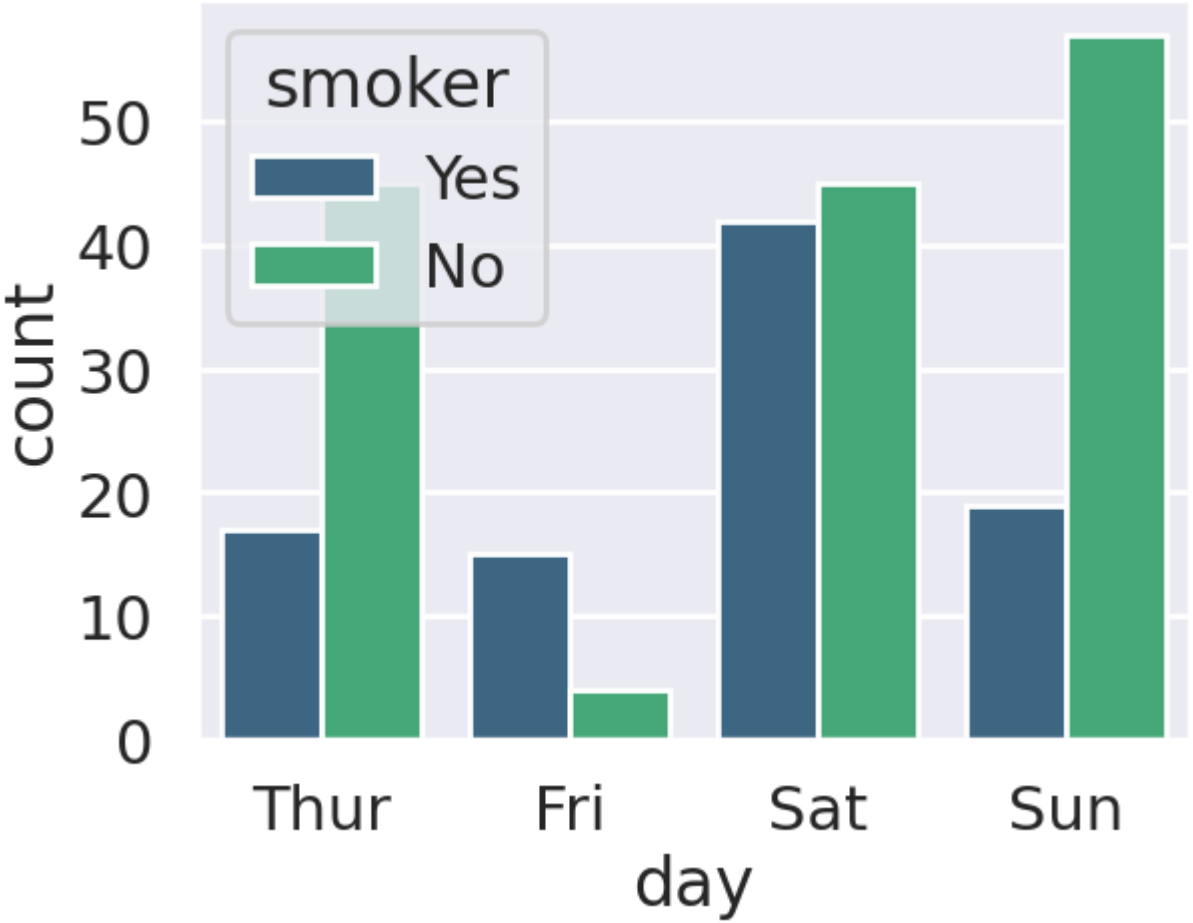
categorical_countplot()

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

# Countplot 생성
sns.countplot(
    x="day",              # x축: 요일
    hue="smoker",         # 색상을 흡연 여부(smoker)로 구분
    data=tips,            # 데이터셋 설정
    palette="viridis"     # 색상 팔레트: viridis
)

# 그래프 출력
plt.show()
```



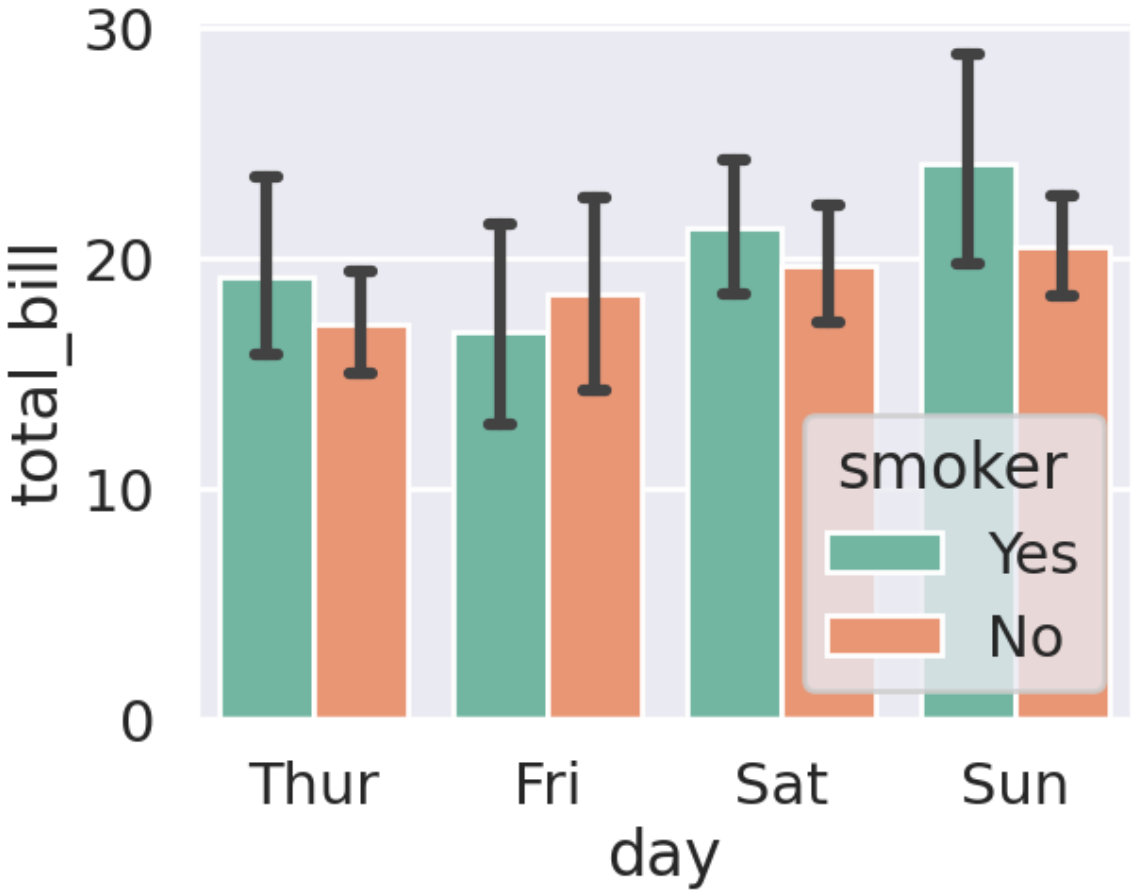
categorical_barplot()

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

# Barplot 생성
sns.barplot(
    x="day",              # x축: 요일
    y="total_bill",       # y축: 총 계산 금액
    hue="smoker",         # 색상을 흡연 여부(smoker)로 구분
    data=tips,            # 데이터셋 설정
    palette="Set2",       # 색상 팔레트: Set2
    capsize=0.2           # 막대의 끝부분 캡 크기 설정
)

# 그래프 출력
plt.show()
```



distribution_kdeplot()

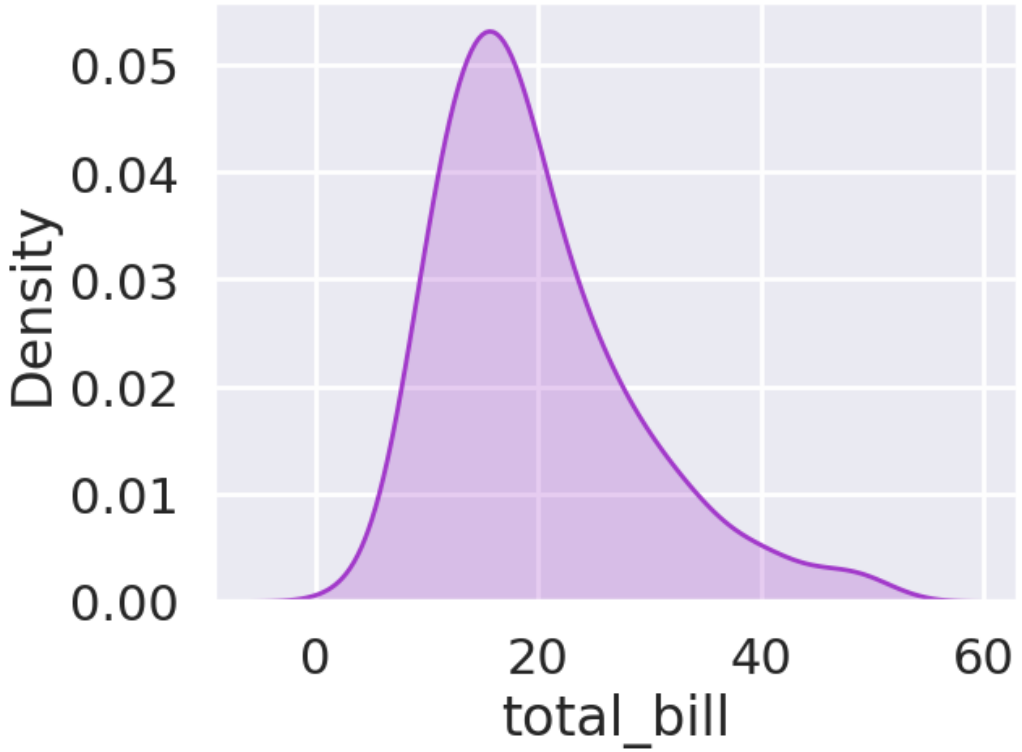
: 커널 밀도 추정(Kernel Density Estimation, KDE)을 사용하여 데이터 밀도를 표시하는 그래프
: 데이터의 분포를 시각적으로 나타내는 데 유용함.

```
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

# KDE Plot 생성
sns.kdeplot(
    tips["total_bill"],  # KDE 플롯에 사용할 데이터 (총 계산 금액)
    shade=True           # 곡선 아래를 채워서 그래프를 시각적으로 강조
)

# 그래프 출력
plt.show()
```



distribution_histplot()

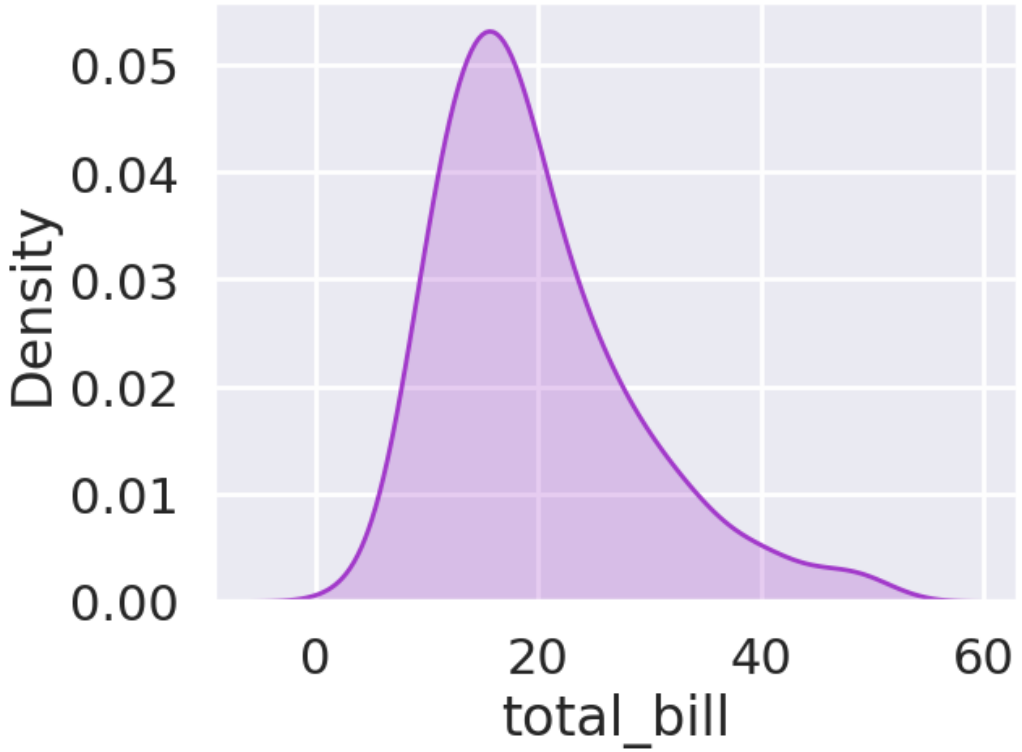
: 커널 밀도 추정(Kernel Density Estimation, KDE)을 사용하여 데이터 밀도를 표시하는 그래프
: 데이터의 분포를 시각적으로 나타내는 데 유용함.

```
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

# KDE Plot 생성
sns.kdeplot(
    tips["total_bill"],  # KDE 플롯에 사용할 데이터 (총 계산 금액)
    shade=True           # 곡선 아래를 채워서 그래프를 시각적으로 강조
)

# 그래프 출력
plt.show()
```



distribution_rugplot()

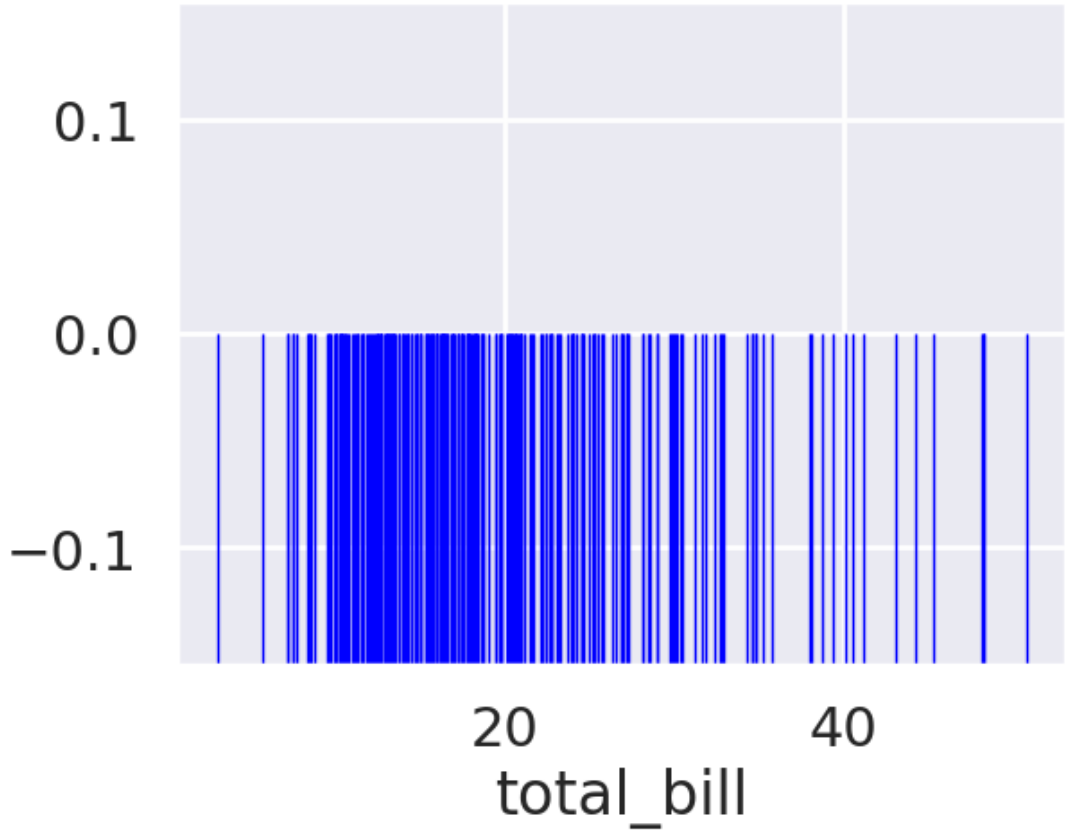
: 데이터 분포의 각 데이터 포인트를 표시하는 간단한 그래프를 생성

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

# Rug Plot 생성
sns.rugplot(
    data=tips,          # 데이터셋 설정
    x="total_bill",     # x축: 총 계산 금액
    height=0.5,         # 러그의 높이 설정
    color="blue"        # 러그의 색상 설정
)

# 그래프 출력
plt.show()
```



distribution_rugplot()

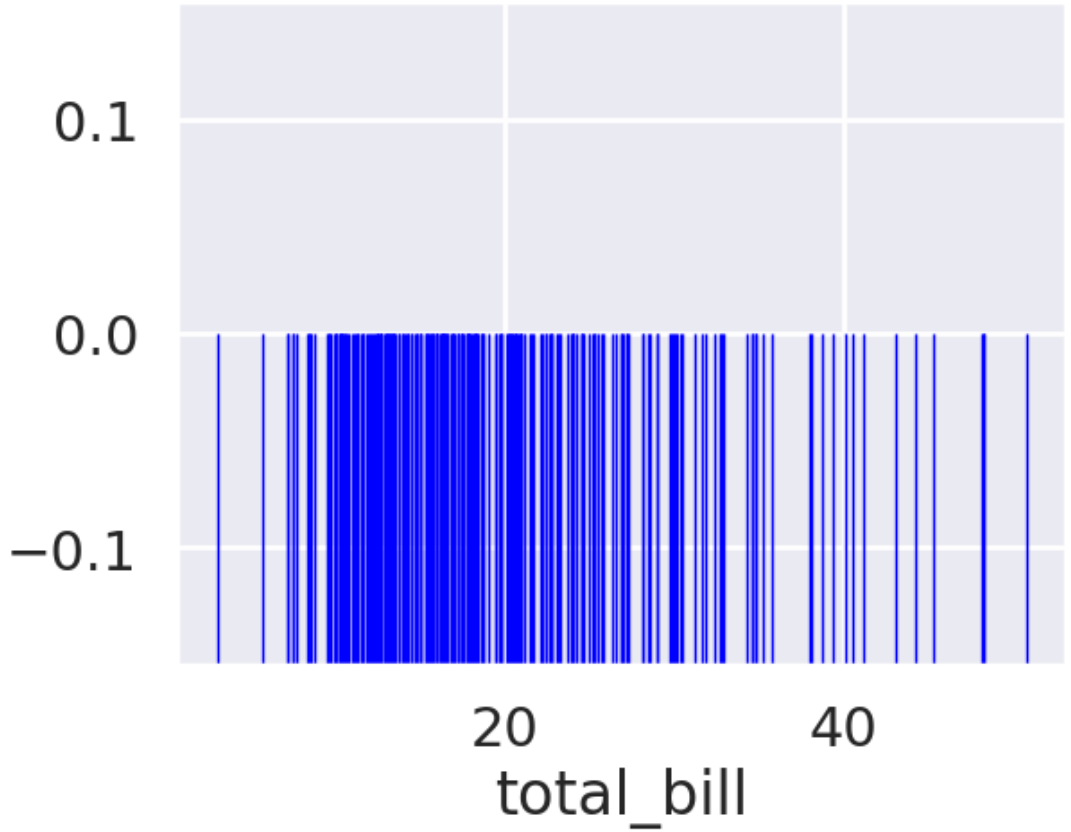
: 데이터 분포의 각 데이터 포인트를 표시하는 간단한 그래프를 생성

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

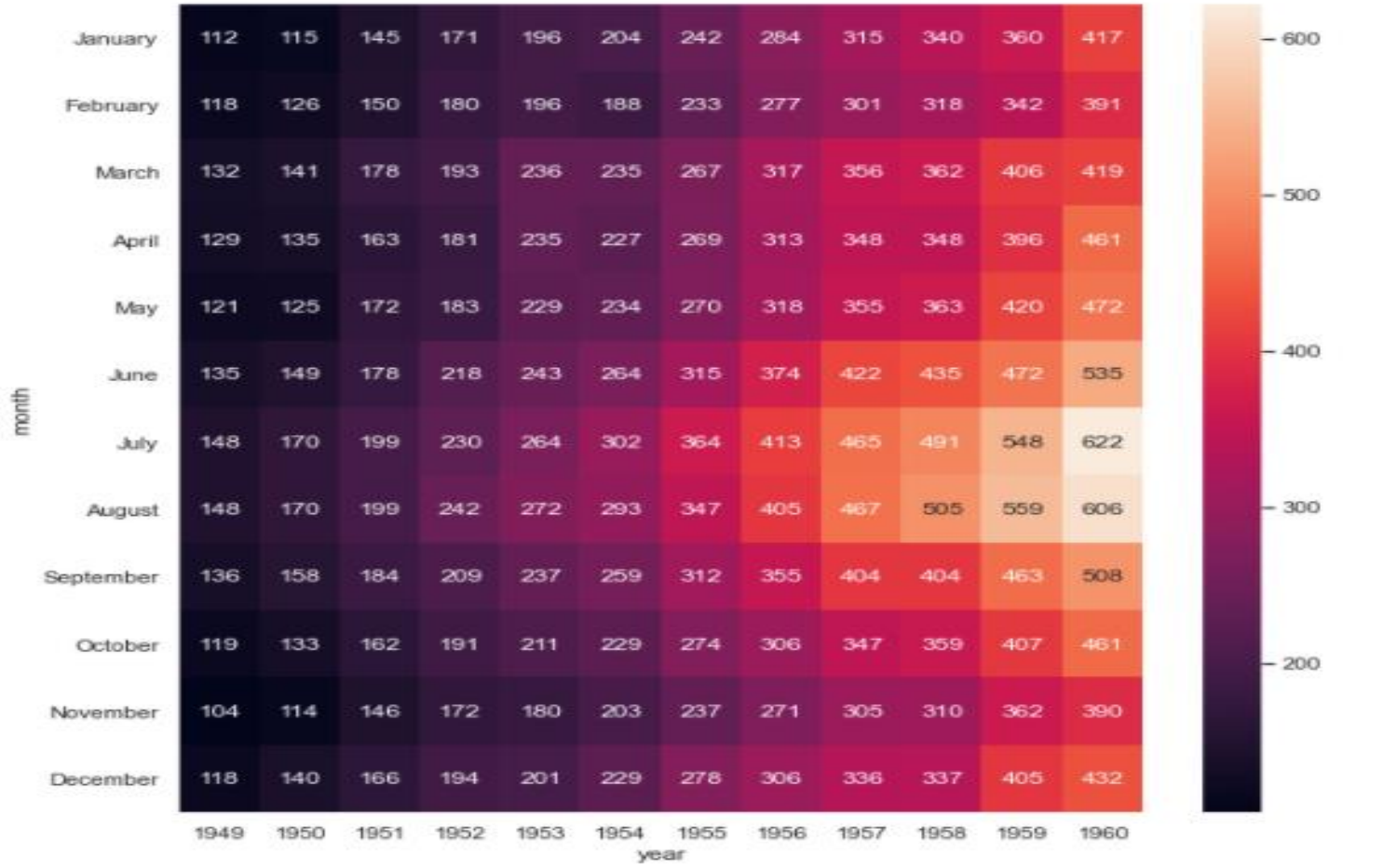
# Rug Plot 생성
sns.rugplot(
    data=tips,          # 데이터셋 설정
    x="total_bill",     # x축: 총 계산 금액
    height=0.5,         # 러그의 높이 설정
    color="blue"        # 러그의 색상 설정
)

# 그래프 출력
plt.show()
```



matrix plot_heatmap

```
import matplotlib.pyplot as plt
flights = sns.load_dataset("flights")
flights = flights.pivot("month", "year", "passengers")
plt.figure(figsize=(10, 10))
ax = sns.heatmap(flights, annot=True, fmt="d")
```



matrix plot_heatmap

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
flights = sns.load_dataset("flights")  # Seaborn의 예제 데이터셋 'flights' 불러오기

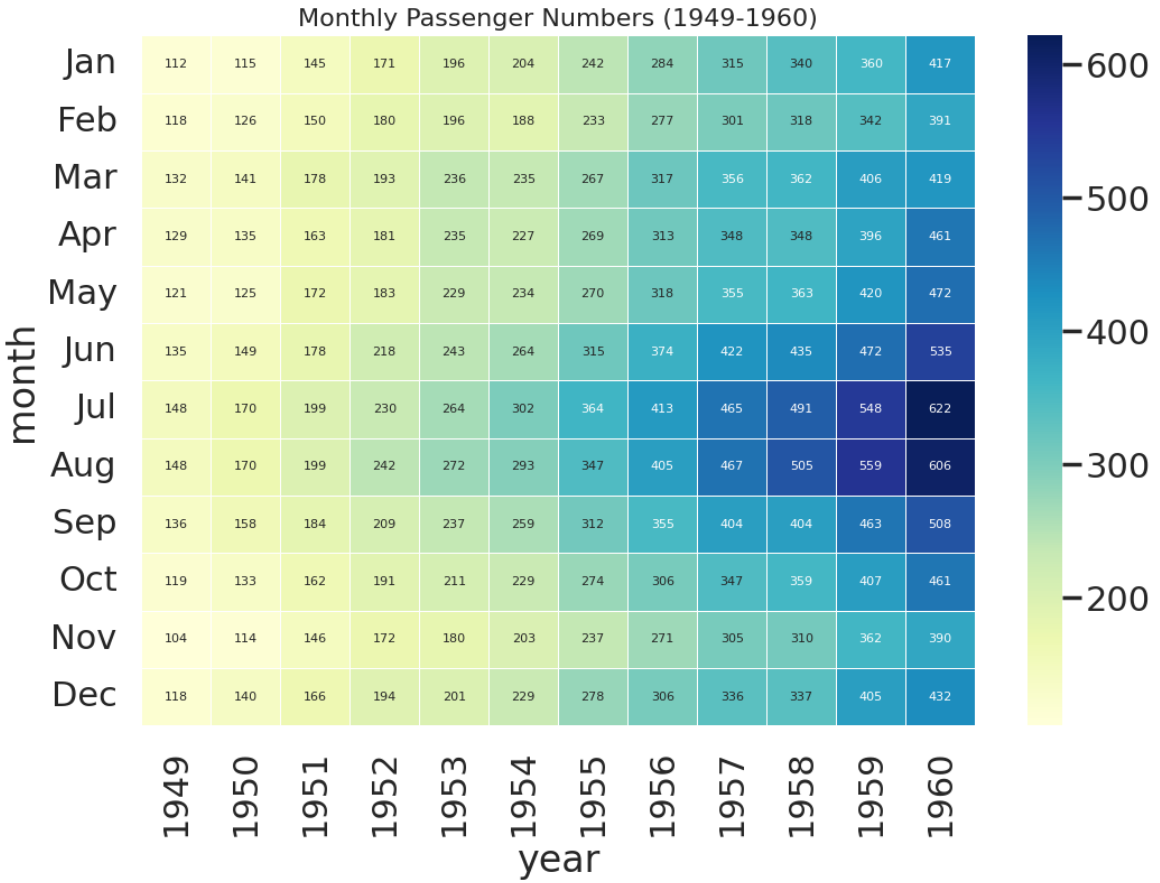
# 데이터프레임 피벗
flights = flights.pivot(index="month", columns="year", values="passengers")  # 피벗 설정

# 그래프 크기 설정
plt.figure(figsize=(12, 8))  # 가로 12인치, 세로 8인치 크기로 설정

# 히트맵 생성
sns.heatmap(
    data=flights,       # 데이터 설정
    annot=True,         # 각 셀에 값 표시
    fmt="d",            # 값 포맷: 정수 형태로 표시
    cmap="YlGnBu",      # 색상 팔레트: YlGnBu (노랑-초록-파랑)
    linewidths=0.5,     # 셀 간의 구분선 설정
    annot_kws={"size": 8}  # 셀 내 텍스트 크기 설정
)

# 그래프 제목 추가
plt.title("Monthly Passenger Numbers (1949-1960)", fontsize=16)  # 제목 및 글자 크기 설정

# 그래프 출력
plt.show()
```



Seaborn

matrix plot_clustermap

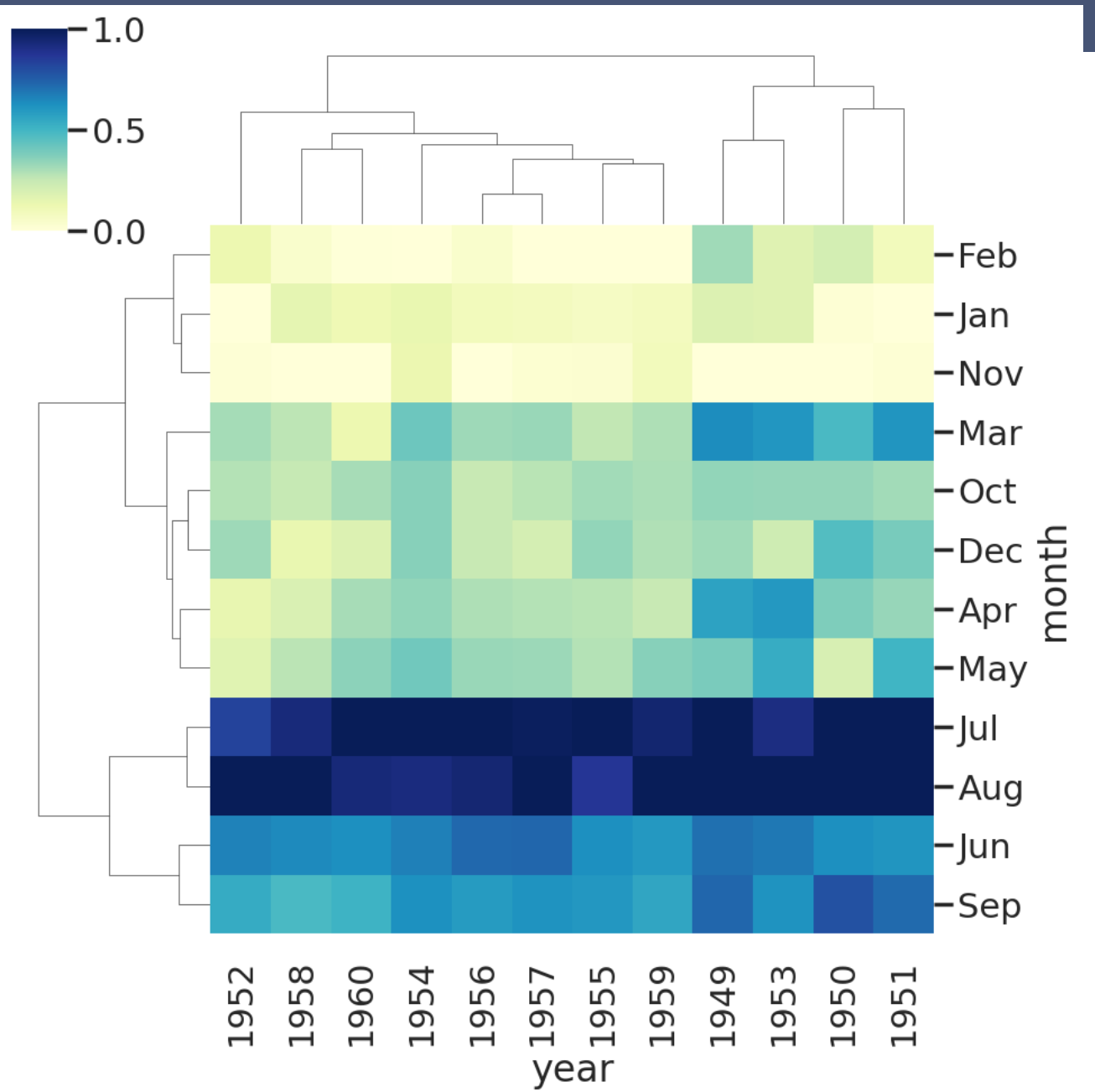
```
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
flights = sns.load_dataset("flights")  # Seaborn의 예제 데이터셋 'flights' 불러오기

# 데이터프레임 피벗
flights = flights.pivot(index="month", columns="year", values="passenger")  # 행: month, 열: year, 값: passenger

# Clustermap 생성
sns.clustermap(
    data=flights,          # 데이터 설정
    cmap="YlGnBu",         # 색상 팔레트: YlGnBu (노랑-초록-파랑)
    standard_scale=1       # 데이터 표준화 (열 방향으로 정규화)
)

# 그래프 출력
plt.show()
```



Regression plot_lmplot()

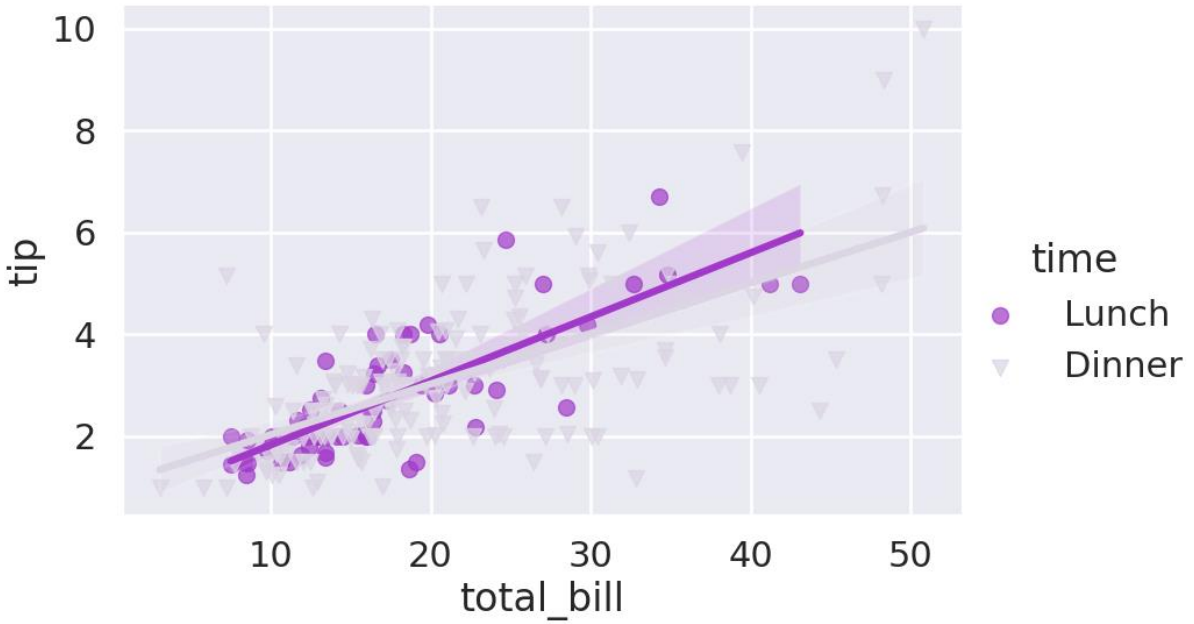
두 변수 간의 선형 관계를 시각화하는데 사용되며, 이를 위해 선형 회귀 모델의 결과를 그래프로 나타냄.

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

# lmplot 생성
sns.lmplot(
    x="total_bill",          # x축: 총 계산 금액
    y="tip",                 # y축: 팁 금액
    data=tips,               # 데이터셋 설정
    hue="time",              # 색상을 시간(time: Lunch/Dinner)으로 구분
    markers=["o", "v"],      # 점의 모양을 시간별로 다르게 설정 (Lunch: "o", Dinner: "v")
    height=6,                # 그래프의 높이 설정 (단위: 인치)
    aspect=1.5,              # 가로-세로 비율 설정
    scatter_kws={"s": 100, "alpha": 0.7}  # 점의 크기와 투명도 설정
)

# 그래프 출력
plt.show()
```



Regression plot_regplot()

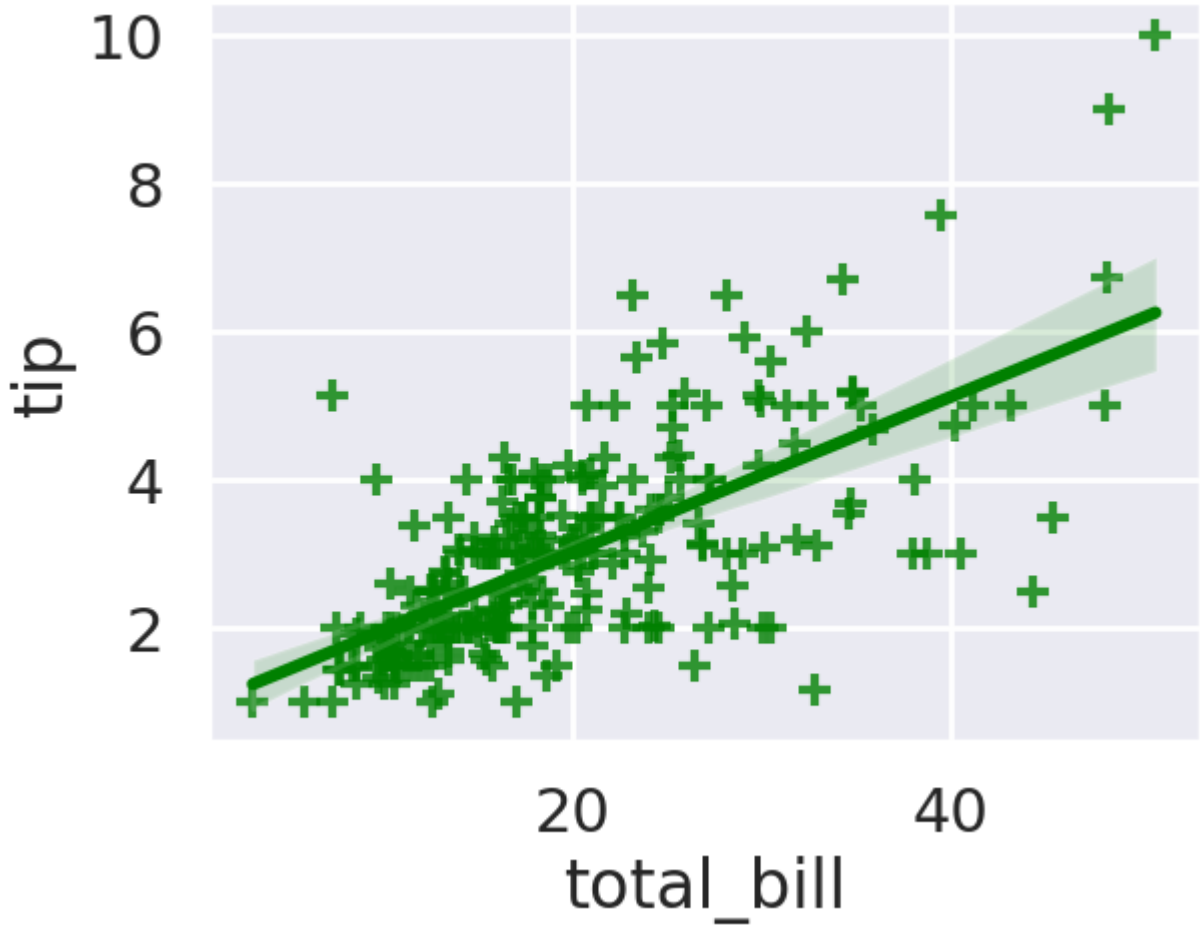
두 변수 간의 선형 관계를 시각화하는데 사용됨.

```
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

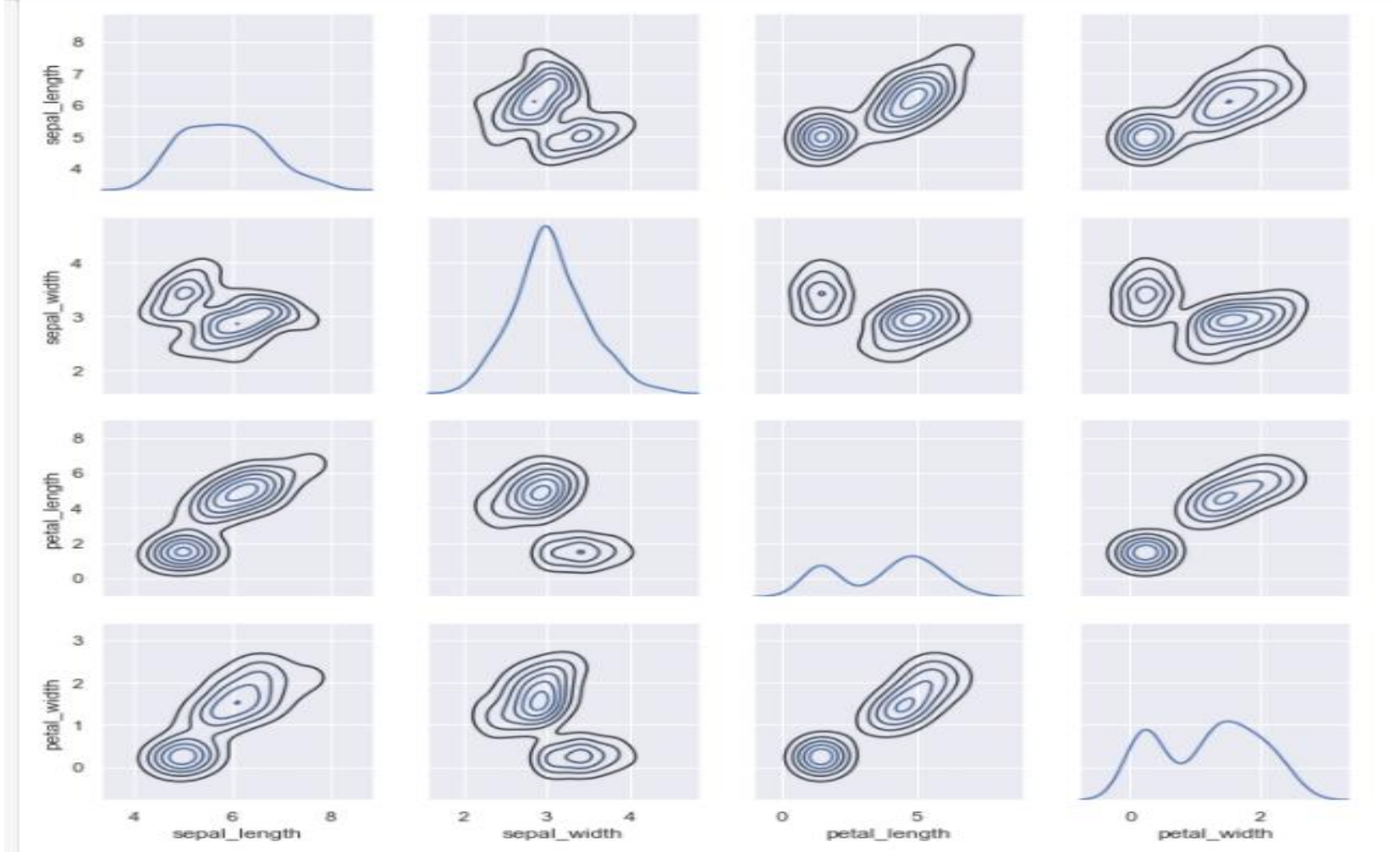
# Regplot 생성
sns.regplot(
    x="total_bill",   # x축: 총 계산 금액
    y="tip",          # y축: 팁 금액
    data=tips,        # 데이터셋 설정
    color="green",    # 선과 점의 색상 설정
    marker="+"        # 점의 모양을 '+'로 설정
)

# 그래프 출력
plt.show()
```



Seaborn_pairplot

```
g = sns.PairGrid(iris)
g.map_diag(sns.kdeplot)
g.map_offdiag(sns.kdeplot, n_levels=6);
```



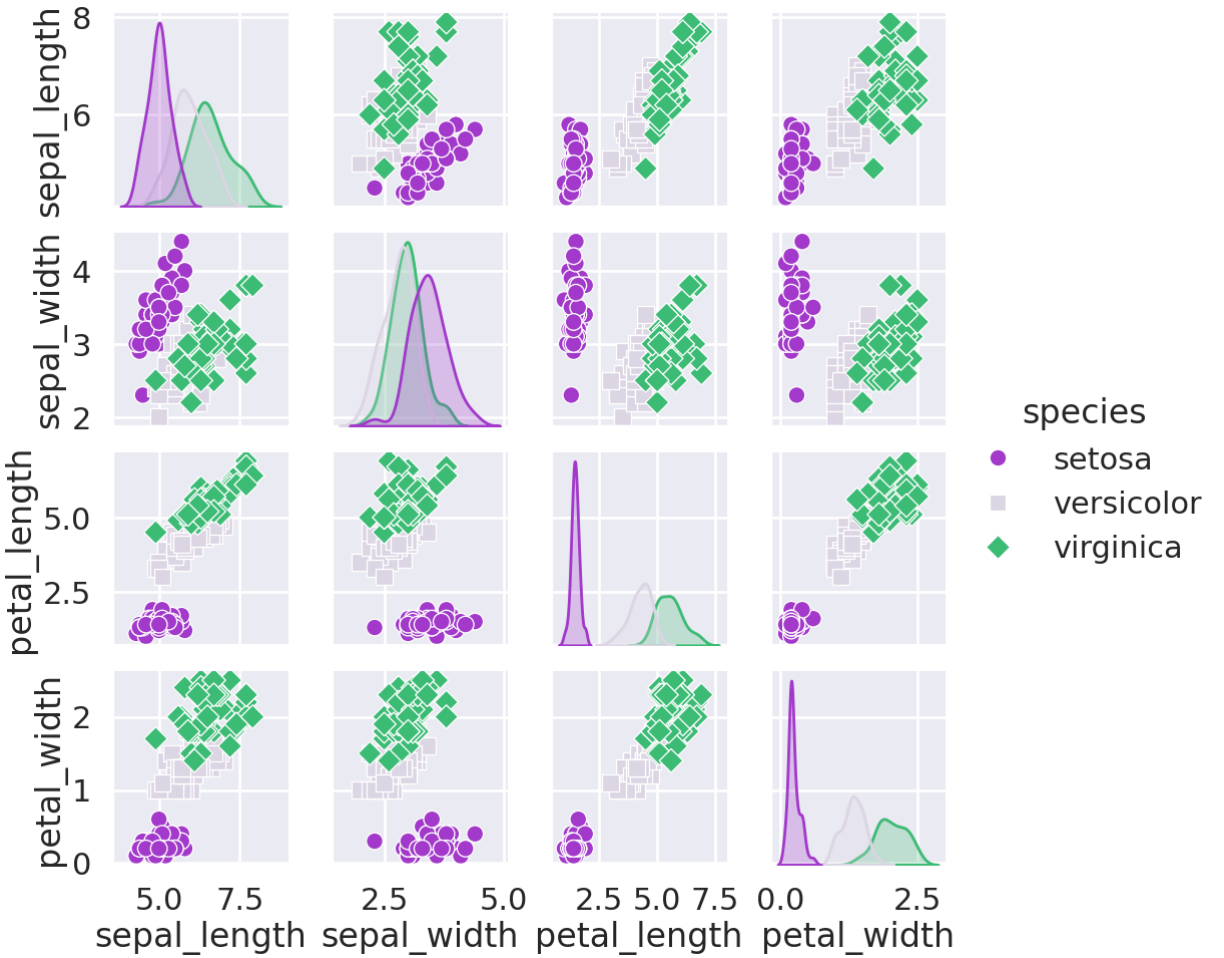
multi-plot grid_pairplot

```
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
iris = sns.load_dataset("iris")  # Seaborn의 예제 데이터셋 'iris' 불러오기

# Pairplot 생성
sns.pairplot(
    data=iris,          # 데이터셋 설정
    hue="species",      # 색상을 종(species)별로 구분
    markers=["o", "s", "D"]  # 종별로 점 모양을 다르게 설정
)

# 그래프 출력
plt.show()
```



multi-plot grid_FacetGrid()

여러 서브플롯을 생성하고 각각 데이터를 시각화 할수 있음.

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

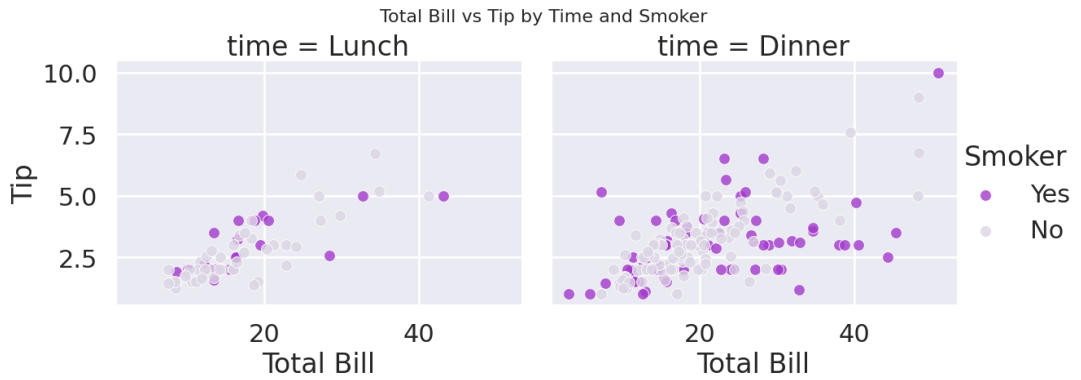
# FacetGrid 생성 (그래프 크기 조정 포함)
g = sns.FacetGrid(tips, col="time", hue="smoker", height=5, aspect=1.2)  # 크기(height)와 비율(aspect) 조정

# 각 Facet에 산점도 매핑 (점 크기와 투명도 설정)
g.map(sns.scatterplot, "total_bill", "tip", s=100, alpha=0.8)  # 점 크기(s)와 투명도(alpha) 조정

# 범례 추가
g.add_legend(title="Smoker")  # 범례 제목 추가

# 축 제목과 전체 제목 설정
g.set_axis_labels("Total Bill", "Tip")  # x축과 y축 라벨 추가
g.fig.subplots_adjust(top=0.85)  # 제목을 위해 위쪽 간격 조정
g.fig.suptitle("Total Bill vs Tip by Time and Smoker", fontsize=16)  # 전체 제목 추가

# 그래프 출력
plt.show()
```



Jointplot_jointplot()

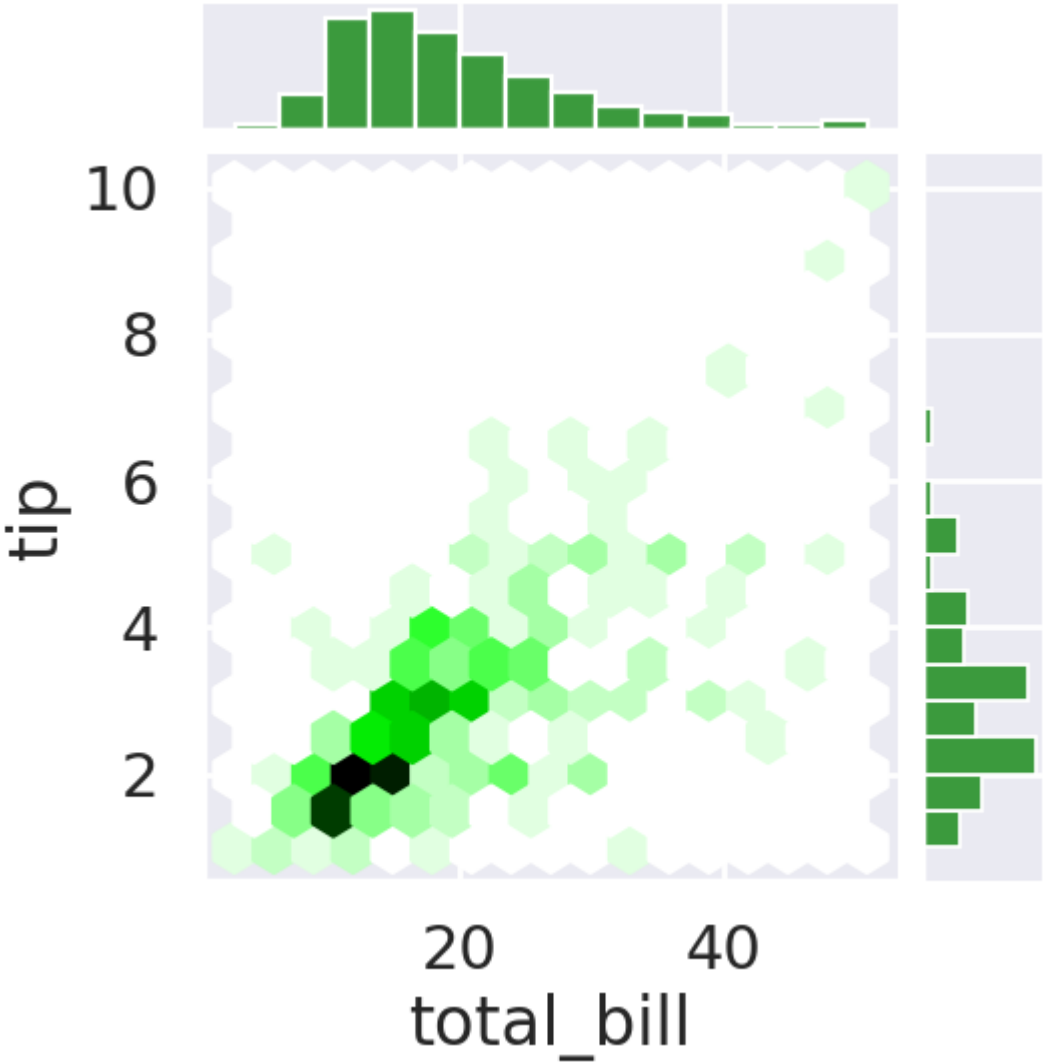
두 변수 사이의 이변량(또는 조인트) 관계와 각 변수의 일변량(또는 마진) 분포를 동시에 표시하는 그래프를 생성

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터셋 로드
tips = sns.load_dataset("tips")  # Seaborn의 예제 데이터셋 'tips' 불러오기

# Joint Plot 생성
sns.jointplot(
    data=tips,          # 데이터셋 설정
    x="total_bill",     # x축: 총 계산 금액
    y="tip",            # y축: 팁 금액
    kind="hex",         # 플롯 유형: 육각형 빈 플롯
    color="g"           # 색상: 녹색
)

# 그래프 출력
plt.show()
```



Seaborn 그래프 스타일

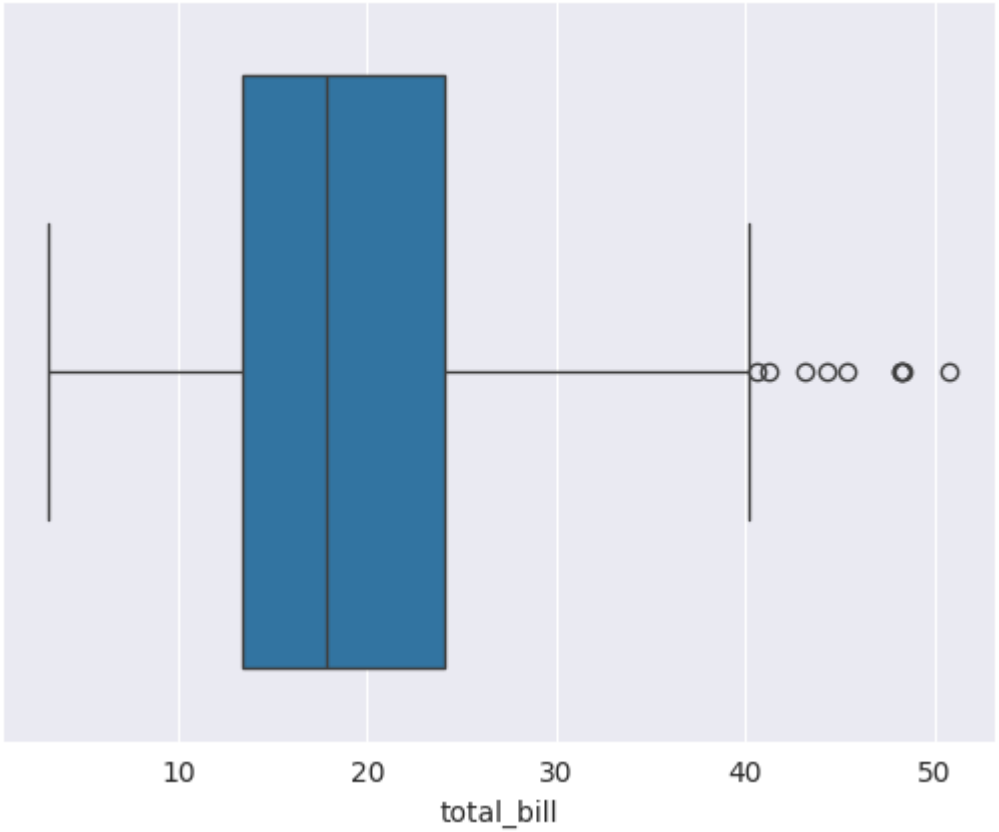
set_style을 통해 스타일 테마를 설정할 수 있음. 스타일 테마 : Darkgrid, Whitegrid, Dark, White가 있음.

```
import seaborn as sns
import matplotlib.pyplot as plt

# 테마 설정
sns.set_style("darkgrid")  # 그래프의 배경 스타일을 'darkgrid'로 설정

# 임의의 데이터를 이용해 그래프 그리기
tips = sns.load_dataset("tips")  # Seaborn의 내장 데이터셋 'tips' 불러오기
sns.boxplot(x=tips["total_bill"])  # 'total_bill' 열의 데이터로 박스플롯 그리기

# 그래프 출력
plt.show()
```



Seaborn 그래프 스타일

set_context() 함수는 그래프의 스케일(scale)을 설정하는데 사용

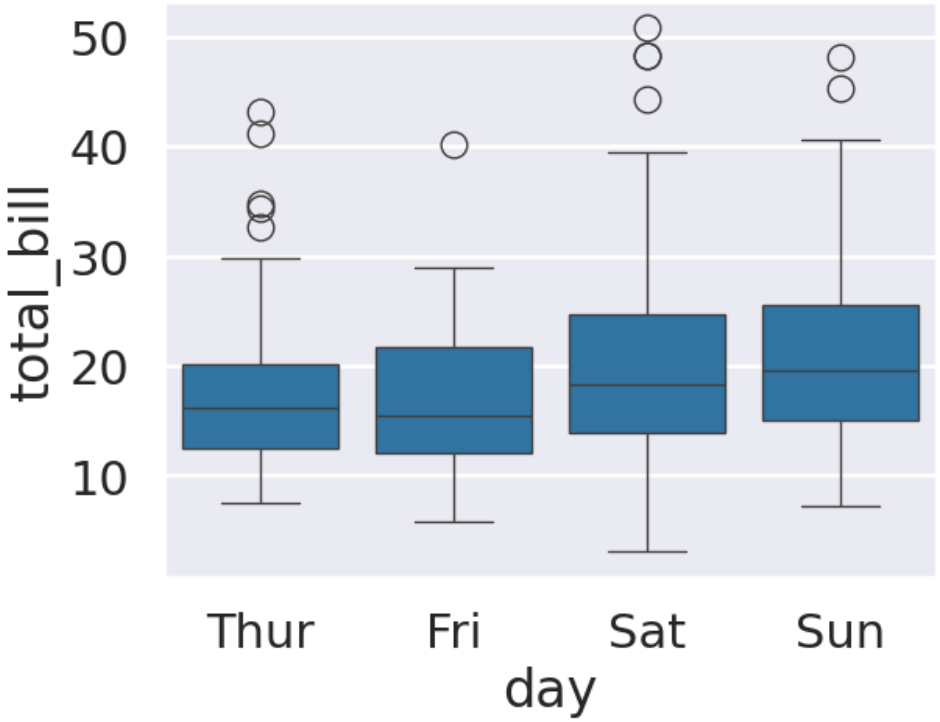
- paper : 가장 작은 스케일로 출판물용으로 적합
- notebook : 주피터 노트북 등에서 그래프를 그릴 때 일반적으로 사용
- talk : 큰 그림을 필요로 하는 강연이나 프레젠테이션에 적합.
- poster : 큰 포스터와 같이 매우 큰 그림에 적합.

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 컨텍스트 설정
sns.set_context('poster')  # 그래프의 요소 크기와 스타일을 'poster' 크기에 맞게 조정

# 이후에 그려지는 그래프에 설정한 컨텍스트가 적용됩니다.
sns.boxplot(
    data=sns.load_dataset('tips'),  # Seaborn의 내장 데이터셋 'tips'를 불러옴
    x='day',                        # x축: 요일 (day)
    y='total_bill'                  # y축: 총 계산 금액 (total_bill)
)

# 그래프 출력
plt.show()
```



Seaborn 그래프 스타일

- set_palette() 함수를 사용해 색상 팔레트를 설정할 수 있음.
- deep, muted, bright, pastel, dartk, colorblind와 같은 다양한 색상 팔레트가 있음.
- seaborn은 색상의 구성 요소를 명시적으로 지정하여 커스텀 색상 팔레트를 생성하는 것을 가능하게 함.

```python
import seaborn as sns
import matplotlib.pyplot as plt

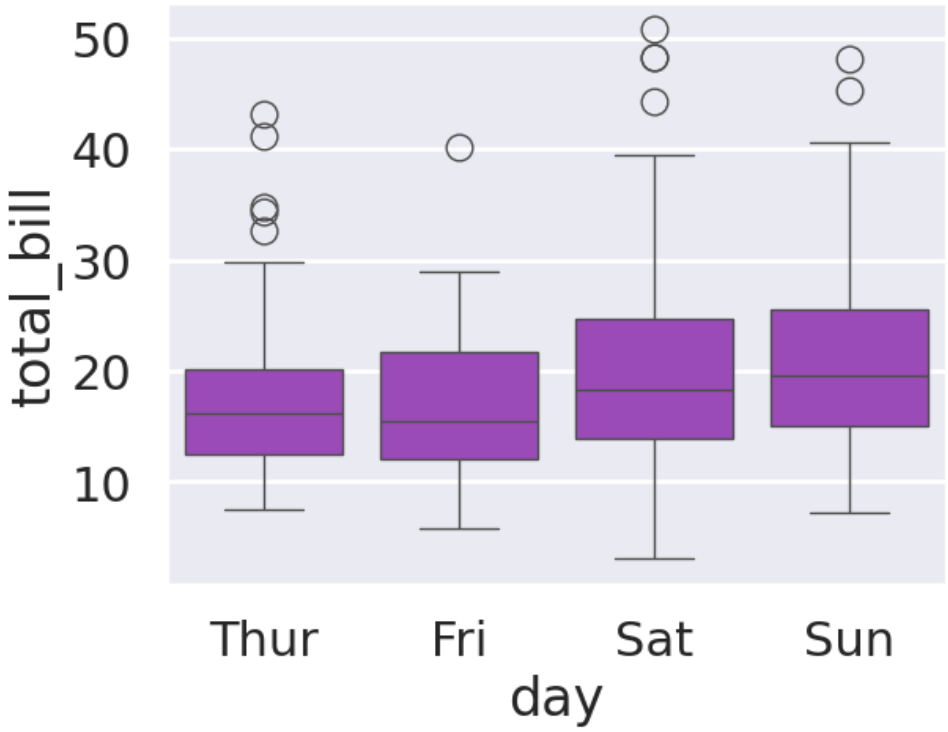
# 컨텍스트 설정
sns.set_context('poster')  # 그래프의 요소 크기를 'poster' 스타일로 설정 (크게 표시)

# Seaborn 팔레트 사용 (예시)
# sns.set_palette(sns.color_palette("husl"))  # 기본 팔레트를 HUSL 색상 팔레트로 설정

# 커스텀 팔레트 사용
custom_palette = ['#A239CA', '#DCD5E4', '#3CBB75', '#D4752E']  # 사용자 정의 색상 팔레트
sns.set_palette(custom_palette)  # 커스텀 색상 팔레트를 설정

# 그래프 생성 및 표시
sns.boxplot(
    data=sns.load_dataset('tips'),  # Seaborn의 'tips' 데이터셋 불러오기
    x='day',                        # x축에 요일(day) 설정
    y='total_bill'                  # y축에 총 계산 금액(total_bill) 설정
)

# 그래프 출력
plt.show()
```



plotnine

plotnine은 R의 ggplot2에 기반해 그래프를 그려주는 라이브러리입니다

```
!pip install plotnine
```

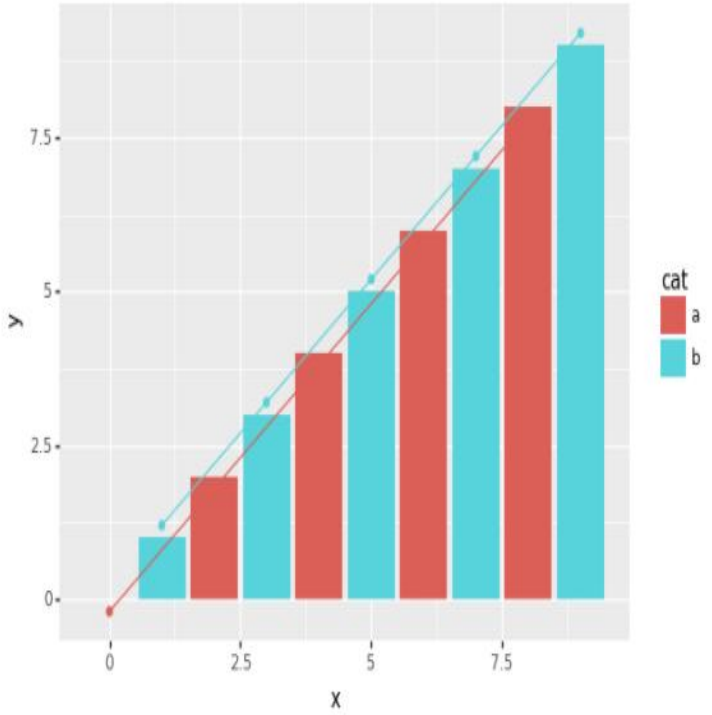
```
!pip3 install plotnine
```

```
import plotnine
import pandas as pd
import numpy as np
from plotnine import *
print("plontnine version :",plotnine.__version__)
```

```
plontnine version : 0.7.1
```

```
n = 10
df = pd.DataFrame({'x': np.arange(n),
                   'y': np.arange(n),
                   'yfit': np.arange(n) + np.tile([-.2, .2], n//2),
                   'cat': ['a', 'b']*(n//2)})
```

```
(ggplot(df)
 + geom_col(aes('x', 'y', fill='cat'))
 + geom_point(aes('x', y='yfit', color='cat'))
 + geom_path(aes('x', y='yfit', color='cat'))
)
```



Folium

- Folium은 지도 데이터(Open Street Map)에 leaflet.js를 이용해 위치 정보를 시각화하는 라이브러리
- 자바스크립트 기반이라 interactive하게 그래프를 그릴 수 있음.
- 그 외에도 pydeck, ipyleaflet 등으로 지도 시각화를 할 수 있습니다

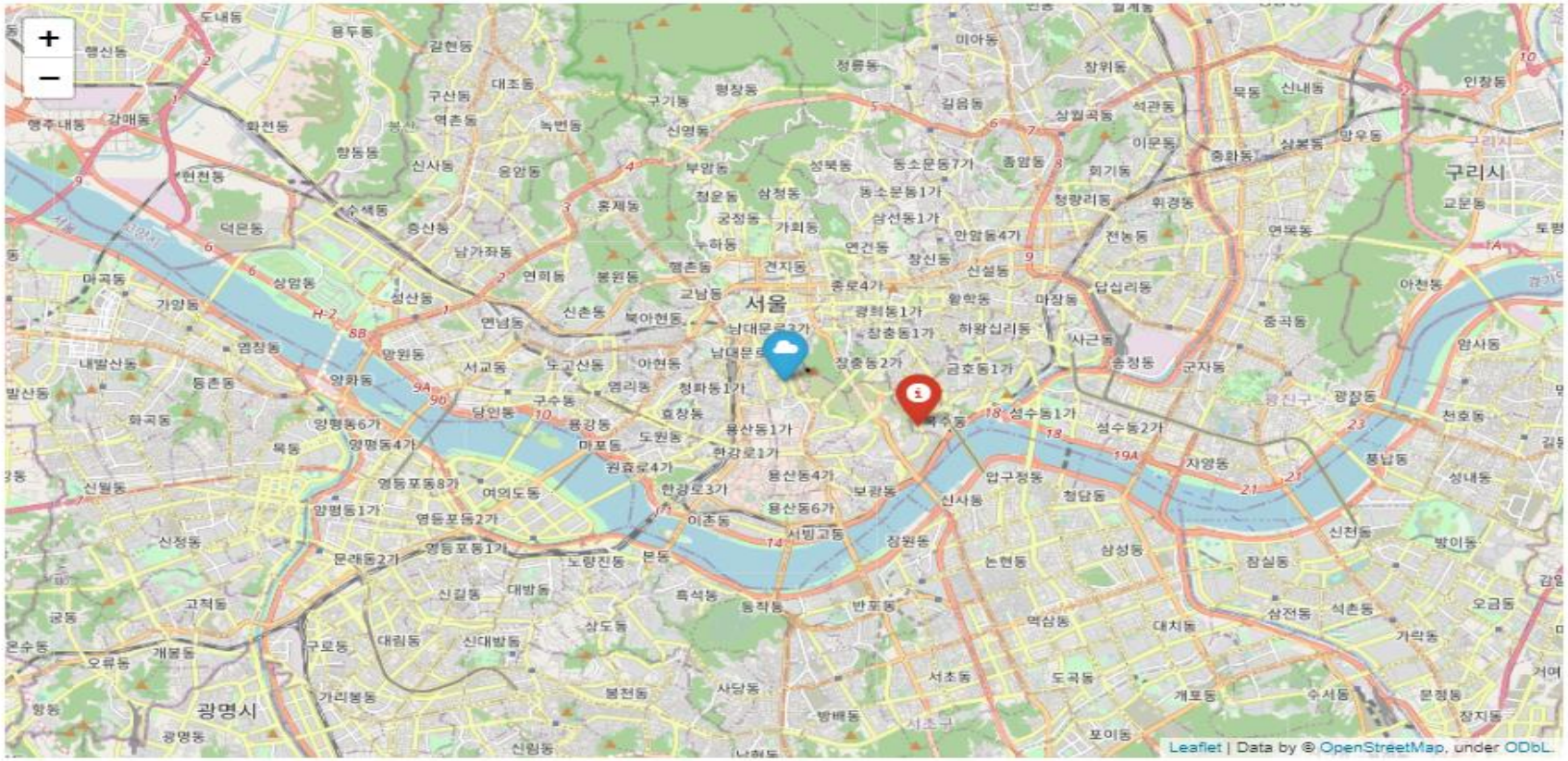
```
!pip install folium
```

참고 : 공식문서(http://python-visualization.github.io/folium/)
: 서울지역 5대 강력범죄 데이터 분석(https://github.com/HyunSu-Jin/seoul_crime/blob/master/seoul_crime.ipynb)

Folium

```
import folium

m = folium.Map(location=[37.5502, 126.982], zoom_start=12)
folium.Marker(location=[37.5502, 126.982], popup="Marker A",
              icon=folium.Icon(icon='cloud')).add_to(m)
folium.Marker(location=[37.5411, 127.0107], popup="한남동",
              icon=folium.Icon(color='red')).add_to(m)
m
```



Plot.ly

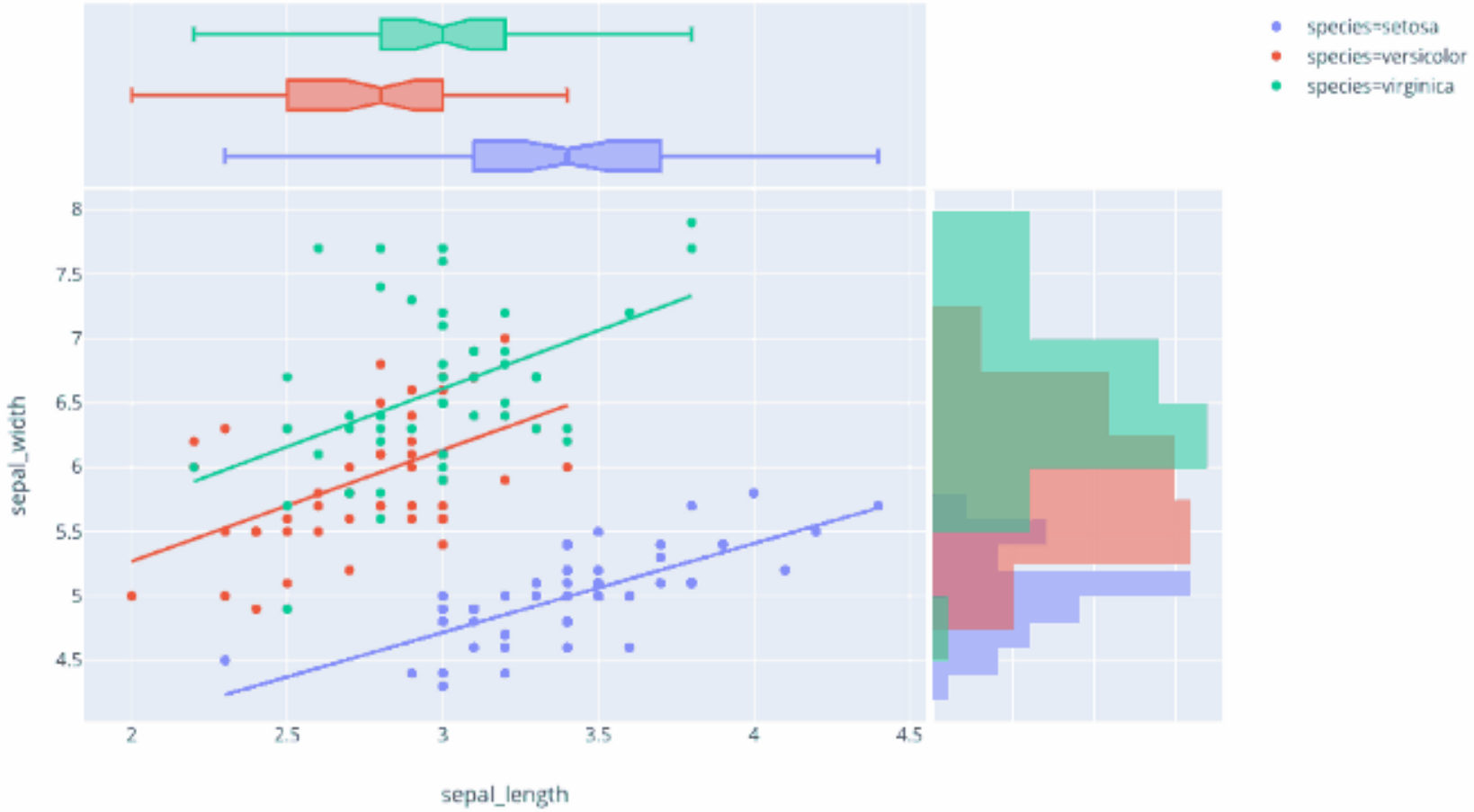
- Plotly는 Interactive 그래프를 그려주는 라이브러리
- Scala , R , Python, JavaScript, MATLAB 등에서 사용할 수 있음.
- 시각화를 위해 D3.js를 사용하고 있음
- 사용해보면 사용이 쉽고, 세련된 느낌을 받음
- Plotly cloud라는 유료 모델이 있음.

```
!pip install plotly
```

참고 : https://plotly.com/python/ipython-notebook-tutorial/

plotly

```
px.scatter(iris, x="sepal_width", y="sepal_length", color="species", marginal_y="histogram",
           marginal_x="box", trendline="ols")
```

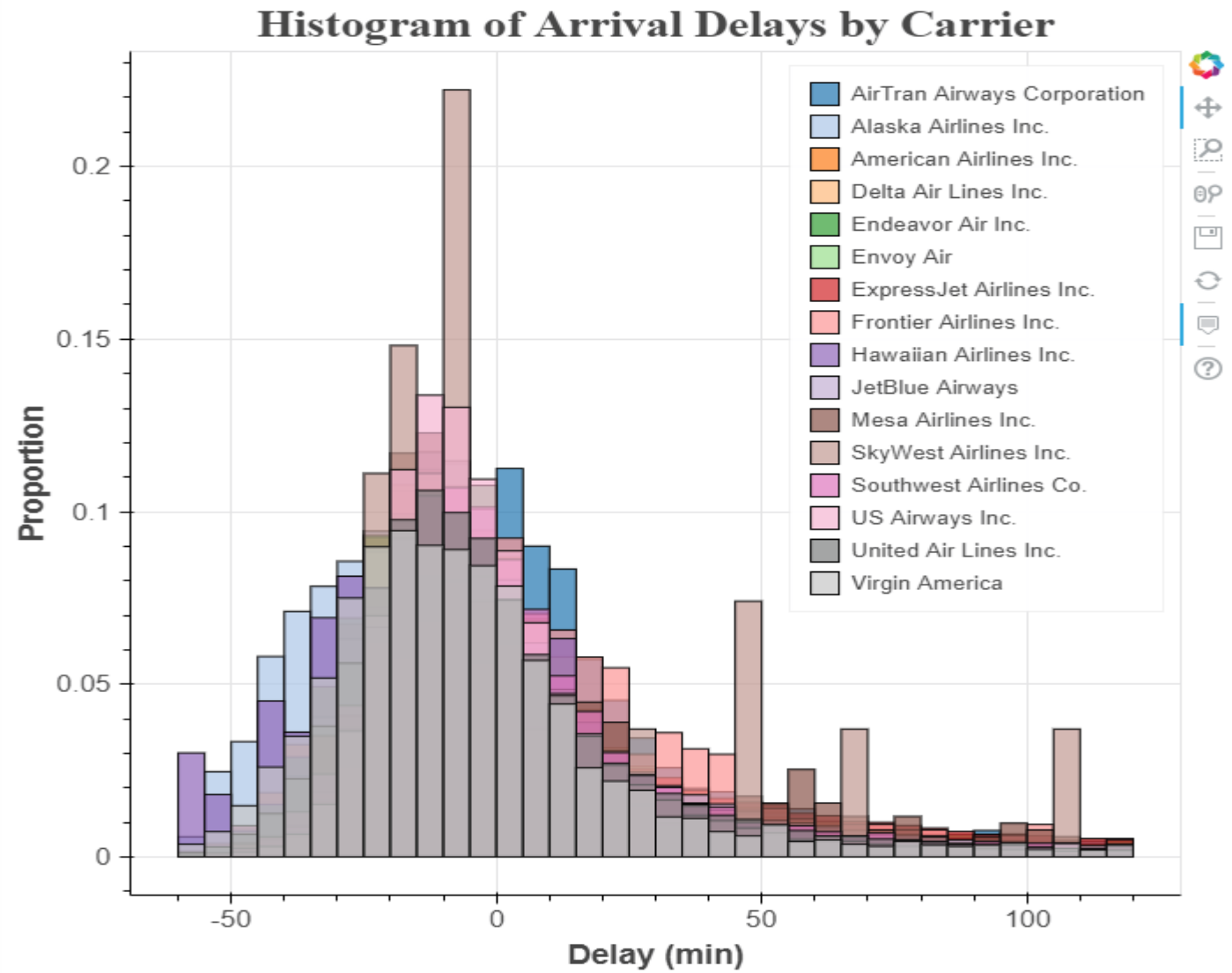


참고 : https://towardsdatascience.com/the-sweet-spot-of-plotly-c6e1342e3544

bokeh

- bokeh는 Interactive 그래프를 그려주는 라이브러리
- 독립형 HTML 문서 또는 서버 운영할 수 있도록 지원
- 표현력이 뛰어나고 다양한 그래픽 지원
- 사용해보면 사용이 쉽고, 세련된 느낌을 받음

bokeh



출처 : https://www.pinterest.co.kr/pin/823595850584848711/

Thank you.

Matplotlib
ryp1662@gmail.com

Copyright ⓒ “Youngpyo Ryu” All Rights Reserved.
This document was created for the exclusive use of “Youngpyo Ryu”.
It must not be passed on to third parties except with the explicit prior consent of “Youngpyo Ryu”.